

République Tunisienne

Établissement universitaire à compléter

Rapport de projet de fin d'études

Conception et développement d'une plateforme de
recrutement augmentée par l'IA

Capgemini Talent Intelligence

Réalisé par :	<i>À compléter</i>
Entreprise d'accueil :	Capgemini / Capgemini Engineering
Encadrant académique :	<i>À compléter</i>
Encadrant professionnel :	<i>À compléter</i>
Année universitaire :	2025-2026

Abstract

This report presents the design and implementation of Capgemini Talent Intelligence, an enterprise recruitment platform built to support the full hiring lifecycle through role-based workflows and AI-assisted decision support. The system covers public landing and authentication flows, CV pool ingestion, structured job management, candidate pipeline tracking with auditable stage history, interview scheduling and reporting, administration, analytics, notifications, governed AI action confirmation, and onboarding follow-up.

From a technical perspective, the platform is implemented with Next.js 16, TypeScript, Bun, Better Auth, Neon PostgreSQL, Drizzle ORM, Tailwind CSS, and shadcn/ui. Its internal organization follows a feature-driven architecture in which business logic is concentrated in service modules, while routes remain thin and role-oriented. This structure enables the application to scale functionally without fragmenting the codebase into disconnected subsystems.

The report gives particular attention to the AI layer, which includes CV data extraction, screening generation, interview guide generation, a tool-enabled conversational agent, explicit confirmation for mutating agent actions, and a hybrid Retrieval-Augmented Generation pipeline combining query rewrite, vector retrieval, lexical retrieval, fusion, re-ranking, and cited context assembly. The conversational layer itself now follows a hybrid UX pattern : a floating contextual assistant remains available inside business dashboards, while a dedicated `/agent` workspace supports deeper analysis with source-backed answers, evidence traces, action confirmations, and deep links back to CVs, candidates, jobs, and governance views. The project evaluation protocol also compares the baseline retrieval flow with a second-phase RAG pipeline. On 40 queries with 85% ground-truth coverage, Phase 2 improves Precision@5 from 4.0% to 5.5%, Precision@10 from 2.0% to 5.5%, MRR@10 from 0.175 to 0.200, and NDCG@10 from 0.042 to 0.071, while maintaining a 0% error rate. The final agent experience also includes structured visual restitution : analytics questions can produce chart cards generated from verified tool outputs, while candidate, pipeline, and governance answers can produce typed response cards with metrics, caveats, deep links, and follow-up actions. The execution trace is also grouped into phases such as retrieval, analysis, confirmation, execution, and verification, making the agent's work easier to audit than a flat technical log.

Overall, the project demonstrates how AI can be integrated into recruitment operations in a controlled, measurable, and business-grounded way. Rather than relying on opaque automation, the platform combines explicit workflows, immutable stage records,

pending action confirmations, role-based governance, structured persistence, and verifiable evaluation signals.

Résumé

Ce mémoire présente la conception et le développement de Capgemini Talent Intelligence, une plateforme de recrutement orientée entreprise couvrant l'ensemble du cycle de recrutement : authentification, vivier de CV, gestion des offres, affectation des candidats, historique auditable des transitions, entretiens, analytique, administration, gouvernance des actions IA et onboarding. L'application repose sur une architecture *feature-driven* construite avec Next.js 16, TypeScript, Bun, Better Auth, Neon PostgreSQL et Drizzle ORM.

L'apport majeur du projet réside dans sa couche d'intelligence artificielle, qui intervient à plusieurs niveaux : extraction des données CV, matching, screening, génération de guides d'entretien, agent conversationnel multi-outils, confirmation explicite des actions agentiques modifiant les données et pipeline RAG hybride avec citations. Cette couche conversationnelle suit désormais un modèle hybride, avec un assistant flottant contextuel et un workspace dédié **/agent** capable d'afficher des réponses sourcées, des traces d'exécution, des demandes de confirmation et des liens profonds vers les écrans métier. Les résultats d'évaluation réalisés dans le cadre du projet montrent une amélioration mesurable de la pertinence de recherche sur 40 requêtes, avec une couverture de vérité terrain de 85%. L'expérience agentique finale ajoute aussi une restitution visuelle structurée : lorsqu'une question porte sur une tendance, un pipeline, des compétences, la demande de postes ou un écart demande-offre, la réponse peut afficher des cartes graphiques construites à partir des mêmes sorties d'outils vérifiées. Les réponses candidat, pipeline et gouvernance peuvent également produire des cartes typées avec métriques, limites, liens profonds et actions de suivi, tandis que la trace d'exécution est regroupée par phases afin de rendre le travail de l'agent plus lisible et plus auditable.

Le projet démontre qu'une IA utile au recrutement doit rester ancrée dans des données structurées, des workflows explicites, des historiques immuables et des garde-fous techniques. La plateforme obtenue constitue ainsi une base crédible pour une digitalisation avancée et gouvernée des opérations de recrutement.

Table des matières

Abstract	i
Résumé	iii
Mots-clés et acronymes	xii
Mots-clés	xii
Acronymes	xii
Introduction générale	1
1 Présentation générale	3
Introduction	3
1.1 Présentation de l'entreprise d'accueil	3
1.1.1 Présentation de Capgemini et de Capgemini Engineering	3
1.1.2 Structure organisationnelle pertinente pour le projet	3
1.1.3 Services et périmètre numérique concernés	4
1.2 Présentation du projet	4
1.2.1 Problématique	4
1.2.2 Étude synthétique des solutions existantes	5
1.2.3 État de l'art scientifique et positionnement	6
1.2.4 Objectifs du projet	7
1.2.5 Solution proposée	8
1.3 Méthodologie adoptée	8
1.3.1 Choix de la méthodologie	8
1.3.2 Comparaison entre approches de gestion de projet	9
1.3.3 Méthodologie retenue	9
Conclusion	10
2 Planification et architecture	11
Introduction	11
2.1 Analyse des besoins et spécification	11
2.1.1 Identification des acteurs	11
2.1.2 Exigences fonctionnelles	11
2.1.3 Exigences non fonctionnelles	12

2.2	Étude analytique	13
2.2.1	Diagramme global des cas d'utilisation	13
2.2.2	Processus métier cible	13
2.2.3	Diagramme de classes global	14
2.2.4	Vue d'ensemble du Product Backlog	15
2.2.5	Découpage en sprints et objectifs	17
2.3	Architecture du projet	17
2.3.1	Architecture externe	17
2.3.2	Architecture applicative globale	18
2.3.3	Architecture interne	20
2.3.4	Architecture du sous-système IA	20
2.4	Environnement de développement et technique	22
2.4.1	Outils de gestion et de documentation	22
2.4.2	Environnement d'exécution et de développement	23
2.4.3	Outils front-end	23
2.4.4	Outils back-end	23
2.4.5	Architecture de la base de données	24
2.4.6	Justification synthétique des choix technologiques	25
	Conclusion	25
3	Module 1 : accès, identité et points d'entrée par rôle	26
	Introduction	26
3.1	Sprint 1 : objectif et périmètre	26
3.1.1	Objectif du sprint	26
3.1.2	Backlog du sprint	27
3.2	Implémentation du sprint	28
3.2.1	Analyse du sprint	28
3.2.2	Vue fonctionnelle globale	28
3.2.3	Authentification et redirection par rôle	29
3.2.4	Protection du shell et contrôle d'accès	29
3.2.5	Administration des comptes	30
3.3	Réalisation	30
3.3.1	Landing page publique	30
3.3.2	Page de connexion	31
3.3.3	Shell de dashboard partagé	32
3.3.4	Première vue métier : tableau de bord TA	32
3.3.5	Gestion des utilisateurs côté administration	33
3.4	Apport du module	33
	Conclusion	34

4	Module 2 : vivier de CV et gestion des offres d'emploi	35
	Introduction	35
4.1	Sprint 2 : objectif et périmètre	35
4.1.1	Objectif du sprint	35
4.1.2	Backlog du sprint	36
4.2	Implémentation du sprint	36
4.2.1	Analyse du sprint	36
4.2.2	Vue fonctionnelle globale	38
4.2.3	Pipeline d'upload et d'enrichissement d'un CV	39
4.2.4	Indexation sémantique et préparation du RAG	40
4.2.5	Création d'une offre et affectation vers le pipeline candidat	40
4.3	Réalisation	41
4.3.1	Page <i>CV Pool</i>	41
4.3.2	Upload en arrière-plan et robustesse d'ingestion	43
4.3.3	Détection de doublons	43
4.3.4	Page <i>Jobs</i>	44
4.3.5	Templates et fermeture contrôlée des offres	45
4.3.6	Page détail d'une offre	45
4.4	Apport du module	46
	Conclusion	47
5	Module 3 : pipeline candidat et gestion des entretiens	48
	Introduction	48
5.1	Sprint 3 : objectif et périmètre	48
5.1.1	Objectif du sprint	48
5.1.2	Backlog du sprint	49
5.2	Implémentation du sprint	50
5.2.1	Analyse du sprint	50
5.2.2	Vue fonctionnelle globale	50
5.2.3	Cycle de vie du candidat	51
5.2.4	Affectation progressive des responsabilités	52
5.2.5	Planification d'un entretien	53
5.2.6	Rapports d'entretien et transition vers l'onboarding	54
5.2.7	Historique auditable des transitions	54
5.3	Réalisation	55
5.3.1	Vue Manager	55
5.3.2	Vue RH	56
5.3.3	Calendrier et coordination transverse	58
5.3.4	Mémoire opérationnelle du processus	58
5.3.5	Articulation avec les modules IA	58
5.4	Apport du module	59
	Conclusion	60

6	Module 4 : intelligence artificielle, matching et recherche augmentée	61
	Introduction	61
6.1	Sprint 4 : objectif et périmètre	61
6.1.1	Objectif du sprint	61
6.1.2	Backlog du sprint	63
6.2	Implémentation du sprint	64
6.2.1	Analyse du sprint	64
6.2.2	Niveaux d'intelligence du projet	65
6.2.3	Matching, screening et autopilot	65
6.2.4	Agent conversationnel, réponses exploitables et garde-fous	66
6.3	Vue dynamique	67
6.3.1	Orchestration agentique	67
6.3.2	Pipeline RAG hybride	67
6.3.3	Réécriture de requête et gestion de la complexité	69
6.4	Réalisation	70
6.4.1	IA dans le détail d'une offre	70
6.4.2	Recherche sémantique et services IA avancés	70
6.4.3	Autopilot d'entretien	71
6.4.4	Page statistiques, workspace agentique et réponses sourcées	72
6.4.5	Fonctionnement détaillé du chat analytique	73
6.5	Évaluation et résultats	75
6.5.1	Analyse de complexité, coût et pistes d'optimisation	77
6.6	Apport du module	78
	Conclusion	78
7	Module 5 : notifications, analytique, administration et gouvernance	79
	Introduction	79
7.1	Sprint 5 : objectif et périmètre	79
7.1.1	Objectif du sprint	79
7.1.2	Backlog du sprint	81
7.2	Implémentation du sprint	82
7.2.1	Analyse du sprint	82
7.2.2	Vue fonctionnelle globale	83
7.2.3	Chaîne de notification et de communication	83
7.2.4	Supervision administrateur	84
7.2.5	Audit, logs et exports	84
7.2.6	Centre de gouvernance et audit consolidé	85
7.2.7	Protection des données personnelles et conformité RGPD	85
7.2.8	Accessibilité et validation UX	86
7.3	Réalisation	87
7.3.1	Cloche de notification et communication in-app	87
7.3.2	Tableau de bord administrateur	88

7.3.3	Analytique administrateur	89
7.3.4	Centre de gouvernance	90
7.3.5	Historique d'activité et logs d'emails	91
7.3.6	Suivi détaillé de l'onboarding	92
7.3.7	Exports et interopérabilité	93
7.3.8	Vérification de durcissement release	93
7.4	Apport du module	94
	Conclusion	95
	Conclusion générale	96
	Bibliographie et webographie	104

Table des figures

1.1	Page d'accueil institutionnelle de la plateforme.	4
2.1	Cas d'utilisation global de la plateforme Capgemini Talent Intelligence . . .	13
2.2	Processus métier cible du recrutement, sous forme BPMN simplifiée	14
2.3	Diagramme de classes global centré recrutement	15
2.4	Architecture interne <i>feature-driven</i> de la solution	18
2.5	Orchestration de l'agent conversationnel et de ses outils	21
3.1	Séquence d'authentification et de redirection par rôle	29
3.2	Page d'accueil publique de la plateforme.	31
3.3	Page de connexion sécurisée de la plateforme.	32
3.4	Tableau de bord TA après authentification.	33
4.1	Séquence d'upload, d'extraction et d'indexation d'un CV	39
4.2	Création d'un poste et affectation d'un CV au pipeline candidat	41
4.3	Page de gestion du vivier de CV.	42
4.4	Dialogue de consultation détaillée d'un CV.	43
4.5	Catalogue des offres d'emploi.	44
4.6	Zone de création ou d'édition d'une offre d'emploi.	45
4.7	Page détail d'une offre avec candidats associés et résultats de matching. . .	46
5.1	Diagramme d'états du pipeline candidat	52
5.2	Séquence de planification, notification et reporting d'un entretien	54
5.3	Fiche candidat côté Manager.	56
5.4	Fiche candidat côté RH.	57
5.5	Interface de planification d'entretien.	57
5.6	Vue calendrier des entretiens.	58
6.1	Chaîne IA de screening et de génération de guide d'entretien	66
6.2	Pipeline RAG hybride du projet	69
6.3	Vue de score IA d'un candidat sur une offre.	70
6.4	Interface d'Auto-Pilot d'entretien.	72
6.5	Workspace agentique et réponse sourcée dans le chat.	73
6.6	Visualisation des métriques RAG et du compromis de latence	76

7.1	Chaîne de notification et de journalisation des emails	84
7.2	Cloche de notification et état de lecture.	88
7.3	Tableau de bord administrateur.	89
7.4	Vue analytique administrateur.	90
7.5	Centre de gouvernance administrateur.	91
7.6	Journal d'activité.	92
7.7	Suivi de l'onboarding.	93

Liste des tableaux

1.1	Comparaison fonctionnelle orientée besoins du projet	5
1.2	Synthèse de l'état de l'art et positionnement du projet	7
1.3	Objectifs structurants de Capgemini Talent Intelligence	8
1.4	Comparaison entre approche agile et approche classique	9
2.1	Acteurs du système	11
2.2	Exigences non fonctionnelles clés	12
2.3	Product Backlog synthétique du projet	16
2.4	Objectifs des sprints de réalisation	17
2.5	Correspondance entre besoins métier et réponses architecturales	19
2.6	Justification des principaux choix technologiques	25
3.1	Backlog fonctionnel du module d'accès et d'identité	27
4.1	Backlog fonctionnel du module vivier de CV et offres	36
4.2	Transformation documentaire du CV et de l'offre	38
5.1	Backlog fonctionnel du module pipeline candidat et entretiens	49
5.2	Transitions principales du pipeline candidat	52
5.3	Rôles et responsabilités dans le pipeline candidat	53
6.1	Backlog fonctionnel du module intelligence artificielle	63
6.2	Comparaison entre recherche simple, recherche sémantique, RAG et chat agentique	68
6.3	Justification analytique des choix IA	69
6.4	Restitutions visuelles générées par le chat analytique	75
6.5	Résultats de l'évaluation RAG phase 2	76
6.6	Analyse de complexité et de coût du sous-système IA	77
7.1	Backlog fonctionnel du module gouvernance et administration	81
7.2	Mesures de protection des données personnelles	86
7.3	Validation UX et accessibilité	87
7.4	Synthèse de validation fonctionnelle	97
7.5	Évaluation de l'atteinte des objectifs initiaux	98
7.6	Menaces à la validité et mesures de maîtrise éthique	101

Mots-clés et acronymes

Mots-clés

Recrutement intelligent, gestion de candidatures, IA générative, agent conversationnel, RAG, pgvector, Next.js, TypeScript, Better Auth, Drizzle ORM, gouvernance RH.

Acronymes

- **AI / IA** : Artificial Intelligence / Intelligence Artificielle
- **ATS** : Applicant Tracking System
- **CV** : Curriculum Vitae
- **HR / RH** : Human Resources / Ressources Humaines
- **LLM** : Large Language Model
- **RAG** : Retrieval-Augmented Generation
- **RBAC** : Role-Based Access Control
- **SSE** : Server-Sent Events
- **TA** : Talent Acquisition
- **UML** : Unified Modeling Language

Introduction générale

La transformation numérique des fonctions *Talent Acquisition* ne se limite plus à la publication d'offres et au suivi administratif des candidatures. Dans les contextes de recrutement à volume élevé, les équipes doivent simultanément centraliser des CV hétérogènes, structurer les données utiles, coordonner plusieurs intervenants métiers, documenter chaque décision et accélérer les délais de traitement sans sacrifier la qualité d'évaluation. Cette pression opérationnelle crée un terrain favorable à l'automatisation, à l'analytique temps réel et à l'intelligence artificielle appliquée aux flux RH.

C'est dans ce cadre que s'inscrit Capgemini Talent Intelligence, une plateforme de recrutement orientée entreprise développée avec `Next.js`¹⁶, `TypeScript`, `Bun`, `NeonPostgreSQL`, `DrizzleORM` et un socle d'IA s'appuyant sur des API compatibles OpenAI/NVIDIA [7][9][10][15][14]. Les documentations officielles des principales technologies citées sont regroupées dans la bibliographie et webographie. La solution couvre l'ensemble du cycle de recrutement : page d'accueil et authentification, gestion d'un vivier de CV, création et suivi des postes, assignation des candidats, historique des transitions, planification des entretiens, rapports d'évaluation, notifications, exports, tableaux de bord administratifs, centre de gouvernance et assistant conversationnel IA.

L'originalité majeure de la solution réside dans sa couche d'intelligence. Le système ne se contente pas d'ajouter une fonctionnalité d'aide à la rédaction ; il intègre un agent conversationnel multi-outils, un pipeline hybride de recherche *Retrieval-Augmented Generation* (RAG), des mécanismes de *grounding* destinés à limiter les réponses non fondées, une confirmation explicite avant les actions IA qui modifient les données et des évaluations quantitatives sur un jeu de 40 requêtes. Le principe RAG s'appuie sur la littérature de référence en récupération augmentée [20], tandis que l'implémentation du projet l'adapte aux contraintes métier RH.

L'objectif de ce rapport est de présenter, dans un cadre académique, la conception et la réalisation de cette plateforme selon une progression structurée : un cadrage général, un chapitre d'analyse et d'architecture, puis cinq chapitres d'implémentation organisés par modules fonctionnels. Cette structuration permet d'exposer à la fois la logique métier, les décisions techniques et la valeur produite pour les différents rôles de l'application : **TA**, **Manager**, **HR** et **Admin**.

La question directrice du mémoire peut donc être formulée ainsi : comment concevoir une plateforme de recrutement qui centralise les opérations métier tout en intégrant une assistance IA utile, mesurable et gouvernée ? Cette question guide la structure du

rapport : chaque chapitre ne présente pas seulement une fonctionnalité isolée, mais une brique nécessaire à cette chaîne complète, depuis l'accès sécurisé jusqu'à la supervision des décisions et des communications.

Le présent mémoire est organisé comme suit :

- le **chapitre 1** présente l'entreprise d'accueil, le contexte métier du recrutement et les objectifs du projet ;
- le **chapitre 2** formalise les besoins, le Product Backlog, les objectifs de sprint, l'architecture applicative, le modèle de données et l'environnement technique ;
- le **chapitre 3** traite le module d'accès, d'identité et d'entrée par rôle ;
- le **chapitre 4** couvre la gestion du vivier de CV et des offres d'emploi ;
- le **chapitre 5** détaille le pipeline candidat et la conduite des entretiens ;
- le **chapitre 6** expose la couche d'intelligence artificielle, le matching, l'autopilot d'entretien et le pipeline RAG ;
- le **chapitre 7** présente la supervision, les notifications, l'analytique et la gouvernance opérationnelle ;
- enfin, une **conclusion générale** synthétise les acquis, la validation et les apports du projet.

Ainsi, ce rapport ne décrit pas seulement une interface web de recrutement, mais une plateforme métier complète, conçue pour fiabiliser les décisions, améliorer la traçabilité des opérations et préparer l'industrialisation d'usages IA dans un contexte RH exigeant.

Chapitre 1

Présentation générale

Introduction

Ce premier chapitre pose le cadre organisationnel et métier du projet. Il présente d’abord l’entreprise d’accueil et le type d’environnement dans lequel la solution s’insère, puis formalise la problématique du recrutement, l’analyse synthétique de solutions existantes et les objectifs poursuivis par Capgemini Talent Intelligence.

1.1 Présentation de l’entreprise d’accueil

1.1.1 Présentation de Capgemini et de Capgemini Engineering

Capgemini se présente comme un acteur mondial du conseil, des services technologiques et de la transformation numérique. La marque *Capgemini Engineering* regroupe plus spécifiquement les activités d’ingénierie, de développement logiciel et de transformation de produits numériques. Dans le contexte de ce projet, cette double identité est importante : le besoin traité est bien un besoin métier RH, mais la réponse attendue est une plateforme logicielle industrialisable, intégrant de la donnée, de l’IA et des interfaces adaptées à plusieurs profils utilisateurs [2][3].

1.1.2 Structure organisationnelle pertinente pour le projet

À l’échelle du projet, l’organisation utile n’est pas une hiérarchie complète de l’entreprise, mais un ensemble d’acteurs qui collaborent sur la chaîne de recrutement :

- les équipes **Talent Acquisition (TA)**, responsables du sourcing, du vivier de CV et du pilotage des postes ;
- les **Managers**, impliqués dans l’évaluation technique ou métier des candidats ;
- les **RH**, garants de la décision finale, de la communication et de la contractualisation ;
- les **Administrateurs**, qui supervisent les comptes, les tableaux de bord globaux et les traces d’activité.

Cette structuration est confirmée dans le projet par la documentation des rôles, par le module d'authentification et par l'organisation des espaces protégés [1].

1.1.3 Services et périmètre numérique concernés

Le projet s'inscrit dans un périmètre numérique combinant :

- la gestion opérationnelle du recrutement ;
- l'assistance à la décision par l'analyse de CV, le matching et les rapports IA ;
- la gouvernance par les exports, les notifications, l'historique d'activité et les indicateurs ;
- l'expérience utilisateur multi-rôles, avec des tableaux de bord différenciés.

Autrement dit, il ne s'agit pas d'un simple ATS documentaire, mais d'un système de travail partagé entre équipes de recrutement et décideurs.

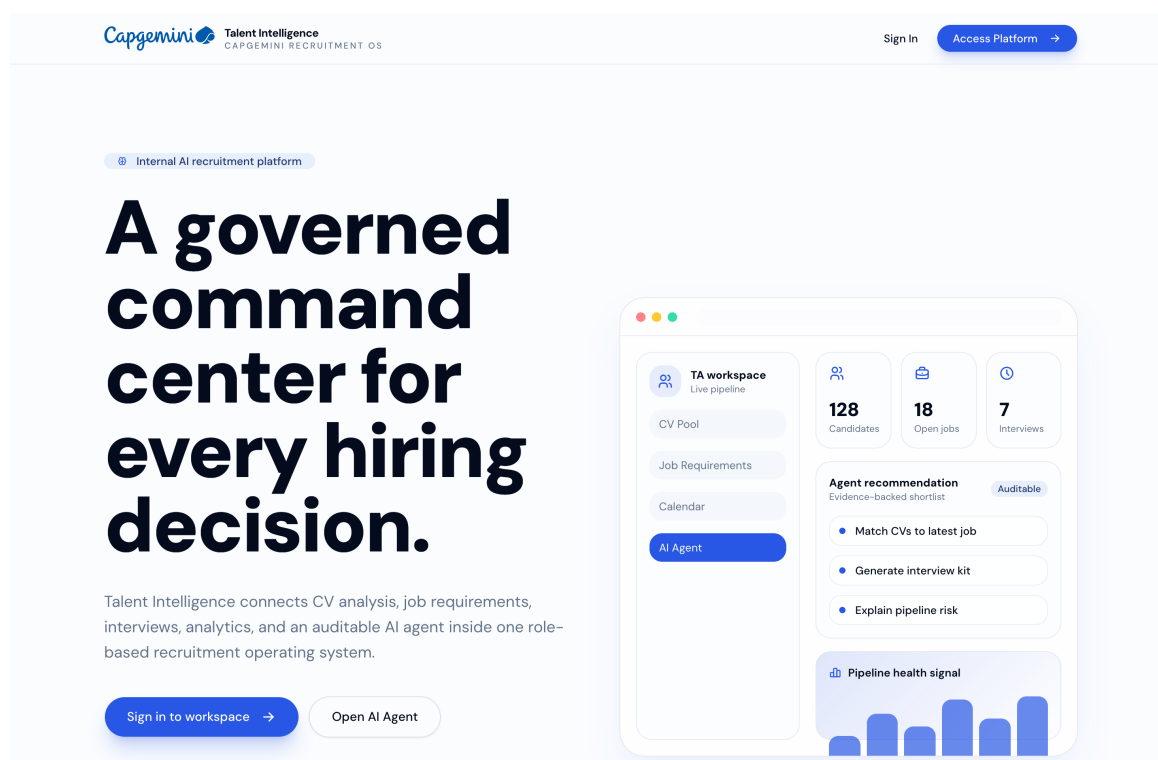


Figure 1.1 : Page d'accueil institutionnelle de la plateforme.

1.2 Présentation du projet

1.2.1 Problématique

Les recrutements d'entreprise rencontrent plusieurs difficultés récurrentes : multiplicité des candidatures, hétérogénéité des formats de CV, dispersion des informations, lenteur

des validations intermédiaires, manque de visibilité sur le pipeline et difficulté à exploiter l'IA sans perdre la maîtrise métier. L'analyse fonctionnelle du projet montre que le besoin adressé par Capgemini Talent Intelligence porte précisément sur cette chaîne complète : ingestion documentaire, structuration, assignation, évaluation multi-étapes, planification d'entretiens et pilotage analytique.

1.2.2 Étude synthétique des solutions existantes

Avant de construire une solution interne, il est pertinent de situer le projet par rapport à des plateformes largement diffusées comme LinkedIn Talent Solutions, Greenhouse et Workday Recruiting, dont les sites officiels sont référencés en bibliographie et webographie [4][5][6]. L'objectif n'est pas d'établir un benchmark de performance absolu, mais d'identifier les écarts entre des offres ATS généralistes et le besoin spécifique du projet.

Table 1.1 : Comparaison fonctionnelle orientée besoins du projet

Critère	LinkedIn Solutions	Talent	Greenhouse	Workday Recruiting
Vivier de CV interne structuré	Partiel		Oui	Oui
Matching IA directement relié au vivier	Limité		Partiel	Limité
Pipeline multi-rôles TA / Manager / HR	Partiel		Oui	Oui
Assistant conversationnel outillé	Non natif		Non natif	Non natif
Recherche sémantique sur CV avec citations	Non mise en avant		Non mise en avant	Non mise en avant
Gouvernance intégrée (activité, emails, onboarding)	Partiel		Partiel	Partiel
Adaptation au périmètre exact du projet	Faible		Moyenne	Moyenne

La lecture de cette comparaison met en évidence trois constats. LinkedIn Talent Solutions est surtout fort pour la visibilité et le sourcing, mais ne répond pas directement au besoin d'un vivier interne entièrement gouverné. Greenhouse couvre mieux les workflows ATS, mais reste moins orienté vers une recherche sémantique interne avec citations et agent conversationnel outillé. Workday Recruiting est pertinent dans une suite RH glo-

bale, mais il répond moins directement à un projet ciblé sur un pipeline interne, modulaire et fortement branché sur les données de recrutement du projet.

Ainsi, l'écart principal ne concerne pas une fonctionnalité unique, mais l'intégration complète entre données internes, workflow multi-rôles, IA contrôlée et gouvernance. C'est précisément cet écart que Capgemini Talent Intelligence cherche à couvrir.

Cette synthèse justifie le choix d'une plateforme dédiée : le besoin ne porte pas seulement sur la publication d'offres ou le suivi d'entretien, mais sur l'orchestration d'un processus complet enrichi par l'IA, avec une forte maîtrise interne des données, des rôles et des règles de décision.

1.2.3 État de l'art scientifique et positionnement

Au-delà des plateformes commerciales, l'état de l'art montre que le recrutement numérique se situe au croisement de plusieurs familles de travaux : les systèmes de recommandation CV-offre, la recherche d'information, les architectures RAG, les agents outillés et l'audit des risques algorithmiques. Les travaux sur le matching de CV, comme Résumatcher publié dans *Expert Systems with Applications* [25], montrent l'intérêt d'un score personnalisé entre profil et poste. Les travaux sur les algorithmes de recrutement rappellent cependant que l'automatisation doit rester gouvernée, documentée et auditable, notamment pour éviter de transformer un gain de productivité en risque de biais ou d'opacité [26].

La littérature récente sur le RAG met en avant un principe important : un modèle génératif devient plus fiable lorsqu'il est contraint par des documents récupérés et citables plutôt que par sa seule mémoire paramétrique [20][22]. La recherche d'information rappelle aussi que les signaux lexicaux et sémantiques sont complémentaires. BM25 reste robuste pour les termes exacts [23], tandis que la fusion de rangs par RRF permet de combiner plusieurs listes classées sans imposer une normalisation fragile des scores [24]. Enfin, les travaux sur les agents LLM et l'utilisation d'outils montrent qu'un assistant devient plus utile lorsqu'il peut raisonner, appeler des fonctions et restituer des preuves, mais seulement si son action reste bornée par un cadre de contrôle [27][28][29].

Table 1.2 : Synthèse de l'état de l'art et positionnement du projet

Approche	Principe dominant	Limite observée	Positionnement de Capgemini Talent Intelligence
ATS et recherche SQL	Filtres structurés, statuts et mots-clés exacts.	Faible tolérance aux synonymes, aux formulations différentes et aux profils incomplets.	Conserver les filtres métier, mais les compléter par des embeddings, des chunks et un ranking hybride.
Matching CV-offre	Calcul d'un score de similarité entre exigences et profil candidat.	Score souvent isolé du workflow complet et difficile à expliquer sans traces.	Relier le score aux exigences du poste, aux écarts, aux guides d'entretien et à l'historique candidat.
Recherche sémantique dense	Projection des CV et requêtes dans un espace vectoriel.	Risque de rapprochements trop flous et difficulté à citer précisément l'origine de la réponse.	Utiliser l'embedding global pour explorer, puis les chunks RAG pour justifier les réponses complexes.
RAG hybride	Récupération de passages puis génération contrôlée par contexte.	Latence et coût plus élevés si chaque requête déclenche toutes les étapes.	Activer réécriture, RRF et reranking surtout pour les requêtes complexes, avec cache et fallback.
Agentic AI	Agent capable d'appeler des outils et de raisonner en plusieurs étapes.	Risque d'action non maîtrisée, d'hallucination ou de fuite de données si les outils ne sont pas filtrés.	Filtrage RBAC, validation Zod, grounding des candidats, confirmations explicites et trace outillée par phases.

Comme le montre le tableau 1.2, la contribution du projet n'est pas de proposer un nouveau modèle fondationnel. Elle consiste à intégrer des techniques connues dans une architecture métier cohérente : données RH centralisées, recherche hybride, RAG cité, agent outillé, contrôle d'accès, confirmation humaine et évaluation quantitative. Cette contribution est donc principalement systémique et applicative : rendre l'IA exploitable dans un processus de recrutement gouverné, plutôt que démontrer une IA isolée du contexte métier.

1.2.4 Objectifs du projet

Les objectifs fonctionnels et techniques peuvent être résumés comme suit.

Table 1.3 : Objectifs structurants de Capgemini Talent Intelligence

Axe	Objectif
Centralisation	Unifier le vivier de CV, les postes, les candidats, les entretiens et les notifications dans une même plateforme.
Accélération	Réduire le temps de tri et de passage entre étapes grâce à l'automatisation et à l'IA.
Fiabilisation	Encadrer les décisions via des rôles explicites, des états de pipeline, des logs et des exports.
Assistance intelligente	Générer des screenings, guides d'entretien, analyses et recherches avancées à partir des données du système.
Gouvernance	Donner aux administrateurs une vision globale de l'activité, des emails et de l'onboarding.
Évolutivité	S'appuyer sur une architecture par fonctionnalités, plus adaptée à l'évolution incrémentale qu'un empilement de pages isolées.

1.2.5 Solution proposée

La solution proposée est une application web full-stack appelée Capgemini Talent Intelligence, organisée autour de quatre rôles métier et d'un noyau de services métier regroupés par fonctionnalité. L'application repose sur :

- une page d'accueil et une authentification par email/mot de passe ;
- des espaces de travail spécifiques pour TA, Manager, HR et Admin ;
- un vivier de CV avec extraction automatique des données ;
- des offres d'emploi structurées, réutilisables sous forme de modèles ;
- un pipeline candidat à douze états ;
- des modules d'entretien, de reporting et d'onboarding ;
- une couche IA combinant matching, génération et agent conversationnel outillé.

1.3 Méthodologie adoptée

1.3.1 Choix de la méthodologie

La structure du projet et la livraison progressive de ses modules rendent une méthodologie itérative particulièrement adaptée. La plateforme n'a pas été construite comme un livrable unique et figé ; elle a évolué par incréments successifs couvrant le contrôle d'accès, la gestion du vivier de CV, les offres, la progression des candidats, les capacités IA, la communication et la gouvernance.

1.3.2 Comparaison entre approches de gestion de projet

Deux grandes approches peuvent être envisagées pour ce type de projet : l'approche classique, qui définit les besoins en détail dès le départ puis exécute le projet de manière séquentielle, et l'approche agile, qui livre la valeur de façon incrémentale, intègre le retour d'expérience au fil du développement et permet d'ajuster le périmètre lorsque les priorités métier se précisent.

Table 1.4 : Comparaison entre approche agile et approche classique

Critère	Approche agile	Approche classique
Mode de livraison	Incrémental et itératif	Séquentiel
Intégration du feedback	Continue, à chaque module livré	Tardive, souvent concentrée en fin de cycle
Adaptation au changement	Élevée, adaptée aux ajustements métier et IA	Faible, car les changements perturbent le plan initial
Visibilité sur l'avancement	Fréquente grâce aux incréments fonctionnels	Limitée entre les jalons majeurs
Adéquation à ce projet	Forte, car le produit combine workflows métier, interfaces et expérimentation IA	Moyenne, car le besoin évolue avec les résultats techniques observés

Comme ce projet combine conception produit, workflows d'entreprise et expérimentation autour de l'IA, l'approche agile apparaît plus adaptée qu'un processus strictement séquentiel. Elle permet de stabiliser d'abord le socle métier, puis d'enrichir progressivement le produit avec l'analytique, l'aide à la décision, l'outillage conversationnel et les modules de gouvernance.

1.3.3 Méthodologie retenue

Une méthode agile et incrémentale a donc été retenue. Le travail a été organisé en modules fonctionnels livrés progressivement, chaque incrément apportant une valeur métier visible à la plateforme. Cette méthodologie est également cohérente avec l'architecture du projet : comme l'application est organisée par fonctionnalités, chaque itération pouvait se concentrer sur une tranche métier cohérente, sans disperser les changements dans des couches techniques déconnectées.

Ce choix présente trois avantages : il rend visible la progression de la valeur métier, il permet d'associer chaque incrément à des routes, composants et services identifiables, et il isole clairement la montée en complexité de la couche IA, qui n'intervient qu'après la mise en place du socle documentaire et opérationnel.

Conclusion

Le projet Capgemini Talent Intelligence répond à une problématique de recrutement d'entreprise plus large qu'un simple suivi de candidatures. Il vise la centralisation des opérations, l'aide à la décision et la traçabilité du processus complet. Le chapitre suivant formalise cette ambition sous forme d'acteurs, d'exigences, d'architecture et de modèle de données.

Chapitre 2

Planification et architecture

Introduction

Après avoir cadré le besoin métier, ce chapitre traduit la vision produit en exigences, en modèle de données et en architecture technique. Il s'appuie sur les artefacts techniques du projet : documentation d'architecture, schéma de données, couche de routage et services métier [1].

2.1 Analyse des besoins et spécification

2.1.1 Identification des acteurs

Le système distingue quatre acteurs principaux, définis à partir de la spécification fonctionnelle et de la couche de contrôle d'accès.

Table 2.1 : Acteurs du système

Acteur	Responsabilités principales
TA	Gérer le vivier de CV, créer les offres, affecter les candidats, piloter le matching et les premiers entretiens.
Manager	Évaluer les candidats au stade managérial, conduire les entretiens, produire des rapports et orienter vers les RH.
HR	Finaliser l'évaluation, décider de l'acceptation ou du rejet, générer et envoyer les emails de décision, déclencher l'embauche.
Admin	Superviser les utilisateurs, l'analytique, les logs d'activité, les emails et l'onboarding global.

2.1.2 Exigences fonctionnelles

Les exigences fonctionnelles les plus structurantes sont les suivantes :

- authentifier les utilisateurs et les rediriger vers un espace adapté à leur rôle ;

- importer des CV PDF ou DOCX, extraire leur contenu et enrichir les métadonnées ;
- rechercher les profils par mots-clés ou par similarité sémantique ;
- créer des postes avec critères *must-have*, *nice-to-have*, séniorité et unité métier ;
- affecter les CV à des postes et suivre les candidats dans un pipeline d'états ;
- conserver un historique auditable des transitions du pipeline candidat ;
- planifier des entretiens, envoyer des invitations et saisir des rapports ;
- générer des évaluations IA, des guides d'entretien et des emails ;
- exposer un assistant conversationnel capable d'appeler des outils métier ;
- soumettre les actions IA modifiant les données à une confirmation explicite ;
- fournir des tableaux de bord, des exports et des vues de gouvernance.

2.1.3 Exigences non fonctionnelles

Les exigences non fonctionnelles retenues pour la solution sont résumées ci-dessous.

Table 2.2 : Exigences non fonctionnelles clés

Axe	Traduction dans la solution
Sécurité	Authentification better-auth , contrôle d'accès par rôle sur les routes, les actions serveur et les outils IA [13].
Validation	Validation Zod sur les entrées utilisateur et les arguments d'outils [18].
Performance	Sélection explicite des colonnes, limites de requêtes, embeddings asynchrones, streaming SSE côté chat.
Évolutivité	Architecture <i>feature-driven</i> , services métier séparés, modules IA découplés, stockage pgvector [17].
Traçabilité	Logs d'activité, email logs, notifications, conversations, messages, historique d'étapes candidat et confirmations d'actions IA stockés en base.
Fiabilité IA	Guardrails sur le nombre d'étapes agentiques, les tokens, le timeout, les échecs d'outils et la confirmation des actions modifiant les données.

2.2 Étude analytique

2.2.1 Diagramme global des cas d'utilisation

La figure suivante synthétise les interactions majeures entre les quatre acteurs et les grands sous-systèmes métier.

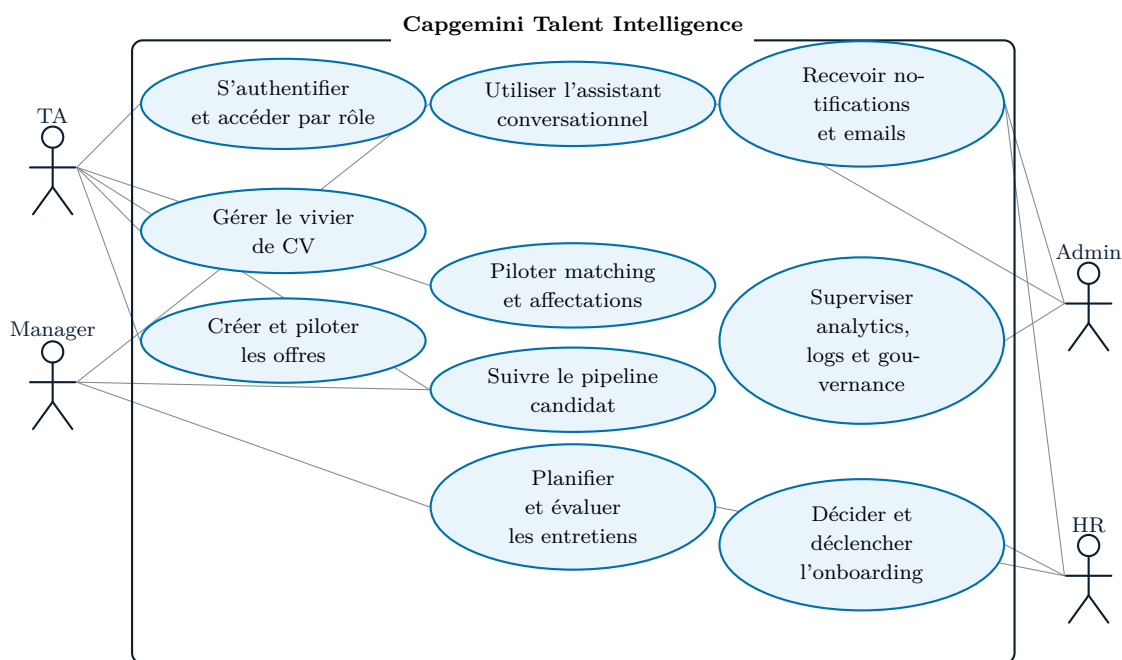


Figure 2.1 : Cas d'utilisation global de la plateforme Capgemini Talent Intelligence

2.2.2 Processus métier cible

Le diagramme de cas d'utilisation identifie les capacités du système, mais il ne suffit pas à montrer la circulation réelle d'une candidature entre les rôles. Pour répondre à cette limite, la figure 2.2 formalise un processus métier de type BPMN simplifié : le TA initialise le besoin et le vivier, l'IA assiste le classement, le Manager évalue la compétence opérationnelle, puis HR arbitre la décision finale et déclenche l'onboarding ou la notification.

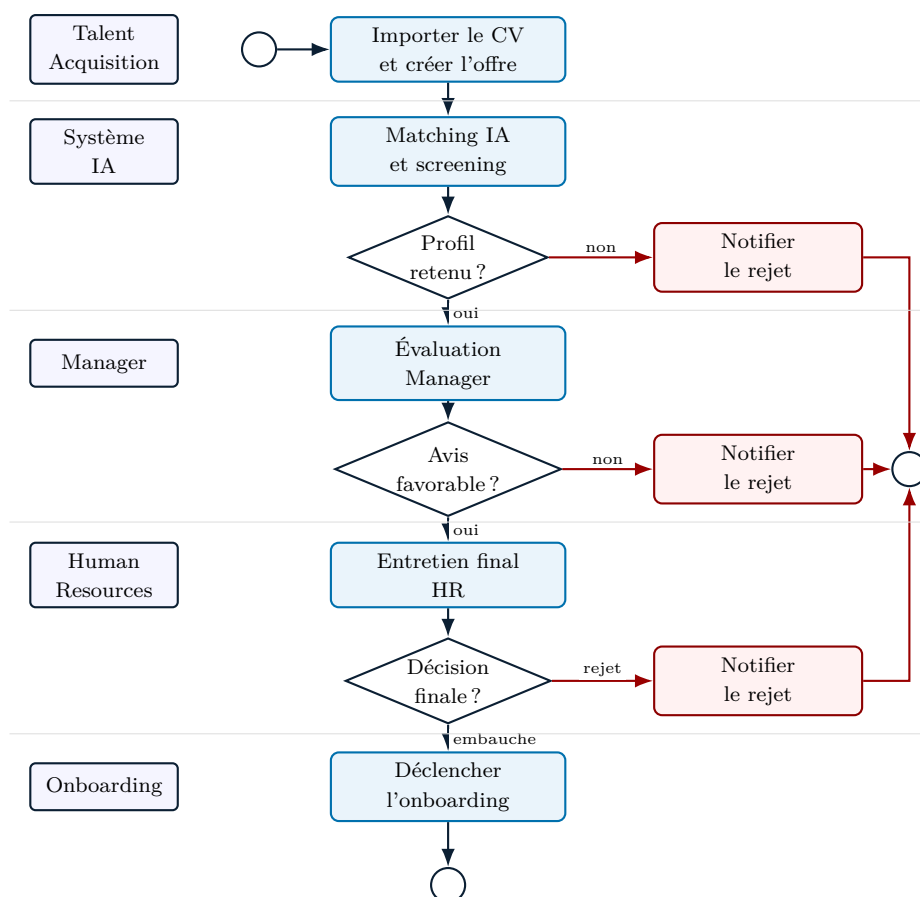


Figure 2.2 : Processus métier cible du recrutement, sous forme BPMN simplifiée

Cette lecture métier justifie ensuite les choix de conception : chaque étape doit posséder un responsable, un état observable, une trace et une règle de transition. Le pipeline candidat et l'historique des changements ne sont donc pas des détails techniques ; ils matérialisent le besoin de gouvernance visible dans le processus.

2.2.3 Diagramme de classes global

Le schéma Drizzle du projet est reformulé en diagramme de classes UML afin de montrer les classes métier, leurs attributs principaux et leurs multiplicités. La figure 2.3 relie ainsi les objets centraux du recrutement : comptes utilisateurs, postes, CV, candidats, évaluations, entretiens, rapports et événements de gouvernance.

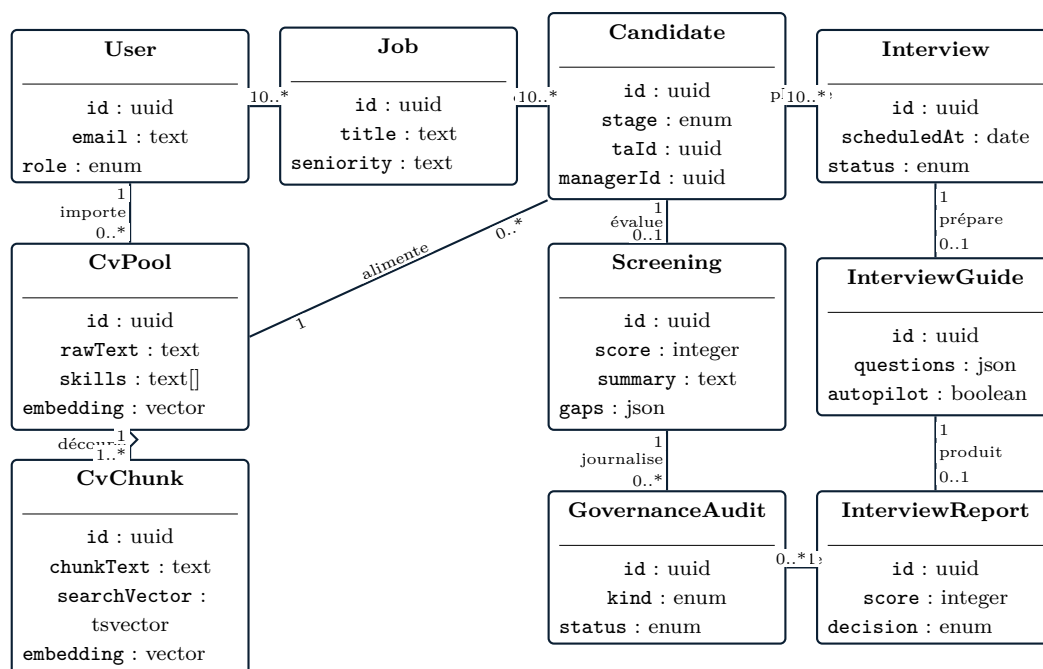


Figure 2.3 : Diagramme de classes global centré recrutement

2.2.4 Vue d'ensemble du Product Backlog

Le *Product Backlog* constitue la passerelle entre le besoin métier décrit au chapitre précédent et la réalisation par sprints présentée dans la suite du rapport. Il ne s'agit pas seulement d'une liste de fonctionnalités : il ordonne les capacités du produit selon leur valeur pour les équipes de recrutement, leurs dépendances techniques et leur rôle dans la constitution progressive des données exploitables par l'IA.

Logique de priorisation. La priorité a été donnée aux éléments qui débloquent les autres modules. Le socle d'accès arrive en premier parce qu'aucune donnée RH ne peut être exposée sans authentification et contrôle de rôle. Le vivier de CV et les offres viennent ensuite, car ils produisent les objets métier fondamentaux. Le pipeline candidat transforme ces objets en processus opérationnel. L'IA intervient lorsque suffisamment de données structurées existent pour produire des scores, des synthèses et des réponses contextualisées. Enfin, la gouvernance consolide les traces, les notifications et les indicateurs nécessaires au pilotage.

Table 2.3 : Product Backlog synthétique du projet

ID	Epic produit	Valeur métier	Items principaux
PB1	Accès et identité	Sécuriser l'entrée dans la plateforme et orienter chaque utilisateur vers son espace.	Landing page, connexion, redirection par rôle, protection des routes, shell de dashboard.
PB2	Vivier de CV	Centraliser les profils et transférer des documents hétérogènes en données consultables.	Upload PDF/DOCX, extraction de texte, métadonnées, recherche, export, détection de doublons.
PB3	Offres d'emploi	Structurer les besoins de recrutement pour préparer le matching et les décisions.	Création de poste, critères <i>must-have</i> , critères <i>nice-to-have</i> , séniorité, fermeture contrôlée.
PB4	Pipeline candidat	Faire progresser un profil affecté dans un workflow multi-rôles traçable.	Affectation, états candidat, responsabilités TA/Manager/HR, planification, rapports d'entretien, historique immuable des transitions.
PB5	Intelligence IA	Assister les recruteurs par des analyses contextualisées et contrôlées.	Screening, matching hybride, guides d'entretien, autopilot, RAG, chat analytique outillé, confirmation des actions modifiant les données.
PB6	Gouvernance	Donner de la visibilité sur l'activité, les communications et l'onboarding.	Notifications, logs d'activité, logs emails, analytics, exports, centre de gouvernance, suivi d'intégration.

Ce backlog permet une lecture produit du projet : chaque epic correspond à une capacité visible par un utilisateur final, mais aussi à une brique de données réutilisable par les modules suivants. Par exemple, les offres structurées alimentent le matching ; les rapports d'entretien enrichissent la traçabilité ; les logs et notifications rendent possible une supervision administrateur crédible.

2.2.5 Découpage en sprints et objectifs

Le backlog est découpé en cinq sprints de réalisation. Chaque sprint possède un objectif explicite, afin que la progression du rapport reste lisible et que chaque chapitre d'implémentation puisse être évalué par rapport à une intention produit précise.

Table 2.4 : Objectifs des sprints de réalisation

Sprint	Module	Objectif principal
Sprint 1	Accès et identité	Mettre en place le point d'entrée sécurisé du produit : page publique, authentification, redirection par rôle et cadre de navigation commun.
Sprint 2	CV et offres	Construire la base documentaire et métier du recrutement : importer les CV, structurer les offres et préparer les traitements de recherche et de matching.
Sprint 3	Pipeline et entretiens	Transformer une affectation CV-offre en suivi candidat complet, avec transitions d'état, historique auditable, responsabilités, planification et rapports.
Sprint 4	Intelligence IA	Ajouter une assistance IA réellement branchée sur les données internes : screening, recherche hybride, guides, autopilot, chat analytique et confirmation des actions modifiant les données.
Sprint 5	Gouvernance	Superviser le produit en exploitation : notifications, emails, activité, analytics, exports, centre de gouvernance et onboarding.

Cette planification montre une trajectoire progressive : d'abord sécuriser l'accès, ensuite créer les données de référence, puis orchestrer le workflow métier, enrichir les décisions par l'IA et terminer par la supervision. Elle évite de présenter les sprints comme des blocs indépendants ; chacun prépare explicitement le suivant.

2.3 Architecture du projet

2.3.1 Architecture externe

D'un point de vue externe, la plateforme peut être vue comme un ensemble de flux entre cinq blocs :

- le client web utilisé par les rôles **TA**, **Manager**, **HR** et **Admin** ;
- l'application `Next.js`¹⁶, qui gère l'interface, les routes, le rendu serveur, les Server Actions et l'API de chat [7] ;
- la base `NeonPostgreSQL`, qui stocke les entités métier, les conversations, les logs et les embeddings [15][16] ;

- les services IA externes appelés pour les générations, les réécritures de requêtes et les embeddings ;
- le service d'emailing utilisé pour les invitations, les décisions et certaines notifications.

Le flux typique est le suivant : un utilisateur agit depuis un dashboard, la requête est authentifiée, la logique métier est exécutée dans les services, les données sont lues ou écrites en base, puis un appel IA ou email est déclenché lorsque le cas d'usage l'exige. Pour le chat analytique, la réponse est renvoyée progressivement au client en streaming SSE.

Cette vue externe permet de distinguer clairement ce qui appartient au produit et ce qui dépend de services tiers. Le cœur métier reste dans l'application et dans la base de données ; les services externes sont utilisés comme capacités spécialisées, sous contrôle des services applicatifs.

2.3.2 Architecture applicative globale

L'architecture adoptée n'est pas une architecture microservices. Le projet affirme explicitement une architecture *Feature-Driven* : la couche de routage reste mince, les composants d'interface sont séparés, et la logique métier réside dans des services organisés par fonctionnalité. Cette décision est particulièrement adaptée à une application web monolithique moderne enrichie par l'IA.

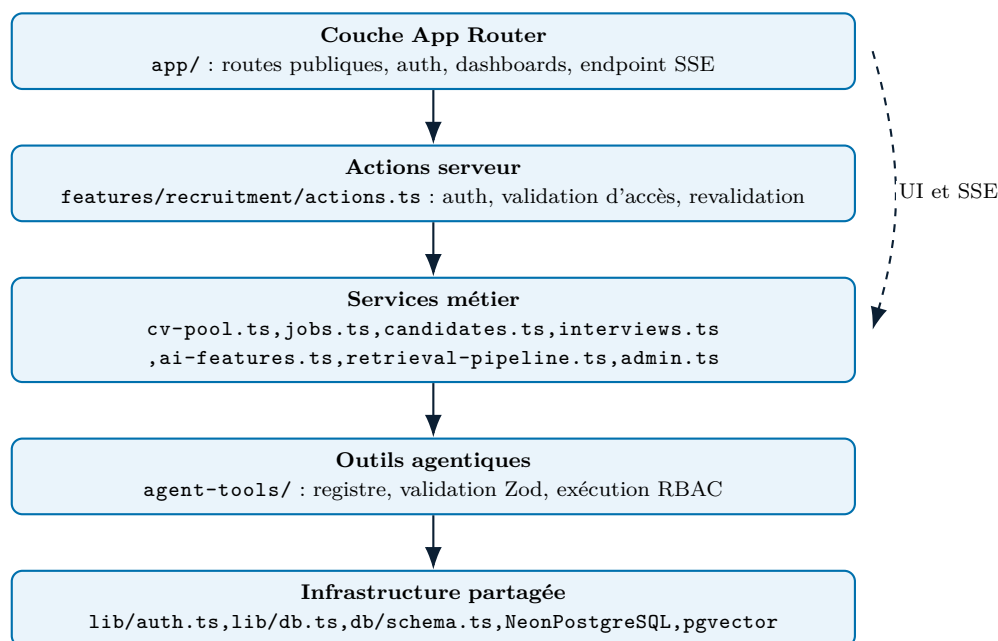


Figure 2.4 : Architecture interne *feature-driven* de la solution

Les frontières de couches sont les suivantes :

- la couche de routage gère le rendu et les points d'entrée HTTP ;
- la couche d'actions serveur encapsule les mutations et la revalidation ;

- la couche de services constitue la source de vérité métier et de persistance ;
- la couche de composants porte l'interface utilisateur ;
- la couche d'utilitaires concentre les fonctions transverses comme l'authentification, l'accès aux données, la limitation et les logs ;
- le schéma de données décrit le modèle persistant.

Tableau de correspondance besoin / module / solution technique. La cohérence de l'architecture se comprend mieux lorsqu'on relie directement les besoins métier aux modules fonctionnels et aux choix techniques qui y répondent.

Table 2.5 : Correspondance entre besoins métier et réponses architecturales

Besoin métier	Module principal	Solution technique retenue
Sécuriser l'accès et séparer les responsabilités	Accès et identité	Authentification par rôles, espaces protégés et shell de dashboard mutualisé.
Transformer des CV hétérogènes en profils exploitables	Vivier de CV	Pipeline d'ingestion, extraction structurée, embeddings et chunking progressif.
Relier les profils aux besoins de recrutement	Offres, candidats et pipeline	Offres structurées, transitions d'état explicites, historique d'étapes et assignations multi-rôles.
Retrouver des profils au-delà des mots-clés	Intelligence IA	Stockage vectoriel avec pgvector , recherche full-text et retrieval hybride.
Interroger le système en langage naturel	Chat analytique	Agent conversationnel outillé, registre d'outils filtré par rôle, confirmation des mutations et streaming SSE.
Auditer et piloter l'activité	Gouvernance	Notifications, logs, centre de gouvernance, analytics, exports et suivi détaillé de l'onboarding.

Ce tableau montre que l'architecture n'est pas le résultat d'un empilement de technologies. Chaque décision technique répond à un besoin métier identifiable et s'insère dans un module fonctionnel précis.

2.3.3 Architecture interne

L'architecture interne retenue est une architecture *feature-driven*, et non une architecture microservices. Ce choix évite de distribuer artificiellement des modules qui partagent les mêmes entités, la même authentification et la même base de données.

Justification du choix *feature-driven*. Le choix *feature-driven* est pertinent parce que le produit s'organise naturellement autour de tranches métier cohérentes : accès, vivier, offres, pipeline candidat, IA et gouvernance. Dans ce contexte, regrouper les services, les actions et les composants par fonctionnalité réduit la dispersion du code et rend plus lisible la responsabilité de chaque module. Cette organisation facilite aussi l'évolution incrémentale du produit, car chaque sprint peut enrichir une capacité métier identifiable sans multiplier les dépendances transverses.

Justification du monolithe moderne. Le projet n'appelle pas naturellement une architecture microservices. Les modules partagent fortement les mêmes entités, la même authentification, le même stockage et les mêmes règles de gouvernance. Un monolithe moderne permet donc de garder une base de vérité unifiée pour les rôles, les candidats, les entretiens, le chat et les traces d'activité. Il réduit également la complexité de déploiement, de cohérence transactionnelle et de sécurité, tout en restant compatible avec des capacités avancées comme le SSE, les actions serveur et l'orchestration de services IA externes.

2.3.4 Architecture du sous-système IA

Le sous-système IA articule plusieurs briques : génération de descriptions de poste, extraction de CV, matching, génération de guides d'entretien, débriefing, assistant conversationnel et pipeline RAG. L'agent statistique est exposé via l'API du chat analytique, qui assemble le contexte, filtre les outils selon le rôle, diffuse la réponse en SSE et restitue les preuves sous forme de sources, graphiques, cartes typées et trace d'exécution par phases.

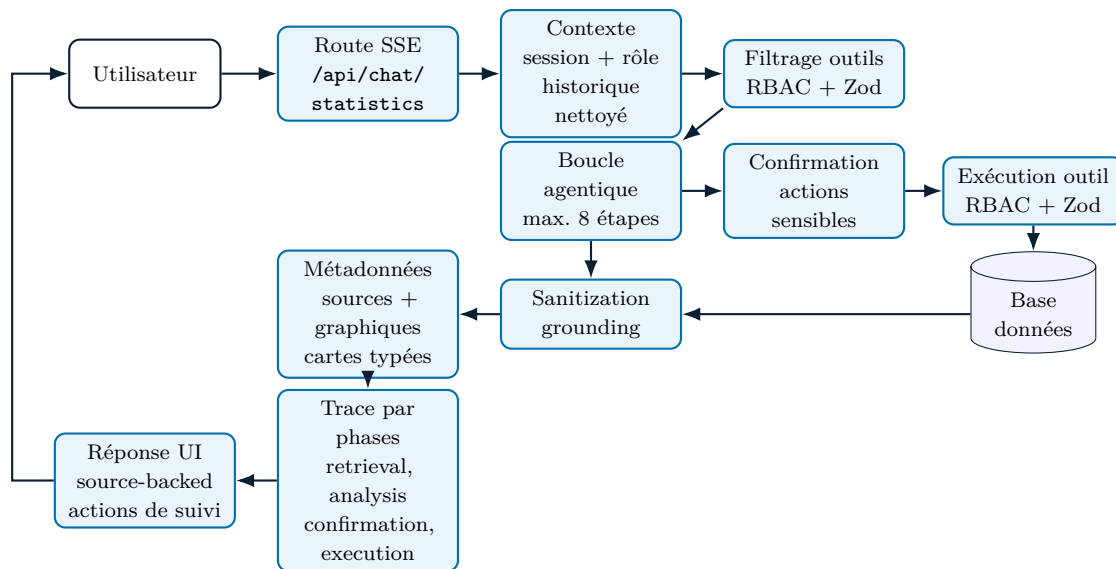


Figure 2.5 : Orchestration de l’agent conversationnel et de ses outils

Justification de pgvector. L’usage de `pgvector` permet de conserver les embeddings dans la même base que les données métier. Ce choix s’appuie sur la documentation officielle de l’extension `pgvector` [17] et évite de déporter la recherche vectorielle vers une infrastructure séparée. Il maintient une forte proximité entre profils, chunks, candidats et services de retrieval. Dans un produit de recrutement où les données doivent rester gouvernées, traçables et liées aux mêmes règles d’accès, cette unification simplifie l’architecture sans empêcher les usages avancés de similarité sémantique.

Justification du retrieval hybride. Le retrieval hybride est retenu parce qu’aucun signal unique n’est suffisant dans un contexte RH. Le principe RAG renvoie aux travaux de Lewis et al. sur la génération augmentée par récupération [20], tandis que la recherche vectorielle s’inscrit dans les approches de similarité à grande échelle mentionnées en bibliographie [21]. La recherche lexicale reste utile pour les intitulés exacts, les technologies nommées ou les mots-clés métier, tandis que la recherche vectorielle capte des proximités sémantiques moins littérales. Leur combinaison via RRF, complétée par un reranking, permet de limiter à la fois les faux négatifs liés au vocabulaire et les rapprochements trop flous liés à la seule similarité vectorielle.

Justification de l’agent outillé. Le choix d’un agent outillé répond à une limite claire des chatbots génériques : produire une réponse fluide ne suffit pas lorsqu’il faut interroger des données sensibles, filtrer les accès et agir sur des objets métier réels. En s’appuyant sur un registre d’outils explicitement validés, l’agent peut raisonner sur plusieurs étapes tout en restant borné par les opérations autorisées. Il devient ainsi une interface d’accès au système, et non un simple générateur de texte détaché du contexte applicatif.

Sécurité et fiabilité IA. Cette couche est encadrée par plusieurs mécanismes de sécurité et de fiabilité :

- RBAC, qui restreint l'accès aux outils, aux vues et aux données selon le rôle utilisateur ;
- validation Zod, qui borne les entrées des outils et évite qu'un appel mal formé ne soit exécuté implicitement ;
- confirmation explicite, qui transforme les outils agentiques modifiant les données en actions en attente avant exécution ;
- *grounding*, qui interdit à la réponse finale de citer des candidats non présents dans les résultats réellement accessibles ;
- cartes typées et trace par phases, qui transforment les sorties d'outils en métriques lisibles, liens profonds, actions de suivi et timeline d'exécution auditable ;
- isolation de cache et de périmètre, qui évite qu'une recherche ou un contexte d'un utilisateur ne contamine celui d'un autre ;
- garde-fous d'exécution, comme les limites d'étapes agentiques, de tokens, de timeout et d'échecs consécutifs ;
- suite d'évaluation agentique, qui vérifie de manière déterministe les politiques de grounding, de confirmation, de cartes de réponse, de traces d'outils et de masquage des données sensibles.

Ces garde-fous sont essentiels, car la crédibilité du projet repose précisément sur une IA utile, gouvernée et mesurable, et non sur une automatisation opaque.

2.4 Environnement de développement et technique

2.4.1 Outils de gestion et de documentation

L'environnement de travail du projet s'appuie sur les artefacts présents dans le dépôt :

- le code applicatif `Next.js` et `TypeScript` ;
- la documentation d'architecture et les prompts techniques conservés dans `ARCHITECTURE.md` et `docs/` ;
- les sources du rapport en Markdown dans `rapport/` et leur version $\text{\LaTeX}$ dans `overleaf/` ;
- les diagrammes du mémoire stockés dans `overleaf/figures/diagrams`.

Cette organisation garde le rapport directement rattaché au projet réel : les choix décrits proviennent des fichiers, scripts, services, schémas et composants effectivement présents dans le dépôt.

2.4.2 Environnement d'exécution et de développement

Le projet est développé dans un environnement JavaScript / TypeScript moderne. `Bun` est utilisé comme runtime principal, gestionnaire de dépendances et orchestrateur de scripts [10]. `Next.js16` fournit le socle full-stack de développement et d'exécution [7]. L'environnement local repose sur `bundev`, tandis que les scripts de base de données, de seed, de réindexation, de benchmark et d'évaluation utilisent également Bun.

Ce choix simplifie l'outillage : un même runtime permet d'exécuter l'application, les scripts techniques, les tests et les tâches liées à l'IA, sans multiplier les gestionnaires de paquets ni les conventions d'exécution.

2.4.3 Outils front-end

Le front-end du projet repose sur les technologies suivantes :

- `Next.js16` avec App Router pour le rendu, les layouts et l'orchestration des routes [7] ;
- `React` pour la composition de l'interface [8] ;
- `TypeScript` en mode strict pour la sécurité de typage [9] ;
- `TailwindCSSv4` et `shadcn/ui` pour les composants et le design system [11][12] ;
- `@tabler/icons-react` pour l'iconographie ;
- `ReactHookForm` pour les formulaires complexes ;
- `TanStackQuery` lorsque le cache client est utile ;
- `Recharts` pour les visualisations analytiques ;
- `FramerMotion` pour certaines micro-interactions.

Ce socle front-end répond au besoin de construire des interfaces riches mais maintenables. Les dashboards TA, Manager, HR et Admin doivent rester lisibles tout en exposant des fonctionnalités avancées comme l'analytique, les assistants IA et les vues de pipeline.

2.4.4 Outils back-end

Le back-end est intégré dans la même application full-stack, mais organisé de manière stricte :

- les routes `Next.js` gèrent les points d'entrée HTTP et le streaming SSE ;
- les Server Actions servent de couche de mutation contrôlée ;
- les services métier concentrent la logique de traitement et les accès aux données ;

- `better-auth` gère l'authentification, les sessions et les rôles [13];
- `Zod` valide les entrées utilisateur et les arguments des outils agentiques [18];
- `Nodemailer` assure l'envoi des emails d'entretien et de décision;
- le SDK compatible `OpenAI` est utilisé avec des fournisseurs IA externes pour les appels LLM et embeddings;
- `Vitest` couvre les tests automatisés observables du projet [19].

Cette organisation permet de mélanger logique métier classique et logique IA sans éclater le projet en sous-systèmes difficiles à maintenir.

2.4.5 Architecture de la base de données

La persistance s'appuie sur `NeonPostgreSQL` et `DrizzleORM` [15][16][14]. Ce choix permet de garder dans la même base :

- les données métier du recrutement : utilisateurs, offres, CV, candidats, entretiens et rapports;
- les données conversationnelles du chat IA;
- les journaux d'activité, notifications et logs d'emails;
- l'historique des transitions candidat via `candidate_stage_history`;
- les confirmations d'actions IA via `pending_agent_actions`;
- les tâches d'onboarding;
- les embeddings vectoriels via `pgvector`.

L'usage de `pgvector` évite l'introduction d'une base vectorielle externe [17]. Les embeddings globaux des CV sont stockés dans le vivier, tandis que les embeddings orientés RAG sont stockés dans les chunks. Ces chunks sont aussi indexés en recherche plein texte, ce qui rend possible une recherche hybride combinant récupération vectorielle et récupération lexicale. Les tables `candidate_stage_history` et `pending_agent_actions` renforcent la gouvernance du modèle. La première conserve les mouvements du pipeline avec l'étape précédente, la nouvelle étape, l'acteur, la raison, la source et l'horodatage. La seconde matérialise les actions IA sensibles avant exécution, avec leur statut, leur expiration et leur résultat d'exécution. Ces deux tables transforment les opérations critiques en preuves consultables et exportables.

2.4.6 Justification synthétique des choix technologiques

Les technologies retenues ne sont pas uniquement des préférences de développement. Elles répondent à des contraintes mesurables du projet : rapidité d'itération, typage strict, gouvernance des données RH, capacité de streaming pour l'agent, recherche hybride et maintien d'un périmètre déployable par une petite équipe. Le tableau 2.6 synthétise cette logique de décision.

Table 2.6 : Justification des principaux choix technologiques

Choix retenu	Alternatives possibles	Justification pour le projet	Risque maîtrisé
Next.js 16 App Router	SPA séparée, API backend isolée	Unifie rendu, routes, actions serveur et streaming SSE dans un même produit full-stack.	Réduit la dispersion entre front-end, API et logique d'orchestration.
TypeScript strict	JavaScript simple	Rend explicites les contrats entre composants, services, actions et outils agentiques.	Diminue les erreurs d'intégration dans les workflows multi-rôles.
Bun	npm, yarn, pnpm, Node scripts dispersés	Accélère les scripts locaux et garde une convention unique pour développement, tests, seed et évaluation.	Évite la fragmentation de l'outillage.
Architecture feature-driven	Découpage par couches techniques ou microservices	Aligne chaque module sur une capacité métier : accès, vivier, jobs, pipeline, IA, gouvernance.	Évite un microservice prématuré et maintient une cohérence transactionnelle.
Better Auth + RBAC	Authentification maison	Fournit sessions et rôles sans réimplémenter une couche sensible.	Limite les failles liées à une authentification artisanale.
Zod	Validation implicite ou types compile-time seuls	Valide les entrées utilisateur et les arguments d'outils à l'exécution.	Empêche l'agent ou les formulaires de déclencher des mutations mal formées.
Neon PostgreSQL + Drizzle pgvector	Base NoSQL ou ORM plus opaque	Garde données RH, logs, conversations, états et embeddings dans un modèle relationnel typé.	Préserve traçabilité, cohérence et requêtage analytique.
	Base vectorielle externe	Ajoute la similarité sémantique sans séparer les embeddings des règles métier et d'accès.	Réduit la complexité d'infrastructure et les risques de divergence des droits.
Tailwind CSS + shadcn/ui	CSS ad hoc ou bibliothèque fermée	Combine rapidité d'interface, accessibilité des primitives et personnalisation visuelle.	Évite de reconstruire des composants interactifs sensibles comme les dialogues.

Cette matrice rend explicite le critère principal de conception : préférer une architecture intégrée, typée et gouvernée à une architecture plus distribuée mais plus coûteuse à fiabiliser. Ce choix est cohérent avec la nature du projet : une plateforme de recrutement interne où la cohérence des données, la sécurité des rôles et la traçabilité des décisions priment sur une scalabilité horizontale prématurée.

Conclusion

Le produit est désormais défini à trois niveaux : les acteurs, les besoins et la structure technique qui permet d'y répondre. Les chapitres suivants décrivent la réalisation concrète de cette architecture, en commençant par le socle d'accès, d'authentification et d'orientation par rôle.

Chapitre 3

Module 1 : accès, identité et points d'entrée par rôle

Introduction

Ce premier module pose le socle d'usage de toute la plateforme. Son rôle ne se limite pas à afficher une page d'accueil et un formulaire de connexion : il formalise la frontière entre l'espace public et les espaces métiers, sécurise l'accès aux données de recrutement et prépare l'injection des fonctionnalités transverses comme le chat analytique. Sans cette couche, les modules de gestion des CV, du pipeline candidat et de l'IA ne pourraient pas être exposés de manière fiable dans un contexte d'entreprise.

Dans Capgemini Talent Intelligence, l'identité n'est pas un sujet périphérique. Elle détermine les données visibles, les actions autorisées, la navigation initiale et même le périmètre de l'agent conversationnel. Ce chapitre doit donc être lu comme la mise en place du premier contrat d'usage du système : entrer dans la plateforme, être reconnu, être orienté vers le bon espace et être bloqué dès que l'on sort du périmètre autorisé.

3.1 Sprint 1 : objectif et périmètre

3.1.1 Objectif du sprint

L'objectif principal est de construire la couche d'accès du produit. Dans le contexte de Capgemini Talent Intelligence, cela signifie :

- proposer une landing page publique qui présente clairement la proposition de valeur ;
- authentifier les utilisateurs par email et mot de passe ;
- rediriger automatiquement chaque utilisateur vers le dashboard correspondant à son rôle ;
- protéger les routes, layouts et vues métiers ;

- fournir un premier niveau d'administration des comptes.

Cette séparation entre **TA**, **Manager**, **HR** et **Admin** n'est pas cosmétique. Elle est réutilisée ensuite dans les pages métiers, dans les Server Actions, dans les services IA et dans l'agent conversationnel, qui doivent tous respecter le périmètre de l'utilisateur connecté.

Au-delà de l'authentification, ce sprint vise aussi trois garanties durables : une confidentialité minimale des données RH, une cohérence de navigation selon le rôle métier et une extensibilité suffisante pour brancher les futurs modules sur un shell commun sans redéfinir la logique d'accès.

3.1.2 Backlog du sprint

Table 3.1 : Backlog fonctionnel du module d'accès et d'identité

ID	Thème	User Story	Statut
1	Marketing produit	En tant que visiteur, je peux consulter une landing page présentant la proposition de valeur de la plateforme.	Réalisé
2	Authentification	En tant qu'utilisateur autorisé, je peux me connecter avec mon email et mon mot de passe.	Réalisé
3	Redirection par rôle	En tant qu'utilisateur connecté, je suis redirigé vers mon espace TA, Manager, HR ou Admin.	Réalisé
4	Protection des routes	En tant qu'utilisateur non authentifié ou non autorisé, je suis bloqué hors des espaces protégés.	Réalisé
5	Administration des comptes	En tant qu'administrateur, je peux consulter les utilisateurs, créer un compte, modifier un rôle et bannir ou débannir un utilisateur.	Réalisé
6	Shell de dashboard	En tant qu'utilisateur connecté, je bénéficie d'un cadre de navigation commun préparant l'accès aux modules métier et au chat analytique.	Réalisé

Ce backlog montre déjà que l'identité traverse toute la plateforme. Elle ne se limite pas au formulaire de connexion ; elle englobe l'interface publique, la redirection, la protection des routes, la structure des dashboards et la gouvernance des comptes.

3.2 Implémentation du sprint

3.2.1 Analyse du sprint

Le comportement du module repose sur quelques briques fonctionnelles fondamentales :

- la landing page publique ;
- l'écran de connexion ;
- le module d'authentification, de récupération de session et de contrôle d'accès par rôle ;
- le layout protégé des espaces dashboard ;
- le shell partagé des espaces métier ;
- l'interface d'administration des utilisateurs ;
- la première vue métier du tableau de bord TA branchée sur ce socle.

Ce module ne doit donc pas être lu comme un simple travail de design d'interface. Il formalise le premier contrat d'usage du système : qui peut entrer, où il atterrit, et à quel périmètre il a droit.

D'un point de vue architectural, le sprint combine trois niveaux de contrôle complémentaires : la présentation publique et privée, la vérification serveur de la session et des rôles, puis la gouvernance des comptes côté administration. Cette superposition réduit le risque qu'une protection repose uniquement sur le client.

3.2.2 Vue fonctionnelle globale

Les usages couverts par ce sprint se regroupent en quatre blocs :

- consulter la page d'accueil publique ;
- se connecter ;
- être redirigé vers un espace adapté au rôle ;
- administrer les comptes si l'utilisateur est **admin**.

Le layout de dashboard prépare déjà l'usage futur de l'IA à deux niveaux : un assistant flottant disponible dans les espaces métier et un workspace dédié **/agent** pour les analyses longues. L'entrée conversationnelle de la plateforme a donc été pensée dès le départ comme une fonctionnalité authentifiée, cloisonnée par rôle et intégrée à la navigation latérale. Ce point est important : l'IA n'est pas ajoutée plus tard dans un espace parallèle. Elle s'inscrit dès les fondations dans la même logique d'authentification, de layout et de séparation des rôles que le reste du produit, tout en offrant un parcours hybride entre aide contextuelle rapide et espace d'analyse approfondie.

3.2.3 Authentification et redirection par rôle

L'écran de connexion contient la logique de connexion. Lors de la soumission du formulaire, la fonction `handleSubmit()` appelle `authClient.signIn.email()` avec l'email et le mot de passe. Si la connexion réussit, le rôle utilisateur est lu depuis la réponse, puis une table de correspondance détermine la destination :

- `ta` → `/ta/dashboard`;
- `manager` → `/manager/dashboard`;
- `hr` → `/hr/dashboard`;
- `admin` → `/admin`.

La même logique existe côté serveur grâce à `getRoleHome()` et `requireRole()`, ce qui évite que la sécurité repose uniquement sur le client.

Le fait de disposer d'un équivalent serveur est essentiel. Même si l'interface connaît la destination à prendre, la plateforme ne fait pas confiance à la seule navigation du navigateur. Les routes sensibles restent protégées par une lecture réelle de la session et des rôles.

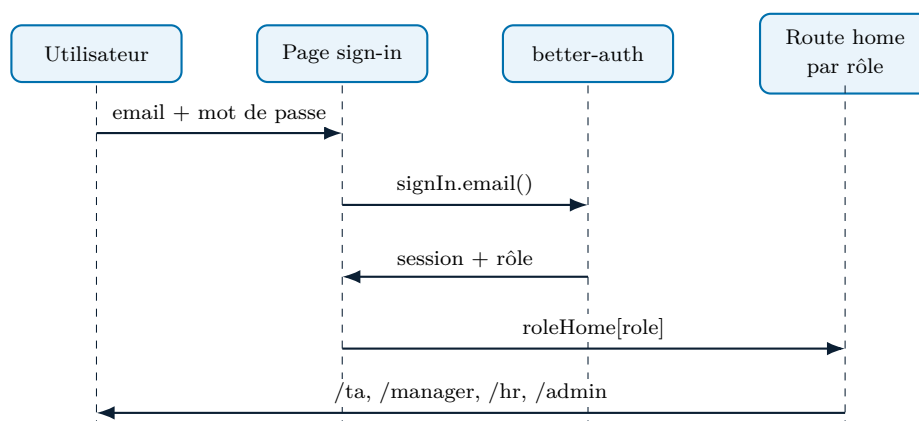


Figure 3.1 : Séquence d'authentification et de redirection par rôle

3.2.4 Protection du shell et contrôle d'accès

Lorsqu'un utilisateur accède à un espace protégé, le layout principal du dashboard vérifie la présence d'une session via `getSession()`. Si aucune session n'existe, une redirection vers `/sign-in` est déclenchée. Sinon, le layout récupère le rôle et le nom de l'utilisateur, les injecte dans `DashboardShell` et monte aussi `StatisticsChat`. Ainsi, toutes les pages métier utilisent un conteneur commun déjà authentifié.

Cette centralisation apporte trois bénéfices majeurs. D'une part, elle évite de dupliquer la logique d'accès dans chaque page. D'autre part, elle garantit une expérience homogène pour tous les rôles. Enfin, elle prépare les futurs composants transverses à s'exécuter dans le même contexte de sécurité que les vues métiers.

3.2.5 Administration des comptes

Sur la page d'administration, le composant `AdminUsersClient` exploite le plugin `adminClient()` de Better Auth côté client. Il permet de :

- lister les utilisateurs avec `authClient.admin.listUsers()` ;
- créer un utilisateur avec `authClient.admin.createUser()` ;
- modifier un rôle avec `authClient.admin.setRole()` ;
- bannir un utilisateur avec `authClient.admin.banUser()` ;
- débannir un utilisateur avec `authClient.admin.unbanUser()`.

Cette partie montre que l'identité n'est pas seulement un écran de login. Elle inclut déjà une gouvernance minimale du cycle de vie des comptes, indispensable dans un environnement B2B où la création de comptes est pilotée par l'organisation et non par de l'auto-inscription.

La possibilité de bannir ou débannir un compte apporte également une souplesse de gouvernance importante. Elle permet de suspendre rapidement un accès sans perturber toute la structure des rôles ni supprimer brutalement l'utilisateur du système.

Scénarios de validation fonctionnelle. La cohérence du module d'accès peut être vérifiée par trois scénarios simples. Premièrement, un utilisateur authentifié doit être redirigé vers l'espace correspondant à son rôle. Deuxièmement, un utilisateur sans session doit être renvoyé vers la page de connexion lorsqu'il tente d'accéder à un dashboard protégé. Troisièmement, les actions d'administration des comptes doivent rester réservées au rôle `admin`. Ces scénarios matérialisent le contrat principal du sprint : aucune donnée métier sensible ne doit être exposée sans session ni rôle approprié.

3.3 Réalisation

3.3.1 Landing page publique

La landing page publique positionne la plateforme comme un moteur de recrutement intelligent. Elle met en avant plusieurs axes : le matching intelligent, l'automatisation des workflows, l'analytique prédictive, la collaboration d'équipe et la sécurité d'entreprise. Des appels à l'action comme *Sign In*, *Access Platform* et *Sign In to Continue* orientent immédiatement l'utilisateur vers le point d'entrée authentifié.

Cette page joue aussi un rôle de clarification stratégique. Avant même la connexion, elle pose le positionnement du produit comme une plateforme de recrutement augmentée par l'IA et non comme un simple portail de candidatures.

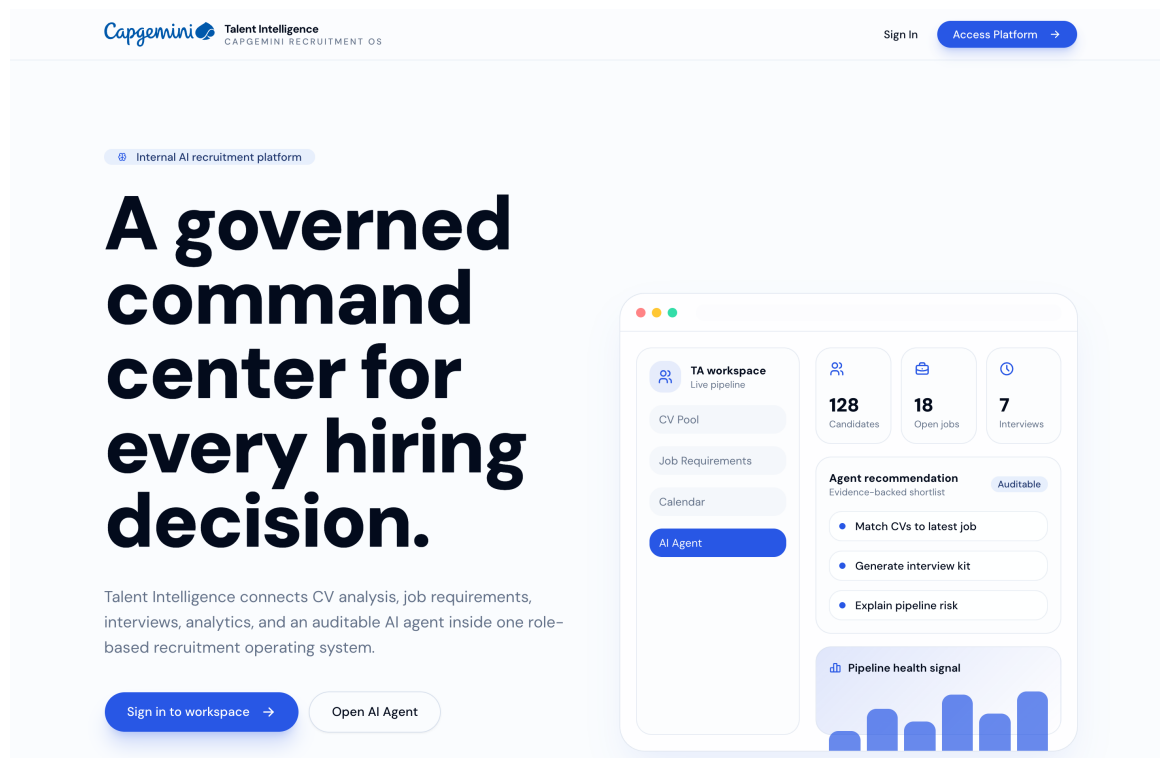


Figure 3.2 : Page d'accueil publique de la plateforme.

3.3.2 Page de connexion

La page de connexion propose un formulaire simple et robuste : champ email, champ mot de passe, affichage ou masquage du mot de passe, état de chargement, message d'erreur et lien *Forgot password ?*. La phrase *Account creation is managed by your administrator* est particulièrement significative : elle confirme que le produit n'est pas pensé comme un portail ouvert, mais comme une plateforme d'entreprise gouvernée.

Sur le plan UX, le formulaire reste volontairement direct. L'objectif est de réduire l'ambiguïté au moment de l'entrée dans la plateforme et de rendre visible le caractère administré de l'accès.

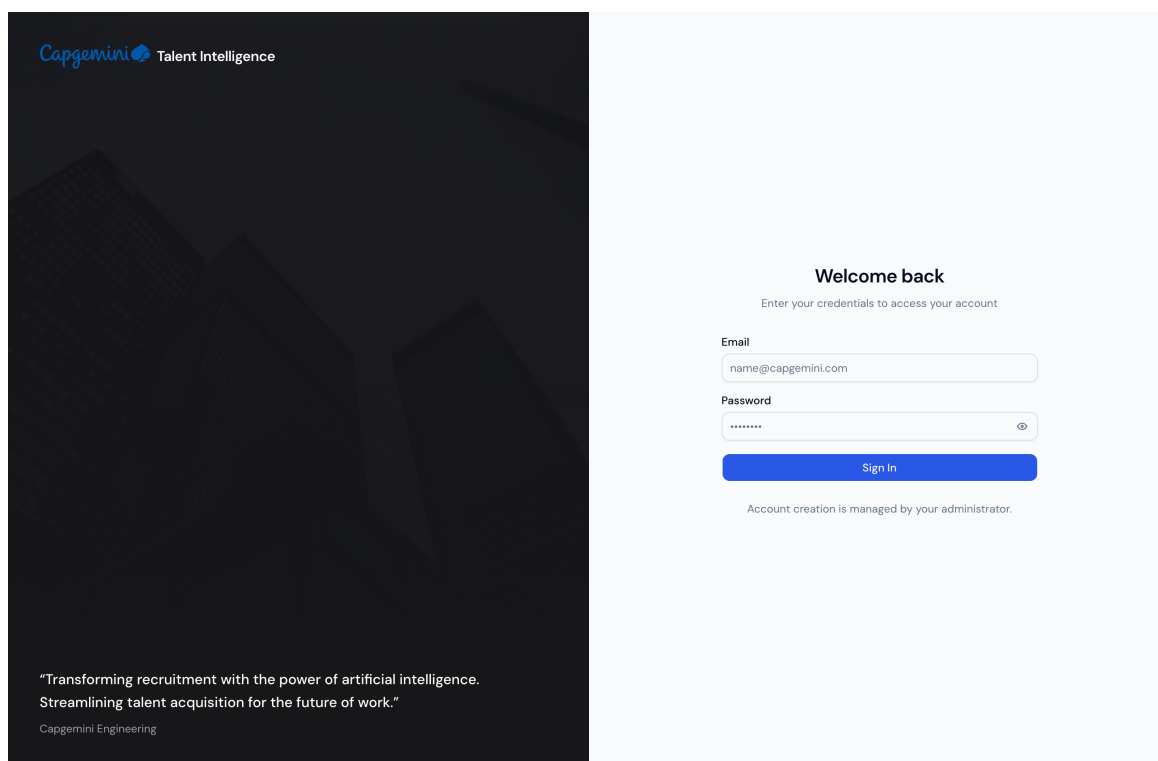


Figure 3.3 : Page de connexion sécurisée de la plateforme.

3.3.3 Shell de dashboard partagé

Le `DashboardShell` constitue un apport structurant du sprint. Il mutualise la navigation latérale, l'en-tête, le contexte utilisateur et l'enveloppe commune de rendu des pages métier. Grâce à ce shell, les modules suivants peuvent s'intégrer progressivement sans reconstruire à chaque fois les fondations de navigation et de sécurité.

En pratique, ce shell joue le rôle de charnière entre sécurité et productivité. Il offre un cadre stable pour toutes les vues TA, Manager, RH et Admin, tout en garantissant qu'elles partagent les mêmes conventions d'identité et de navigation.

3.3.4 Première vue métier : tableau de bord TA

La page de tableau de bord TA illustre la première exploitation de ce socle. Elle exige le rôle `ta` ou `admin`, puis charge les statistiques globales via `getDashboardStatsAction()` et le planning d'entretiens du jour via `getTodayInterviewScheduleAction()`. Ce tableau de bord montre que le sprint ne s'arrête pas à l'identité : il ouvre déjà la porte au pilotage métier réel.

Ce point est important pour la lecture du rapport : l'accès protégé n'est pas une fin en soi. Il débouche immédiatement sur une valeur métier visible, ce qui prouve que le socle construit est déjà exploitable dans l'usage quotidien.

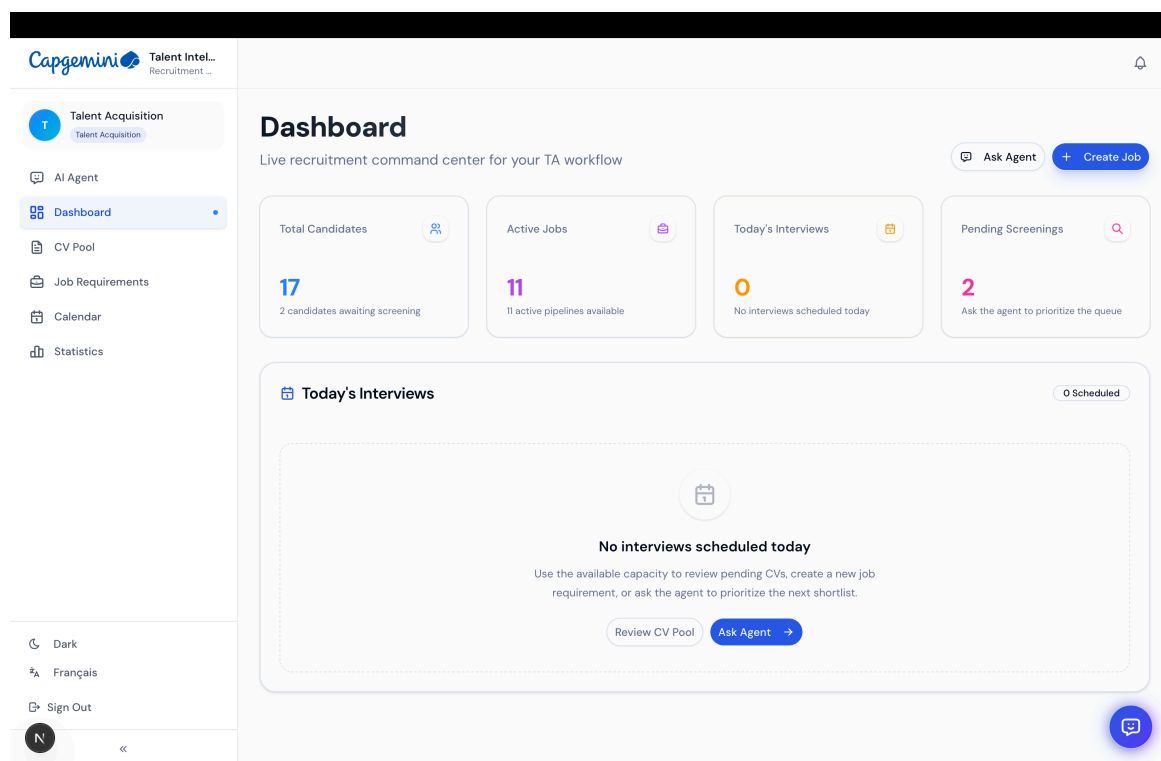


Figure 3.4 : Tableau de bord TA après authentification.

3.3.5 Gestion des utilisateurs côté administration

La page d'administration des utilisateurs affiche un espace *User Management* avec le sous-titre *Manage user accounts and roles*. Le composant `AdminUsersClient` rend cette promesse effective à travers la consultation de la liste des utilisateurs, la création d'un compte, la modification du rôle et le bannissement ou débannissement.

Cette fonctionnalité matérialise un principe de responsabilité : dans une plateforme RH, les utilisateurs autorisés à voir ou manipuler les candidatures doivent eux-mêmes être gérés de manière explicite et traçable.

3.4 Apport du module

Ce premier module apporte plusieurs fondations durables au projet :

- un point d'entrée public compréhensible pour le produit ;
- une authentification alignée sur un usage d'entreprise ;
- une séparation explicite par rôle, réutilisée ensuite dans les routes, les services et l'IA ;
- un shell commun qui accélère tous les développements suivants ;
- un environnement authentifié, contrôlé et traçable pour les futurs usages analytiques.

Autrement dit, même si ce module ne contient pas encore le cœur du pipeline RAG, il construit l'infrastructure d'usage sans laquelle une IA de recrutement ne pourrait pas être exposée de manière sûre.

On peut donc le considérer comme la fondation de confiance du produit. Sans cette base, les modules suivants auraient pu exister techniquement, mais pas comme un système crédible dans un cadre professionnel exigeant.

Conclusion

Ce premier module établit les fondations d'accès et d'identité de Capgemini Talent Intelligence. Il transforme une simple interface publique en point d'entrée vers une plateforme gouvernée par les rôles, protégée côté serveur et prête à accueillir les fonctionnalités métier et IA des modules suivants. Le chapitre suivant s'appuie sur ce socle pour traiter le premier cœur documentaire et opérationnel du produit : le vivier de CV et la gestion des offres d'emploi.

Chapitre 4

Module 2 : vivier de CV et gestion des offres d'emploi

Introduction

Ce deuxième module matérialise le premier cœur opérationnel du produit. Après avoir sécurisé l'accès et structuré les rôles, il fallait construire la base documentaire et métier sur laquelle reposera toute la suite de la plateforme. Dans Capgemini Talent Intelligence, cela signifie deux choses : constituer un vivier de CV exploitable et formaliser les besoins de recrutement à travers des offres structurées. Sans ce module, il n'y aurait ni pipeline candidat fiable, ni matching pertinent, ni recherche sémantique, ni pipeline RAG réellement utile.

Ce chapitre marque le passage d'un système simplement authentifié à un système capable de manipuler une matière métier réelle. Les CV sont hétérogènes, souvent peu structurés et parfois redondants. Les offres doivent quant à elles rester lisibles pour les recruteurs tout en devenant suffisamment normalisées pour servir aux futurs traitements IA.

4.1 Sprint 2 : objectif et périmètre

4.1.1 Objectif du sprint

L'objectif de ce sprint est de transformer deux objets bruts en objets intelligibles par le système :

- le CV, qui arrive sous forme de document PDF ou DOCX ;
- l'offre d'emploi, qui doit être exprimée de manière suffisamment structurée pour être comprise à la fois par les recruteurs et par les services IA.

Le sprint met donc en place une page de gestion du vivier, l'import de CV avec extraction automatique, l'indexation sémantique, la détection des doublons, la recherche et

l'export du vivier, la création et la gestion des offres ainsi que la préparation du matching et du futur pipeline candidat.

Ce module est stratégique parce qu'il constitue la jonction entre l'interface et l'intelligence. C'est à partir d'ici que les données commencent à devenir une matière première exploitable pour le matching, le RAG et l'agent conversationnel.

4.1.2 Backlog du sprint

Table 4.1 : Backlog fonctionnel du module vivier de CV et offres

ID	Thème	User Story	Statut
7	CV Pool	En tant que TA, je peux importer et gérer des CV dans un vivier centralisé.	Réalisé
8	Extraction IA	En tant que TA, je peux obtenir automatiquement les informations structurées d'un CV importé.	Réalisé
9	Recherche	En tant que TA, je peux rechercher des profils par texte, compétences et signaux sémantiques.	Réalisé
10	Détection de doublons	En tant que TA, je peux identifier des doublons potentiels dans le vivier.	Réalisé
11	Export documentaire	En tant que TA, je peux exporter un ou plusieurs CV depuis le vivier.	Réalisé
12	Gestion des offres	En tant que TA, je peux créer et visualiser des offres d'emploi structurées.	Réalisé
13	Templates d'offres	En tant que TA, je peux réutiliser une offre comme modèle.	Réalisé
14	Préparation du matching	En tant que TA, je peux relier les CV et les offres pour préparer le matching et l'affectation au pipeline.	Réalisé

La composition du backlog montre bien que ce sprint ne se limite pas à stocker des fichiers. Il couvre déjà la qualité documentaire, la recherche, l'export, la structuration des besoins et la préparation des futurs traitements intelligents.

4.2 Implémentation du sprint

4.2.1 Analyse du sprint

Le sprint s'appuie principalement sur les éléments fonctionnels suivants :

- la page du vivier de CV ;

- l'interface de gestion et de consultation des profils ;
- la file d'upload en arrière-plan ;
- le service d'ingestion, d'extraction et de recherche des CV ;
- le service de détection des doublons ;
- le service de génération des embeddings ;
- le service de découpage des CV en chunks orientés RAG ;
- la page de gestion des offres ;
- l'interface de création et de listing des offres ;
- le service métier de gestion des offres ;
- la page de détail d'une offre et l'espace de pilotage associé au matching.

Ce module construit simultanément le patrimoine documentaire RH et la représentation structurée de la demande de recrutement. Il constitue donc la vraie base de travail des modules d'IA.

Il installe aussi un principe de robustesse important : l'import principal ne doit pas être bloqué par le temps de calcul des traitements sémantiques secondaires. Le système préfère livrer rapidement un résultat utile, puis compléter l'enrichissement en arrière-plan.

Qualité des données et transformation documentaire. L'un des enjeux majeurs du sprint concerne la qualité de la donnée d'entrée. Un CV importé peut être rédigé dans des styles très différents, contenir une mise en page complexe, mélanger plusieurs langues ou ne pas exposer les mêmes champs d'information qu'un autre document. Le système doit donc absorber une forte hétérogénéité entre PDF et DOCX, tout en acceptant qu'une extraction reste parfois partielle lorsque certaines informations sont absentes, ambiguës ou difficilement récupérables automatiquement.

Cette contrainte explique plusieurs choix du module : conservation du texte brut, extraction progressive, enrichissement différé, détection des doublons et séparation entre stockage principal et indexation sémantique. Dans la logique du rapport, ce point est essentiel, car la qualité du matching et du RAG dépend directement de la qualité documentaire initiale. Une base bruitée, redondante ou mal structurée affaiblit mécaniquement la pertinence des traitements IA qui s'appuient ensuite sur elle.

Tableau de transformation documentaire. Le passage du document brut à l'objet métier exploitable peut être résumé ainsi :

Table 4.2 : Transformation documentaire du CV et de l'offre

Entrée brute	Traitement	Sortie métier
CV PDF ou DOCX hétérogène	Parsing du fichier et extraction du texte brut	Document lisible par le système
Texte brut du CV	Extraction structurée des signaux métier : identité, compétences, expériences, formation, langues, résumé	Profil exploitable dans le vivier
Profil extrait	Vérification de doublons, normalisation partielle et enrichissement progressif	Donnée documentaire plus fiable
Profil enrichi	Génération d'un embedding global	Support de recherche sémantique directe
Sections du profil	Découpage en chunks thématiques, embeddings par chunk et index lexical	Support du retrieval hybride et du futur RAG
Offre structurée	Validation, normalisation des critères et stockage métier	Objet comparable avec les profils du vivier

Ce tableau montre que le CV n'est plus un simple fichier et que l'offre n'est plus un simple formulaire. Tous deux deviennent des objets métier progressivement transformés pour être réutilisables par les recruteurs, comparables par les services de matching et mobilisables par les couches RAG et agentiques.

4.2.2 Vue fonctionnelle globale

Les usages couverts par ce sprint se regroupent en trois blocs.

1. **Gestion des CV** : importer un document, parser son contenu, extraire les informations principales, stocker le profil dans le vivier, le consulter, le rechercher, le supprimer et l'exporter.
2. **Gestion des offres** : créer une offre, la décrire avec ses contraintes métier, la réutiliser comme modèle et la fermer de manière contrôlée.
3. **Préparation de l'intelligence de recrutement** : générer des embeddings, découper les CV en chunks, rendre possible la recherche sémantique et le RAG, puis préparer la relation entre un CV, une offre et un futur candidat.

Cette structuration montre que la plateforme traite à la fois l'offre et le profil comme deux objets complémentaires devant être rendus comparables par la machine sans perdre leur lisibilité métier.

4.2.3 Pipeline d'upload et d'enrichissement d'un CV

Le flux d'upload est l'un des plus importants du sprint. Il passe par `uploadCvAction()` puis par les services du module `cv-pool`. La logique est conçue pour que l'ingestion principale ne soit pas bloquée par les traitements avancés. La séquence peut être résumée ainsi :

- le fichier est envoyé via la file d'upload côté client ;
- le service enregistre le CV dans `cv_pool` ;
- le document est parsé selon son format ;
- l'IA extrait les informations structurées ;
- les champs métier sont mis à jour ;
- la génération de l'embedding est déclenchée en arrière-plan ;
- la génération des chunks pour le RAG est déclenchée en arrière-plan ;
- la vérification de doublons est exécutée ;
- l'interface se rafraîchit avec le résultat.

Cette architecture traduit un choix de robustesse : l'utilisateur n'attend pas que tout le pipeline IA soit terminé pour bénéficier de l'import principal. Elle traduit aussi un choix de qualité progressive : le CV devient d'abord exploitable, puis s'enrichit ensuite avec une profondeur sémantique plus fine.

Lecture opérationnelle du parcours. Un parcours représentatif illustre l'apport du module : un recruteur dépose un CV, le système extrait d'abord le texte et les métadonnées principales, puis le profil devient consultable dans le vivier. Les traitements plus lourds, comme l'embedding et le découpage en chunks, enrichissent ensuite le document pour les usages de recherche sémantique et de RAG. Cette séparation entre disponibilité rapide et enrichissement progressif évite de confondre ingestion documentaire et calcul IA complet.

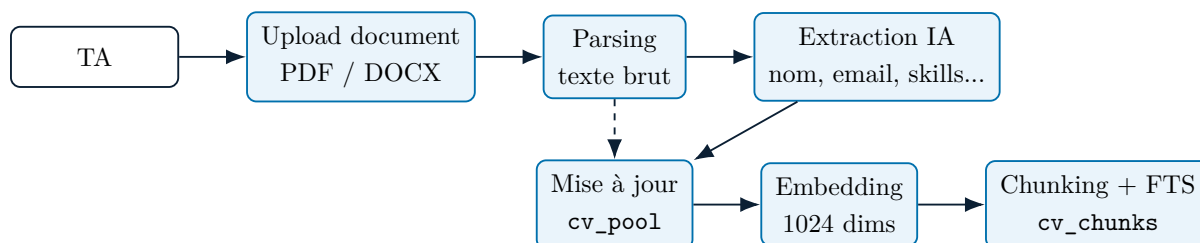


Figure 4.1 : Séquence d'upload, d'extraction et d'indexation d'un CV

4.2.4 Indexation sémantique et préparation du RAG

Après extraction, le projet génère deux formes d'indexation : un embedding global du CV dans `cv_pool` et un ensemble de chunks spécialisés dans `cv_chunks`. L'embedding global sert à la recherche sémantique directe sur les profils. Les chunks, eux, servent à une granularité plus fine adaptée au RAG.

Le service `chunking.ts` découpe notamment les informations selon plusieurs sections : expérience, compétences, formation, résumé et langues. Chaque chunk reçoit ensuite son propre embedding et un index `tsvector`, ce qui prépare la récupération hybride combinant similarité vectorielle et recherche lexicale.

Cette séparation entre représentation globale et représentation fragmentée est centrale. Elle permet à la fois de retrouver un profil proche d'une intention de recherche, puis de justifier ce rapprochement à l'aide d'extraits précis et citables.

Différenciation des niveaux de recherche et d'indexation. Pour clarifier la logique du module, il est utile de distinguer cinq niveaux complémentaires :

- la recherche simple, fondée sur des champs directement lisibles comme le nom, l'email, le fichier ou certaines compétences ;
- la recherche sémantique, qui s'appuie sur l'embedding global du CV pour rapprocher des profils au-delà des correspondances littérales ;
- le chunking, qui ne sert pas à rechercher mieux un profil complet, mais à découper son contenu en unités documentaires plus précises ;
- la préparation du RAG, qui exploite ces chunks, leurs embeddings et leur index lexical pour récupérer des passages pertinents avec citations ;
- le retrieval hybride, qui combine les signaux lexicaux et vectoriels afin de limiter les angles morts d'une recherche purement textuelle ou purement sémantique.

Autrement dit, l'embedding global répond surtout à la question : quel profil ressemble à cette intention de recherche ? Les chunks répondent plutôt à la question : quels passages précis du profil justifient cette réponse ? Cette distinction est structurante pour tout ce qui suivra dans le matching, le RAG et l'agent conversationnel.

4.2.5 Création d'une offre et affectation vers le pipeline candidat

La création d'une offre passe par `createJobAction()` puis `createJob()`. Les données sont validées avant insertion. L'offre conserve une structure explicite : titre, description, compétences indispensables, compétences souhaitables, séniorité, business unit, statut et indicateur de template.

Même si le cœur du pipeline candidat est traité dans le chapitre suivant, ce module prépare déjà le lien métier principal : le passage de *CV dans le vivier* à *candidat dans une offre*. Cette transition transforme un actif documentaire en objet de pipeline.

Elle rend aussi possible un futur matching crédible, car la plateforme dispose désormais de deux objets structurés pouvant être comparés selon des critères métier et sémantiques.

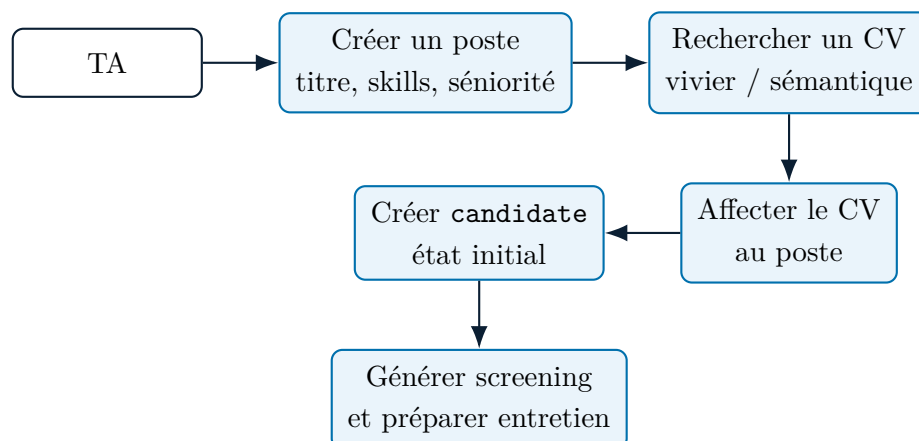


Figure 4.2 : Création d'un poste et affectation d'un CV au pipeline candidat

4.3 Réalisation

4.3.1 Page *CV Pool*

La page du vivier de CV charge la liste des CV ainsi que les statistiques du vivier via `listCvPoolAction()` et `getCvPoolStatsAction()`. Le composant `CvPoolClient` fournit une interface riche qui permet :

- l'upload par sélection ou glisser-déposer ;
- l'usage d'une file globale d'upload ;
- la recherche locale sur nom, email, fichier et compétences ;
- la sélection multiple ;
- la suppression unitaire ou par lot ;
- l'ouverture d'un dialogue de revue détaillée du CV extrait ;
- le téléchargement du fichier brut ;
- l'export individuel Excel et l'export multiple ZIP.

La page présente aussi un niveau de synthèse à travers `CvPoolStats` : nombre total de CV, top compétences, distribution des langues et tendance d'upload sur les sept derniers jours. La version actuelle va plus loin qu'une simple revue documentaire : le dialogue de consultation détaillée embarque désormais un bloc d'*evidence readiness* qui explicite les faits observés, les données manquantes et les risques documentaires avant toute action IA. Les actions *Explain CV with Agent* et *Draft Questions with Agent* ouvrent ensuite le workspace `/agent` avec un prompt prérempli qui réinjecte ce contexte de preuve plutôt

que de repartir d'une page blanche. Cette interface montre ainsi que le vivier n'est pas une simple liste de fichiers. C'est une base de profils analysés, consultables et réexploitables, avec une lecture à la fois opérationnelle, analytique et déjà branchée sur l'assistance agentique.

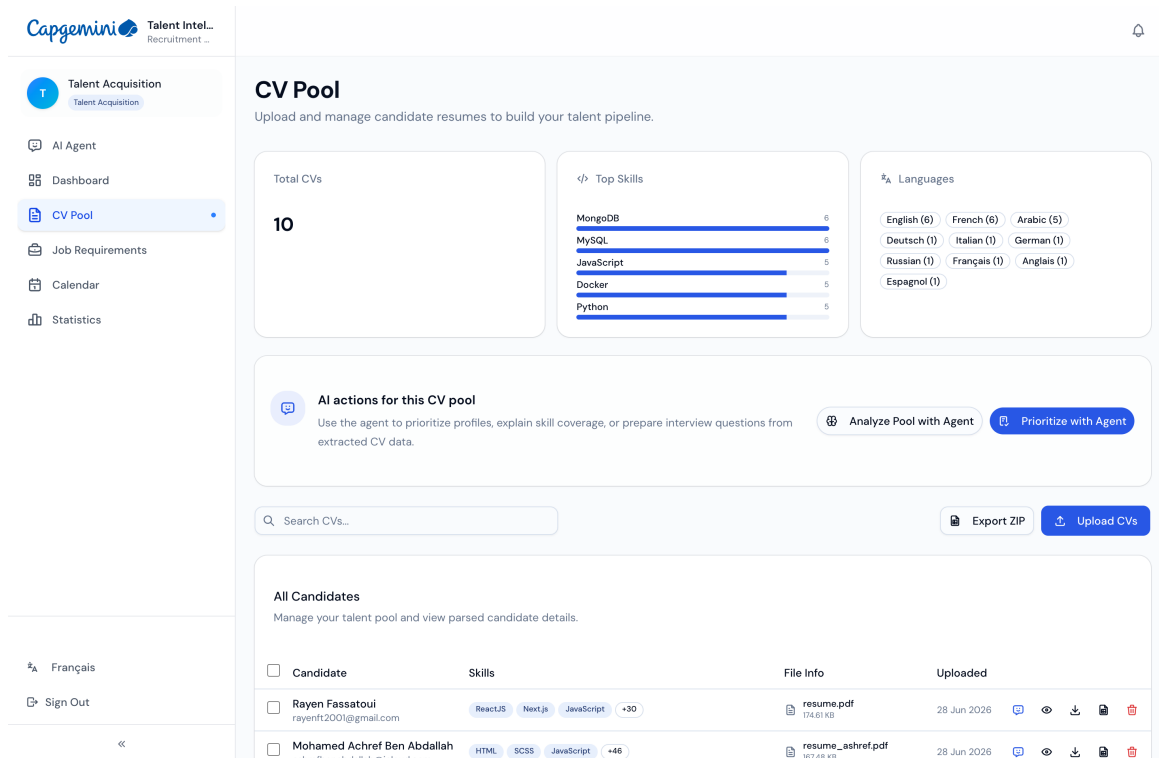


Figure 4.3 : Page de gestion du vivier de CV.

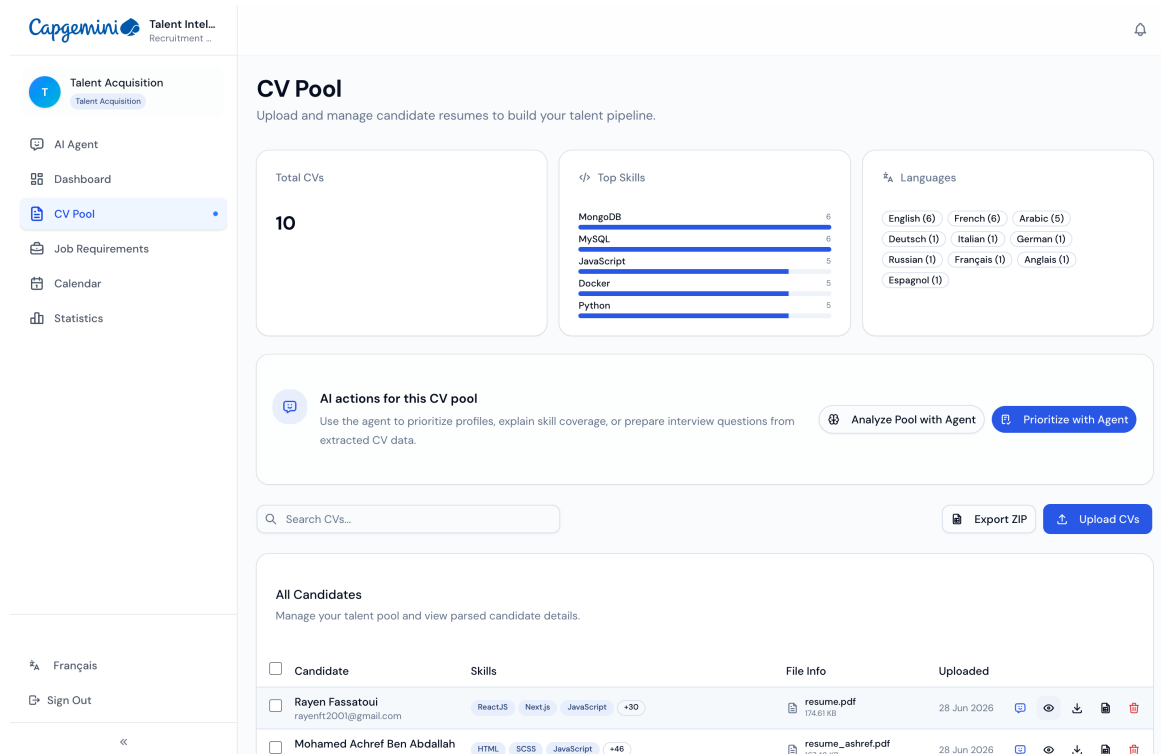


Figure 4.4 : Dialogue de consultation détaillée d'un CV.

4.3.2 Upload en arrière-plan et robustesse d'ingestion

Le composant `upload-provider.tsx` introduit une file d'upload avec états `queued`, `uploading`, `success` et `error`, validation de format, limite de taille, *retries* et *backoff* exponentiel. Cette logique améliore nettement l'expérience utilisateur et la robustesse du module, car elle absorbe les traitements asynchrones sans bloquer l'usage.

Elle répond à une contrainte très concrète : un recruteur peut déposer plusieurs CV à la suite et continuer à travailler pendant que l'enrichissement documentaire progresse en tâche de fond.

4.3.3 Détection de doublons

Le projet intègre un mécanisme de détection des doublons dans `duplicate-detection.ts`. Le service compare notamment l'email extrait, la similarité de nom et le numéro de téléphone. Cette fonctionnalité améliore la qualité documentaire du vivier et, par conséquent, la qualité du matching et de la recherche RAG qui s'appuient sur ces données.

La qualité de la base est ici un prérequis de la qualité algorithmique. Une couche IA branchée sur des profils redondants ou incohérents produirait mécaniquement des résultats moins fiables.

4.3.4 Page *Jobs*

La page de gestion des offres s'appuie sur `listJobsAction()` et `getJobsStatsAction()`. Le composant `JobsClient` combine des cartes statistiques, la liste des offres, un dialogue de création d'offre, la visualisation des statuts et des signaux sur la demande en compétences et en séniorité.

Les statistiques associées incluent le total des jobs, la répartition par séniorité, par statut, par business unit et les compétences les plus demandées. Cette page transforme les besoins recruteurs en objets comparables, mesurables et exploitables par les services de matching.

Elle joue aussi un rôle de normalisation : en imposant une structure cohérente à la formulation des postes, elle facilite les analyses globales et la réutilisation ultérieure dans les traitements IA.

L'offre d'emploi n'est donc pas traitée comme un simple formulaire administratif. Elle devient un objet de décision qui articule besoin métier, contraintes de recrutement et future lecture algorithmique. C'est précisément cette structuration qui permet ensuite de comparer de manière cohérente un poste, un profil, un screening et un résultat de matching.

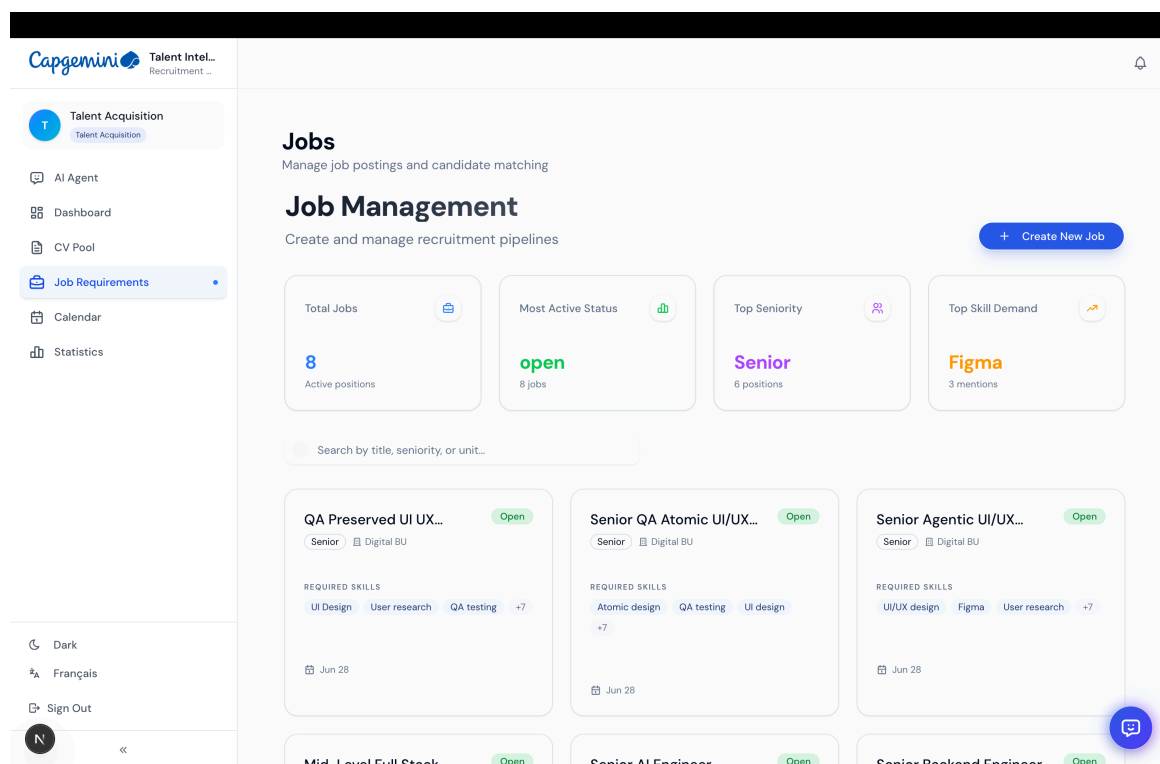


Figure 4.5 : Catalogue des offres d'emploi.

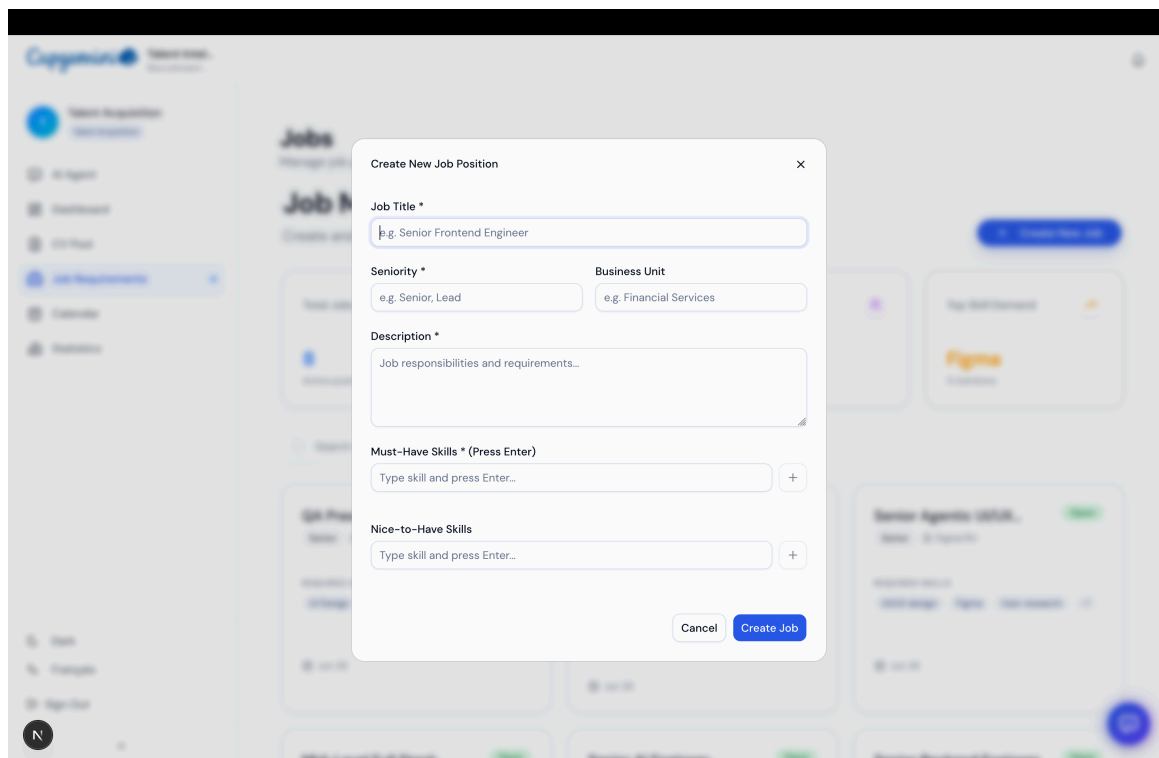
The image shows a web application interface with a modal window titled "Create New Job Position". The modal contains several input fields: "Job Title *" with a placeholder "e.g. Senior Frontend Engineer", "Seniority *" with a placeholder "e.g. Senior, Lead", and "Business Unit" with a placeholder "e.g. Financial Services". Below these is a "Description *" field with a placeholder "Job responsibilities and requirements...". At the bottom, there are two sections: "Must-Have Skills * (Press Enter)" and "Nice-to-Have Skills", each with a text input field and a "+" button. The modal has "Cancel" and "Create Job" buttons at the bottom right. The background shows a blurred view of the application's sidebar and main content area.

Figure 4.6 : Zone de création ou d'édition d'une offre d'emploi.

4.3.5 Templates et fermeture contrôlée des offres

Le service `jobs.ts` supporte l'enregistrement d'une offre comme template, la liste des templates, la création d'une offre à partir d'un template et la fermeture contrôlée d'un poste. Cette fermeture est sécurisée par des règles métier : elle est interdite s'il reste des entretiens planifiés ou des candidats encore actifs dans le pipeline. Cette règle protège la cohérence fonctionnelle du système.

Elle illustre une approche importante du projet : une action d'administration simple en apparence reste soumise à l'état réel du workflow métier.

4.3.6 Page détail d'une offre

La page de détail d'une offre charge l'offre, les candidats qui lui sont liés et la liste des managers. Le composant `JobDetailClient` transforme ensuite cette page en espace de pilotage avancé, avec génération de screening IA, consultation du score, génération et édition des questions d'entretien, planification, envoi d'emails, mise à jour des étapes candidat, assignation à un manager et matching *inline* de CV. L'interface distingue désormais explicitement les actions directes des actions de raisonnement IA. L'ouverture du matching de CV et celle des entretiens reposent sur des onglets pilotés par l'URL, ce qui rend les liens profonds robustes et permet au recruteur d'atterrir directement sur le bon sous-espace. À côté de cela, les entrées agentiques restent dédiées à l'explication du matching, à la revue d'exigences et à la synthèse raisonnée. Ce point montre que ce module n'est pas isolé. Le détail d'une offre devient le point de convergence entre la demande de recrutement, la

base documentaire des profils, les parcours directs du produit et les futurs mécanismes de décision.

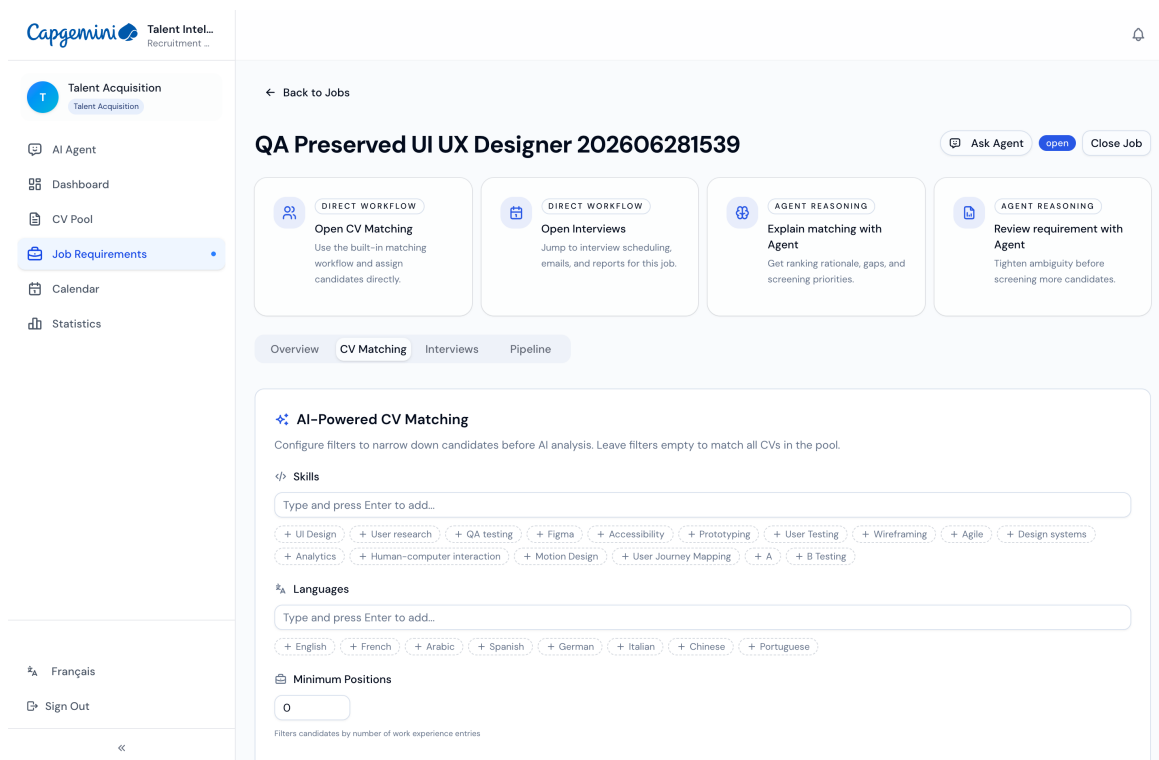


Figure 4.7 : Page détail d'une offre avec candidats associés et résultats de matching.

4.4 Apport du module

Ce module apporte une contribution majeure au projet :

- il transforme des documents bruts en objets structurés ;
- il crée la matière première nécessaire au matching et au RAG ;
- il structure la demande métier à travers des offres exploitables ;
- il améliore la qualité documentaire via la détection de doublons ;
- il prépare la relation fondamentale $CV \rightarrow offre \rightarrow candidat$;
- il installe les bases techniques de la recherche sémantique et de la récupération de contexte.

Dans la logique globale du rapport, ce module représente le moment où la plateforme commence réellement à devenir intelligente, car les données qu'elle manipule ne sont plus seulement visibles à l'écran : elles deviennent analysables, indexables et interrogeables par l'IA.

Il faut surtout retenir que cette intelligence n'apparaît pas magiquement dans le chapitre suivant. Elle devient possible ici, parce que la plateforme commence à transformer des fichiers bruts en structures de données propres, reliées et réexploitables.

Conclusion

Ce deuxième module a permis de construire le vivier documentaire et la base métier du recrutement. Les CV sont désormais importés, analysés, enrichis, exportables et progressivement indexés pour la recherche sémantique et le RAG. Les offres d'emploi sont quant à elles structurées de manière cohérente pour soutenir le matching et les décisions de recrutement. Le chapitre suivant s'appuiera sur cette base pour traiter la progression des candidats dans le pipeline et la coordination des entretiens.

Chapitre 5

Module 3 : pipeline candidat et gestion des entretiens

Introduction

Une fois les CV structurés et les offres définies, la valeur du produit se déplace vers le pilotage du processus de recrutement lui-même. Ce troisième module met en place la colonne vertébrale procédurale de Capgemini Talent Intelligence : faire progresser un candidat dans un pipeline explicite, assigner les bons évaluateurs au bon moment, planifier les entretiens, formaliser les décisions et préparer la transition vers l'onboarding.

Ce module est central dans l'architecture du projet, car il relie les objets documentaires du sprint précédent aux futurs apports IA. C'est ici que les changements d'état, les rapports, les affectations et les décisions deviennent traçables et réexploitables.

5.1 Sprint 3 : objectif et périmètre

5.1.1 Objectif du sprint

L'objectif est de transformer un CV affecté à une offre en un candidat piloté de bout en bout dans un workflow de recrutement multi-rôles. Il s'agit plus précisément de mettre en place :

- un pipeline candidat explicite ;
- l'assignation progressive des responsabilités entre TA, Manager et RH ;
- la planification des entretiens ;
- la saisie de rapports structurés ;
- la prise de décision à chaque étape ;
- la transition finale vers le statut **hired** et l'onboarding.

Cette partie est essentielle, car les screenings, les guides d’entretien, l’autopilot et les futurs *insights* n’ont de sens que s’ils s’insèrent dans un pipeline métier clairement défini.

Le sprint répond aussi à un enjeu de gouvernance humaine : plusieurs rôles interviennent successivement sur un même dossier, et chacun doit pouvoir agir dans un cadre clair sans perdre l’historique des décisions déjà prises.

5.1.2 Backlog du sprint

Table 5.1 : Backlog fonctionnel du module pipeline candidat et entretiens

ID	Thème	User Story	Statut
15	Affectation candidat	En tant que TA, je peux affecter un CV à une offre afin de créer un candidat dans le pipeline.	Réalisé
16	Progression de stage	En tant que recruteur, je peux faire évoluer un candidat d’une étape à une autre.	Réalisé
17	Assignment Manager	En tant que TA, je peux affecter un candidat à un manager pour l’étape managériale.	Réalisé
18	Assignment RH	En tant que manager, je peux transférer un candidat accepté vers un RH identifié.	Réalisé
19	Planification d’entretien	En tant qu’évaluateur, je peux planifier un entretien avec date, heure et lien de réunion.	Réalisé
20	Rapport d’entretien	En tant qu’évaluateur, je peux enregistrer un rapport structuré avec score, notes et décision.	Réalisé
21	Calendrier	En tant que TA, je peux consulter un calendrier consolidé des entretiens.	Réalisé
22	Décision RH	En tant que RH, je peux accepter, rejeter ou marquer un candidat comme hired.	Réalisé
23	Onboarding	En tant que système, je peux initialiser une checklist d’onboarding pour un candidat embauché.	Réalisé
23-bis	Historique d’étapes	En tant qu’évaluateur ou administrateur, je peux consulter l’historique auditable des transitions d’un candidat.	Réalisé

Ce backlog couvre toute la circulation du candidat : création, coordination, entretien, décision finale et prolongement post-recrutement.

5.2 Implémentation du sprint

5.2.1 Analyse du sprint

Le sprint s'appuie principalement sur les éléments fonctionnels suivants :

- le service de gestion du pipeline candidat ;
- le service de planification, d'agenda et de calendrier des entretiens ;
- le service de rapports d'entretien et de décisions ;
- le service d'historique des transitions candidat ;
- le service de checklist d'onboarding après embauche ;
- le parcours manager, avec sa liste de candidats et sa vue de détail ;
- le parcours RH, avec sa liste de candidats et sa vue de détail ;
- la vue calendrier et le composant de coordination transverse des entretiens ;
- l'interface d'évaluation détaillée côté manager ;
- l'interface d'évaluation détaillée côté RH.

Ce module organise la circulation d'un candidat à travers plusieurs responsabilités humaines, chacune avec ses propres décisions et ses propres données. Il ne s'agit donc pas d'une simple vue de candidatures reçues, mais du moteur procédural du recrutement.

5.2.2 Vue fonctionnelle globale

Les usages couverts se regroupent en quatre blocs :

1. **Création et suivi du candidat** : affecter un CV à une offre, créer un candidat et suivre son état dans le pipeline.
2. **Coordination des rôles** : assigner un Manager, assigner un RH et exposer à chaque rôle uniquement les candidats qui lui sont confiés.
3. **Gestion des entretiens** : planifier un entretien, envoyer une invitation, marquer un entretien comme complété et consulter le calendrier consolidé.
4. **Décision et clôture** : enregistrer un rapport, accepter ou rejeter un candidat, le marquer comme `hired` et initialiser l'onboarding.

Cette structuration montre que le produit orchestre désormais un processus collaboratif complet, et non plus seulement la gestion d'objets documentaires.

5.2.3 Cycle de vie du candidat

Le pipeline est formalisé dans le schéma de données via une suite d'états explicites. Le candidat peut progresser selon le chemin principal suivant :

`new` → `ta_screening` → `ta_interview` → `ta_accepted` → `manager_interview` → `manager_accepted` → `hr_interview` → `hr_accepted` → `hired`

À cette trajectoire s'ajoutent les branches de rejet `ta_rejected`, `manager_rejected` et `hr_rejected`. Cette modélisation est importante, car elle montre que le processus de recrutement n'est pas implicite : chaque étape porte un sens métier, une responsabilité et des données associées.

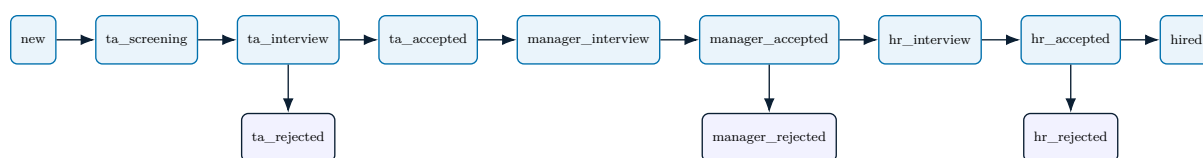
Elle permet également d'éviter l'ambiguïté organisationnelle : lorsqu'un candidat se trouve dans un état donné, la plateforme sait quels acteurs sont concernés, quelles actions sont permises et quelles informations doivent être affichées. Chaque mouvement est maintenant doublé d'une trace dans `candidate_stage_history`. Cette table conserve l'étape précédente, la nouvelle étape, l'acteur, la raison, la source du changement et la date. Elle complète donc le statut courant du candidat par une mémoire chronologique, indispensable pour comprendre comment une décision a été atteinte.

Tableau de transitions du pipeline candidat. La logique d'état peut être synthétisée par le tableau suivant :

Table 5.2 : Transitions principales du pipeline candidat

État courant	cou-	Acteur respon-	sable	Action principale	État suivant possible
new		TA		analyser le profil et lancer la présélection	ta_screening
ta_screening		TA		planifier ou conclure l'évaluation initiale	ta_interview, ta_rejected
ta_interview		TA		enregistrer le rapport d'entretien TA	ta_accepted, ta_rejected
ta_accepted		TA		affecter un manager	manager_interview
manager_interview		Manager		conduire l'entretien manager	manager_accepted, manager_rejected
manager_accepted		Manager		affecter un RH pour validation finale	hr_interview
hr_interview		RH		conclure l'entretien RH et décider	hr_accepted, hr_rejected
hr_accepted		RH		valider l'embauche	hired
hired		RH / Admin		déclencher et suivre l'onboarding	tâches d'intégration actives

Ce tableau rend explicite un point important : chaque transition correspond à une responsabilité humaine identifiable. Le pipeline n'est donc pas seulement une suite de statuts techniques, mais une représentation formalisée du processus de décision du recrutement.

**Figure 5.1** : Diagramme d'états du pipeline candidat

5.2.4 Affectation progressive des responsabilités

Le service `assignManagerToCandidate()` positionne explicitement `assignedManagerId` et fait passer le candidat à l'état `manager_interview`. De même, `assignHrToCandidate()` renseigne `assignedHrId` et fait passer le candidat à l'état `hr_interview`. Au lieu de laisser les responsabilités flotter de manière implicite, le système associe chaque étape clé à un acteur identifié, ce qui améliore la gouvernance du pipeline et la traçabilité des décisions.

Cette approche réduit le risque de zone grise fréquent dans les recrutements, où un candidat change de main sans que la responsabilité opérationnelle soit clairement matérialisée

dans l'outil.

Matrice synthétique des rôles. La répartition des responsabilités peut être résumée de la manière suivante :

Table 5.3 : Rôles et responsabilités dans le pipeline candidat

Rôle	Données principale- ment visibles	Décisions possibles	Responsabilité princi- pale
TA	CV, offres, candidats en entrée de pipeline, agenda initial	affecter un CV, planifier l'entretien TA, rejeter en amont, transmettre au manager	ouvrir et qualifier le dossier candidat
Manager	candidats affectés, rapports TA, guide manager, entretien managérial	accepter, rejeter, transférer vers un RH	produire la décision d'évaluation métier ou technique
RH	candidat finaliste, rapports antérieurs, entretien RH, email de décision, onboarding	accepter, rejeter, marquer comme hired	formaliser la décision finale et prolonger vers l'intégration

Cette matrice montre que la plateforme ne distribue pas seulement des écrans différents. Elle distribue des périmètres d'action distincts, ce qui permet d'aligner l'outil sur l'organisation réelle du recrutement.

5.2.5 Planification d'un entretien

Le service `scheduleInterview()` valide les données d'entrée, crée l'entretien, aligne l'état du candidat sur le niveau concerné, puis déclenche les mécanismes de notification et de log associés. Ensuite, selon l'interface utilisée, un email peut être envoyé au candidat via `sendInterviewEmailAction()`.

Le flux général est le suivant : choix du candidat et de l'étape, saisie de la date, de l'heure et du lien de réunion, création de l'entretien, éventuelle mise à jour du stage, envoi de l'invitation puis suivi jusqu'à **completed** ou **cancelled**.

Ce flux couple donc opérationnel et traçabilité : chaque entretien planifié est à la fois un événement métier, un objet d'agenda et une source future de reporting.

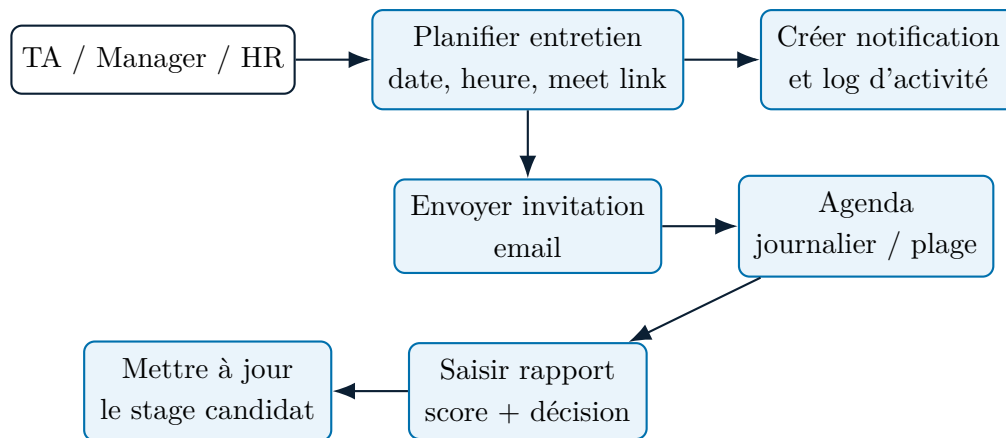


Figure 5.2 : Séquence de planification, notification et reporting d'un entretien

5.2.6 Rapports d'entretien et transition vers l'onboarding

Le service `saveInterviewReport()` enregistre un rapport structuré contenant notamment l'identifiant de l'entretien, le candidat, l'étape, les notes, les réponses du candidat, l'évaluation globale, le score et la décision. Cette structure transforme un entretien en objet exploitable, comparable et réutilisable.

Lorsque le candidat atteint l'état `hired`, le système crée automatiquement une checklist par défaut via `createOnboardingChecklist()`. La plateforme ne s'arrête donc pas à la décision finale ; elle prolonge le suivi au-delà du recrutement.

L'entretien n'est plus une simple discussion suivie d'un avis informel. Il devient une production de données métier structurées, reliées à une étape précise et réexploitables plus tard dans l'analyse globale du recrutement.

5.2.7 Historique auditable des transitions

Le service `updateCandidateStage()` ne met pas seulement à jour le champ `stage`. Il écrit aussi une ligne immuable dans `candidate_stage_history`. Lors de l'affectation initiale, `assignCvToJob()` enregistre une création de pipeline avec la source `assignment`. Les évolutions suivantes peuvent provenir d'une action manuelle, d'un traitement groupé, d'une affectation manager, d'une affectation RH, d'un rapport d'entretien ou d'une action agentique confirmée.

Cette conception évite une limite classique des pipelines de recrutement : connaître uniquement l'état courant sans savoir qui l'a modifié ni pourquoi. Les fiches Manager et RH affichent cette mémoire via `CandidateStageHistoryTimeline`. Les détails d'acteur restent visibles selon le rôle, ce qui permet de concilier transparence métier et restriction d'accès aux informations internes.

L'historique sert aussi de pont vers la gouvernance administrateur. Les mêmes lignes qui expliquent le parcours d'un candidat alimentent ensuite le centre de gouvernance, les filtres d'audit et les exports CSV. Une transition n'est donc pas seulement une mutation applicative ; elle devient une preuve opérationnelle réutilisable.

Règles métier structurantes. Plusieurs règles implicites du workflow méritent d’être formulées explicitement. Un manager n’intervient qu’après qualification initiale par TA. Le transfert vers le RH n’a de sens que si l’évaluation managériale a abouti à un avis positif. Le statut `hired` ne doit être atteint qu’au terme de la validation RH. Enfin, l’onboarding ne démarre qu’après cette décision finale, afin d’éviter toute ouverture prématurée d’un suivi d’intégration.

Ces règles sont importantes pour la robustesse métier du système : elles empêchent qu’un candidat progresse de manière incohérente dans le pipeline et garantissent que chaque étape repose sur une décision antérieure traçable.

Scénario end-to-end candidat. Le scénario métier central part d’un CV affecté à une offre. Le TA qualifie le dossier, planifie ou conclut une première étape, puis transmet le candidat au Manager si le profil est retenu. Le Manager produit ensuite une évaluation structurée et peut transférer le dossier vers un RH. Le RH formalise la décision finale, envoie la communication nécessaire et déclenche l’état `hired` lorsque l’embauche est validée. Ce scénario montre que le module ne se limite pas à changer un statut : il organise la responsabilité, la preuve et la continuité du dossier candidat.

5.3 Réalisation

5.3.1 Vue Manager

La vue manager charge uniquement les candidats affectés au manager courant dans les états `manager_interview`, `manager_accepted` et `manager_rejected`. La page détail récupère ensuite le candidat, les rapports TA, le guide d’entretien manager, l’entretien manager courant, la liste des RH et le guide autopilot si disponible. Le composant `ManagerCandidateDetailClient` organise l’évaluation en étapes successives : génération des questions d’entretien, édition et sauvegarde des questions, planification de l’entretien, envoi de l’email, marquage de l’entretien comme complété, saisie du rapport puis décision finale de rejet ou de transfert vers un RH identifié. La fiche manager intègre en plus un panneau d’*evidence readiness* qui expose le score de screening disponible, le dernier état d’entretien, la décision antérieure, les éléments manquants et les risques avant tout raisonnement IA. Les actions *Ask Agent*, *Explain score*, *Summarize feedback* et *Decision risks* réutilisent ensuite ce contexte de preuve pour produire des prompts plus sûrs et plus situés. La fiche affiche également la timeline `Stagehistory`, qui permet au manager de replacer son évaluation dans la chronologie complète du dossier sans perdre la séparation des responsabilités. Cette logique reflète bien le rôle du manager dans le produit : il ne se contente pas de visualiser un profil, il produit une décision structurée sur la base d’un dossier enrichi et explicitement qualifié.

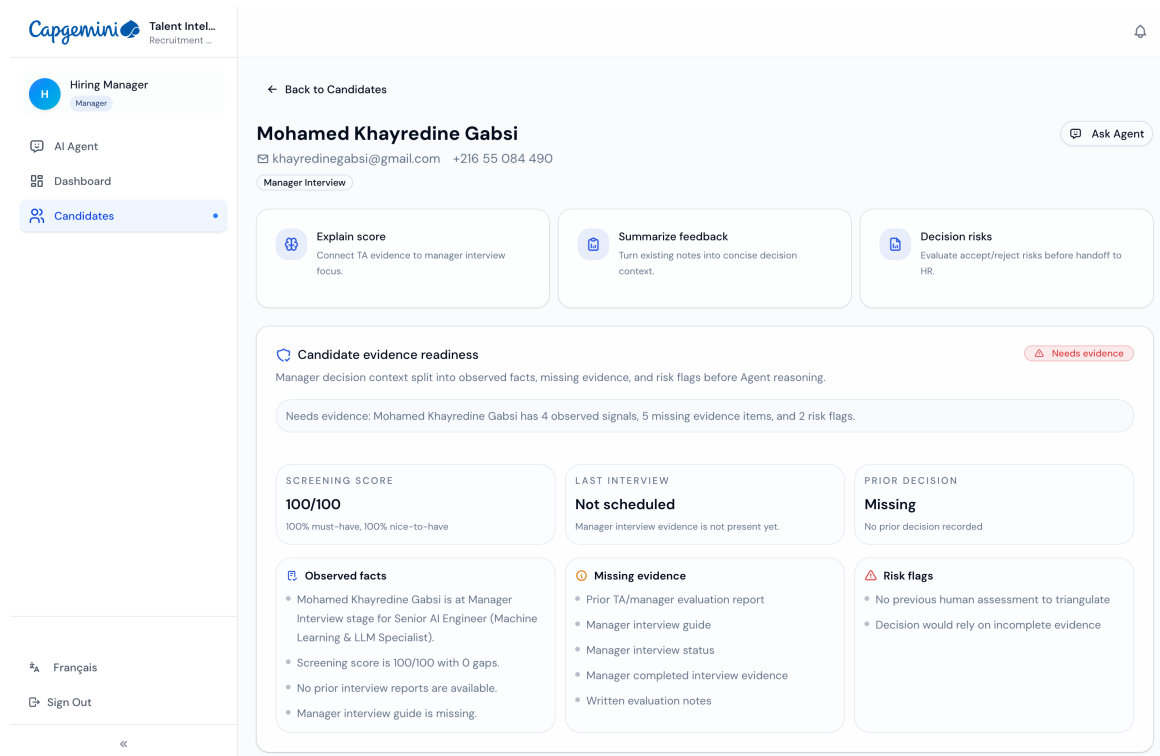


Figure 5.3 : Fiche candidat côté Manager.

5.3.2 Vue RH

La vue RH charge les candidats assignés au RH courant dans les états `hr_interview`, `hr_accepted`, `hr_rejected` et `hired`. La page détail agrège le candidat, les rapports précédents TA et Manager, le guide RH, l'entretien RH courant et l'autopilot RH. Le composant `HRCandidateDetailClient` ouvre alors plusieurs actions : consultation des rapports antérieurs, planification optionnelle d'une réunion RH, marquage de la réunion comme complétée, décision `hr_accepted` ou `hr_rejected`, génération IA d'un email de décision, envoi de cet email et passage au statut `hired`. Comme côté manager, un panneau d'*evidence readiness* résume les faits observés, les trous documentaires et les signaux de risque avant la décision finale. Les actions agentiques RH réinjectent ce contexte pour produire des rapports de décision et des analyses de risque séparant mieux les constats des recommandations. La fiche RH reprend la même logique d'historique afin de vérifier le chemin déjà parcouru par le candidat avant la décision finale. Cette lecture réduit le risque de décision isolée, car l'étape RH s'appuie sur les traces TA et Manager déjà enregistrées. Cette étape montre bien que le RH joue à la fois un rôle décisionnel et un rôle de communication formelle avec le candidat, avec un niveau de justification plus explicite qu'une simple lecture de notes brutes.

The screenshot displays the 'Candidate evidence readiness' section for Rayen Fassatoui. The interface includes a sidebar with navigation options like 'HR Director', 'AI Agent', 'Dashboard', 'Candidates', and 'Export'. The main content area shows a 'Back to Candidates' link, the candidate's name, email, and phone number, and an 'Ask Agent' button. Below this, there are three action cards: 'Hire/no-hire risks', 'Summarize feedback', and 'Draft decision report'. The 'Candidate evidence readiness' section provides a summary of evidence, including a screening score of 81/100, last interview status (Not scheduled), and prior decision status (Missing). It also lists observed facts, missing evidence, and risk flags.

Rayen Fassatoui
rayenft2001@gmail.com +216 95 004 044
Hired

Hire/no-hire risks
Surface decision risk before final HR action.

Summarize feedback
Condense TA and manager evidence for HR.

Draft decision report
Prepare a defensible decision narrative.

Candidate evidence readiness
Needs evidence
HR decision context split into observed facts, missing evidence, and risk flags before Agent reasoning.
Needs evidence: Rayen Fassatoui has 4 observed signals, 5 missing evidence items, and 2 risk flags.

SCREENING SCORE
81/100
100% must-have, 33% nice-to-have

LAST INTERVIEW
Not scheduled
HR interview evidence is not present yet.

PRIOR DECISION
Missing
No prior decision recorded

Observed facts

- Rayen Fassatoui is at Hired stage for Frontend React Developer.
- Screening score is 81/100 with 0 gaps.
- No prior interview reports are available.
- HR interview guide is missing.

Missing evidence

- Prior TA/manager evaluation report
- HR interview guide
- HR interview status
- HR completed interview evidence
- Written evaluation notes

Risk flags

- No previous human assessment to triangulate
- Decision would rely on incomplete evidence

Figure 5.4 : Fiche candidat côté RH.

The screenshot displays the 'Interviews' section for the job 'QA Preserved UI UX Designer 202606281539'. The interface includes a sidebar with navigation options like 'Talent Acquisition', 'AI Agent', 'Dashboard', 'CV Pool', 'Job Requirements', 'Calendar', and 'Statistics'. The main content area shows a 'Back to Jobs' link, the job title, and buttons for 'Ask Agent', 'open', and 'Close Job'. Below this, there are four action cards: 'Open CV Matching', 'Open Interviews', 'Explain matching with Agent', and 'Review requirement with Agent'. The 'Interviews' section includes a 'Schedule Interview' button and a table with columns for Candidate, Stage, Date & Time, Link, Status, and Actions. The table currently shows no active candidates for interview.

QA Preserved UI UX Designer 202606281539
Ask Agent open Close Job

Open CV Matching
Use the built-in matching workflow and assign candidates directly.

Open Interviews
Jump to interview scheduling, emails, and reports for this job.

Explain matching with Agent
Get ranking rationale, gaps, and screening priorities.

Review requirement with Agent
Tighten ambiguity before screening more candidates.

Overview CV Matching Interviews Pipeline

Interviews
Schedule Interview

Candidate	Stage	Date & Time	Link	Status	Actions
No active candidates for interview. Go to Pipeline to accept candidates into the interview stage.					

Figure 5.5 : Interface de planification d'entretien.

5.3.3 Calendrier et coordination transverse

La vue calendrier donne accès à `CalendarView`, qui fournit une vue consolidée des entretiens. Le service `getInterviewCalendar()` récupère les entretiens d'un utilisateur sur une plage de dates, ce qui permet une lecture plus large que la simple liste journalière. Cette vue complète les pages de détail candidat et offre une lecture transversale de la charge de recrutement.

Elle apporte une vision organisationnelle souvent absente des outils simplifiés : voir le recrutement non seulement candidat par candidat, mais aussi comme une activité collective à synchroniser entre plusieurs agendas et plusieurs étapes.

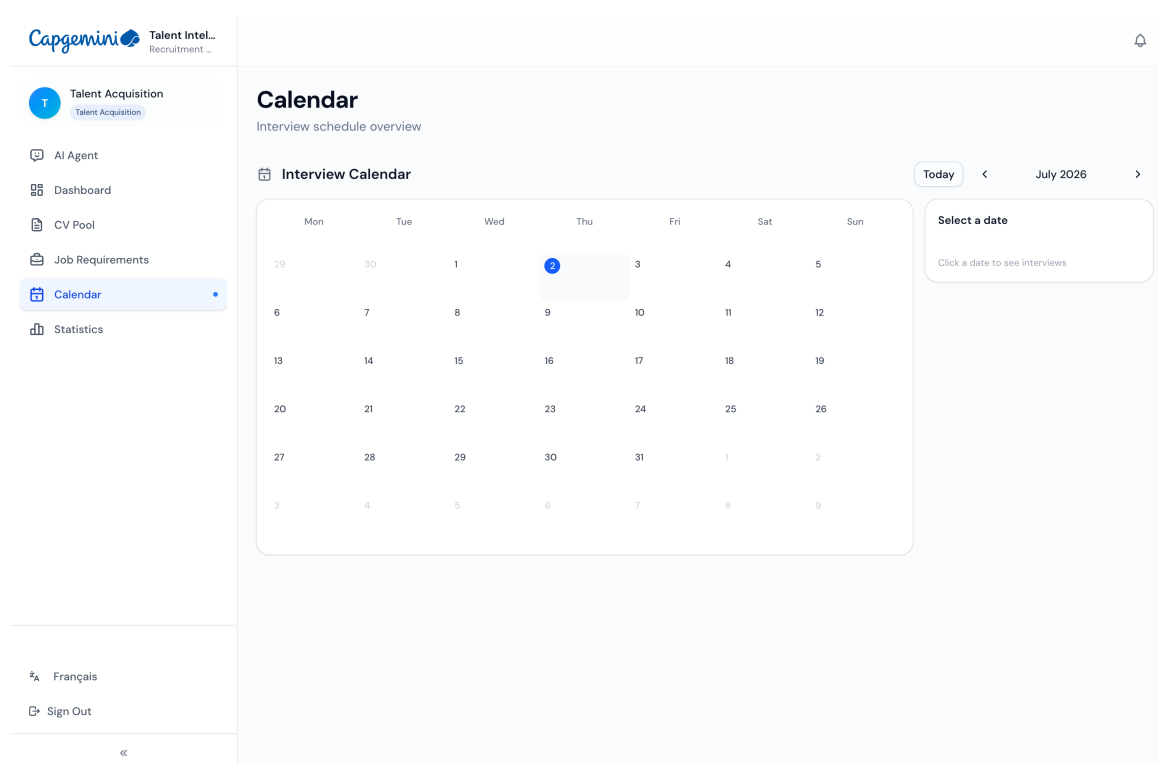


Figure 5.6 : Vue calendrier des entretiens.

5.3.4 Mémoire opérationnelle du processus

Ce module introduit la mémoire procédurale du recrutement. Les rapports, décisions, états d'entretien, notifications et tâches d'onboarding forment ensemble une trace continue de ce qui s'est passé. Cette matière sera réexploitée ensuite par les modules IA pour produire des débriefings, des recommandations et des exports de décision.

La mémoire produite ici a donc une double fonction : sécuriser le pilotage humain du processus et alimenter les futures couches d'analyse et de synthèse.

5.3.5 Articulation avec les modules IA

Même si le chapitre suivant détaille davantage l'intelligence artificielle, ce sprint prépare déjà plusieurs usages concrets :

- les guides d'entretien sont récupérés et affichés selon l'étape du pipeline ;
- les vues Manager et HR intègrent l'`InterviewAutoPilotGuideView` lorsque le guide est disponible ;
- les panneaux d'*evidence readiness* exposent les faits disponibles, les données manquantes et les risques avant l'appel à l'agent ;
- les rapports d'entretien fournissent la matière nécessaire aux débriefings, synthèses et recommandations futures ;
- la structure du pipeline permet de contextualiser les décisions, les transitions et les prochaines actions.

Autrement dit, ce sprint rend l'IA situationnelle. Les données ne sont plus seulement documentaires : elles sont contextualisées par un moment précis du processus de recrutement et par un niveau explicite de préparation de la preuve. Cette contextualisation permet ensuite à la couche IA de produire une aide plus utile qu'une génération générique, car elle sait à quelle étape se trouve le candidat, quelles décisions ont déjà été prises et quelles questions restent ouvertes.

5.4 Apport du module

Ce module apporte une contribution structurante au projet :

- il transforme un CV affecté en candidature pilotée ;
- il explicite les responsabilités de chaque rôle ;
- il donne un cadre formel aux entretiens et aux décisions ;
- il rend les étapes de recrutement traçables et justifiables ;
- il transforme chaque transition en preuve chronologique consultable ;
- il prépare la transition vers l'onboarding ;
- il enrichit fortement la base métier qui sera exploitée par les modules IA.

Dans l'économie générale du rapport, ce module correspond au moment où la plateforme cesse d'être uniquement documentaire pour devenir un système de décision et de coordination.

Cette transformation n'est pas seulement technique. Elle améliore aussi la qualité du recrutement lui-même : meilleure visibilité, responsabilités mieux distribuées, décisions plus explicites et continuité plus forte entre les acteurs.

Conclusion

Ce troisième module a donné au produit sa structure procédurale complète : progression du candidat, historique des transitions, assignation des rôles, planification des entretiens, rapports, décisions et démarrage de l'onboarding. Il fournit la continuité métier indispensable à une plateforme de recrutement intelligente. Le chapitre suivant s'appuiera sur ce socle pour détailler la couche d'intelligence artificielle, notamment le matching, la recherche sémantique, le RAG et l'assistance agentique.

Chapitre 6

Module 4 : intelligence artificielle, matching et recherche augmentée

Introduction

Ce quatrième module constitue le différenciateur principal du projet. À ce stade, la plateforme dispose déjà d'un vivier de CV structuré, d'offres détaillées et d'un pipeline candidat opérationnel. L'enjeu n'est donc plus seulement de stocker, afficher ou faire circuler des données, mais de les transformer en intelligence exploitable.

Ce module ne correspond pas à l'ajout d'un simple assistant textuel. Il met en place une couche IA complète qui intervient directement dans les opérations de recrutement : matching, screening, recherche sémantique, RAG, génération de guides d'entretien, comparaison de candidats, synthèses, optimisation d'offres et agent conversationnel outillé. Les fondements du RAG et de la recherche par similarité sont rattachés aux références scientifiques citées en bibliographie et webographie [20][21].

Le point clé n'est pas uniquement la présence de modèles IA, mais la manière dont ils sont branchés sur le produit. Les traitements intelligents ne vivent pas à côté du système ; ils s'insèrent dans les pages métier, dans les services de recherche, dans les workflows d'entretien et dans les mécanismes de sécurité.

6.1 Sprint 4 : objectif et périmètre

6.1.1 Objectif du sprint

L'objectif est d'intégrer l'intelligence artificielle au cœur du processus de recrutement afin d'assister les utilisateurs sans rompre la cohérence métier ni la sécurité des données. Plus précisément, ce sprint vise à :

- accélérer le tri initial des profils ;
- fournir des scores et synthèses de screening ;
- rendre possible une recherche sémantique sur le vivier ;

- introduire un pipeline RAG hybride pour les recherches complexes ;
- générer des guides d'entretien contextualisés ;
- exposer ces capacités à travers un agent conversationnel outillé ;
- mesurer l'effet réel des améliorations apportées au pipeline de recherche.

Le sprint répond aussi à une exigence de crédibilité : l'IA ne doit pas être cosmétique. Elle doit améliorer la pertinence des recherches, accélérer les évaluations, enrichir les entretiens et rester contrôlable dans ses entrées comme dans ses sorties.

6.1.2 Backlog du sprint

Table 6.1 : Backlog fonctionnel du module intelligence artificielle

ID	Thème	User Story	Statut
24	Screening IA	En tant que TA, je peux générer un screening IA d'un candidat sur une offre.	Réalisé
25	Matching par compétences	En tant que TA, je peux classer des CV selon leur adéquation avec une offre.	Réalisé
26	Recherche sémantique	En tant que recruteur, je peux rechercher des profils par similarité sémantique.	Réalisé
27	RAG hybride	En tant qu'utilisateur avancé, je peux lancer une recherche augmentée sur les chunks de CV avec citations.	Réalisé
28	Guides d'entretien IA	En tant qu'évaluateur, je peux générer des questions et guides d'entretien contextuels.	Réalisé
29	Autopilot d'entretien	En tant qu'évaluateur, je peux consulter un guide enrichi structuré par axes d'évaluation.	Réalisé
30	Agent conversationnel	En tant qu'utilisateur autorisé, je peux interroger la plateforme via un agent multi-outils, obtenir une réponse exploitable, visualiser des indicateurs utiles et consulter une trace par phases.	Réalisé
30-bis	Confirmation des actions IA	En tant qu'utilisateur, je dois confirmer explicitement toute action agentique qui modifie les données de recrutement.	Réalisé
30-ter	Cartes de réponse typées	En tant qu'utilisateur, je peux lire des cartes candidat, pipeline ou gouvernance construites à partir des sorties d'outils vérifiées.	Réalisé
31	Smart insights	En tant que TA ou Admin, je peux consulter des statistiques enrichies et des insights IA.	Réalisé
32	Évaluation RAG	En tant qu'équipe projet, nous pouvons mesurer la qualité et les compromis du pipeline RAG.	Réalisé
32-bis	Évaluation agentique	En tant qu'équipe projet, nous pouvons vérifier automatiquement les garde-fous de grounding, de confirmation, de restitution structurée et de trace outil.	Réalisé

Le backlog montre clairement que la couche IA n'est pas mono-usage. Elle touche à la recherche, au scoring, à la préparation d'entretien, à la conversation analytique et à l'évaluation quantitative du système lui-même.

6.2 Implémentation du sprint

6.2.1 Analyse du sprint

Ce sprint s'appuie principalement sur les éléments fonctionnels suivants :

- le service de matching lexical, sémantique et hybride ;
- le pipeline RAG de récupération documentaire ;
- le service de génération des embeddings NVIDIA ;
- les services avancés de génération et d'analyse IA ;
- le service de guides d'entretien et les composants Auto-Pilot ;
- le mécanisme de contrôle des noms cités par l'agent ;
- la couche d'orchestration conversationnelle en SSE ;
- le mécanisme de confirmation des actions agentiques modifiant les données ;
- le registre des outils métier de l'agent ;
- la page statistique du recruteur et son chat analytique ;
- les cartes de réponse typées candidat, pipeline et gouvernance ;
- le regroupement de la trace d'outils par phases d'exécution ;
- le rapport d'évaluation quantitative du pipeline RAG ;
- la suite d'évaluation agentique et UI du chat.

Ce sprint est transversal : il touche à la fois aux données, à la récupération documentaire, aux modèles IA, à l'expérience utilisateur et aux mécanismes de sécurité.

Cette transversalité est importante, car elle montre que la valeur IA n'a pas été concentrée dans un seul bloc démonstratif. Elle a été distribuée au plus près des besoins métier, là où elle peut réellement améliorer le travail des recruteurs.

6.2.2 Niveaux d'intelligence du projet

La couche IA n'est pas monolithique. Elle se compose de plusieurs niveaux complémentaires :

1. **Intelligence documentaire** : extraction structurée des CV, génération d'embeddings, découpage en chunks orientés RAG, indexation vectorielle et lexicale.
2. **Intelligence de recrutement** : matching par compétences, matching sémantique, screening stocké en base, recommandations et synthèses IA.
3. **Intelligence d'entretien** : génération de questions, guides éditables, Auto-Pilot Interview Guide, débriefings et questions de suivi.
4. **Intelligence conversationnelle** : agent multi-outils, raisonnement en plusieurs étapes, accès contrôlé aux services métier, réponses ancrées dans les résultats disponibles, cartes typées et trace d'exécution par phases.

Cette structuration montre que l'IA n'est pas résumée à un seul modèle. Elle est distribuée dans plusieurs services spécialisés selon les usages et intégrée progressivement à mesure que les données métier se structurent.

6.2.3 Matching, screening et autopilot

Le projet distingue plusieurs formes de matching : un matching par critères explicites centré sur l'alignement entre compétences du CV et exigences du poste, une recherche sémantique fondée sur des embeddings NVIDIA NV-EmbedQA E5 V5 de dimension 1024 et une recherche hybride qui combine ranking lexical et vectoriel par *Reciprocal Rank Fusion*. Cette logique de similarité et de récupération augmentée s'inscrit dans les principes documentés par les travaux de recherche cités [20][21].

Le screening complète ce matching en ajoutant une lecture décisionnelle : score global, écarts, correspondances *must-have* et *nice-to-have*, puis résumé synthétique. Les services de guides d'entretien produisent quant à eux un guide de questions éditable et un *Auto-Pilot Interview Guide* plus riche, incluant briefing, questions techniques, questions de mitigation des écarts et questions comportementales adaptées au niveau de séniorité.

Cette combinaison évite de dépendre d'un seul mécanisme. Certaines requêtes bénéficient d'une lecture très structurée, d'autres d'une lecture sémantique plus souple, et les cas complexes profitent de la fusion de plusieurs signaux.

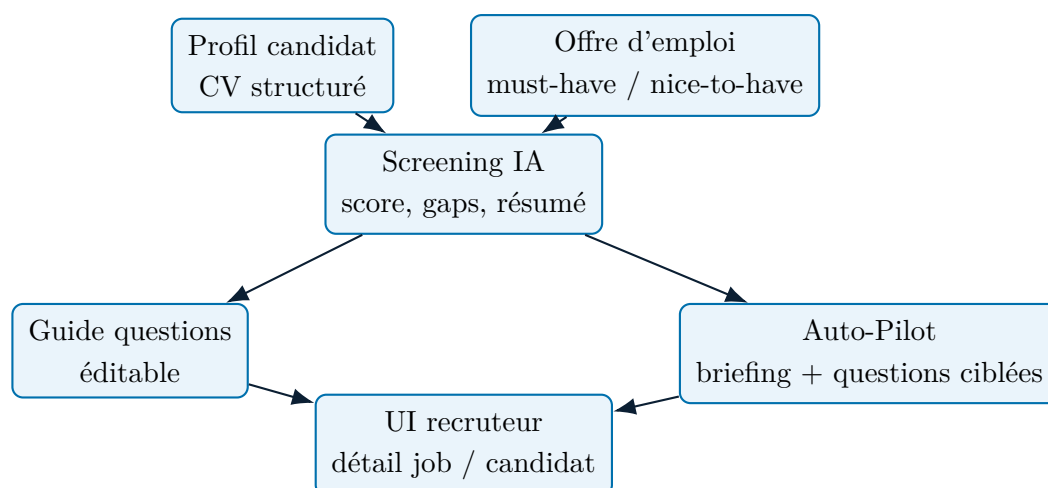


Figure 6.1 : Chaîne IA de screening et de génération de guide d'entretien

6.2.4 Agent conversationnel, réponses exploitables et garde-fous

L'agent conversationnel ne se limite plus à une simple fenêtre de chat sur la page statistique. Il suit désormais un schéma hybride : une assistance flottante reste disponible dans les espaces métier pour les demandes rapides, tandis qu'un workspace dédié `/agent` prend en charge les analyses longues, les traces d'exécution, les pièces jointes et l'historique conversationnel complet. Sur le plan technique, cet agent s'appuie sur un registre de plus de cinquante outils répartis en catégories : CV Pool, Jobs, Candidates, Matching, Interviews, AI Features, Communication, Dashboard et Admin. Le registre centralise les définitions, la validation Zod, l'exécution et le filtrage RBAC par rôle. Le projet ne laisse cependant pas l'agent répondre librement sur des personnes qu'il n'a pas effectivement résolues. Le mécanisme de grounding reconstruit l'ensemble de candidats autorisés à partir des résultats d'outils et valide les noms présents dans la réponse finale. Cette logique complète les garde-fous définis dans la couche d'orchestration : nombre maximal d'étapes, volume de sortie, timeout, seuil d'échec d'outils, séparation entre faits observés et recommandations, restitution finale orientée décision et transformation des résultats en cartes typées lorsque cela améliore la lecture. Ces garde-fous répondent à une exigence forte du domaine RH : une réponse fluide mais non justifiée peut être plus dangereuse qu'une absence de réponse. Le système préfère donc encadrer l'agent, afficher sa *evidence trace*, regrouper les outils par phases d'exécution et pointer ses limites plutôt que maximiser artificiellement sa liberté de génération. Pour les outils qui modifient les données, le projet ajoute un garde-fou supplémentaire : l'action n'est pas exécutée immédiatement. Le service `pending-agent-actions.ts` crée une entrée `pending_agent_actions` avec les arguments sanitisés, un résumé métier, un statut `pending` et une expiration. L'interface affiche alors une carte de confirmation enrichie, avec niveau de risque, entités affectées, impact attendu, compte à rebours et arguments exacts ; l'utilisateur peut confirmer ou annuler. Après confirmation, l'outil est exécuté, puis l'action passe à `executed` ou `failed`. Ainsi, l'agent peut proposer une mutation, mais la responsabilité finale reste explicitement humaine.

6.3 Vue dynamique

6.3.1 Orchestration agentique

Le flux global du chat est le suivant : authentification, construction du contexte, filtrage RBAC des outils, boucle agentique, demande de confirmation si l'outil modifie les données, exécution d'outil, sanitization, grounding, construction des métadonnées de preuve, génération éventuelle de graphiques et de cartes typées, puis diffusion SSE de la réponse. Cette architecture donne à l'agent une vraie capacité d'action tout en gardant la main sur le périmètre autorisé, sur la validation des entrées et sur les mutations sensibles.

Elle matérialise la forme la plus visible du projet côté utilisateur final : dialoguer avec le système en langage naturel tout en restant relié à des outils, à des permissions et à des résultats vérifiables.

6.3.2 Pipeline RAG hybride

Le service `retrieval-pipeline.ts` implémente explicitement le pipeline suivant : *rewrite* → *vector retrieval* → *lexical retrieval* → *RRF fusion* → *rerank* → *context assembly*. Les chunks sont stockés dans `cv_chunks`, indexés à la fois par embedding et par `tsvector` pour la recherche plein texte.

Ce point est essentiel : la génération n'est pas réalisée à vide. Le système commence par récupérer un contexte pertinent issu des données réellement disponibles, puis seulement ensuite transmet ce contexte à la couche de réponse ou d'analyse.

Le RAG n'est donc pas utilisé comme un effet de mode. Il sert à réintroduire une base de justification dans les réponses IA, notamment pour les recherches complexes où une simple liste de profils ne suffit plus.

Comparaison des modes de recherche et d'assistance. Pour bien situer les apports du sprint, il est utile de distinguer les principaux niveaux d'accès à l'information :

Table 6.2 : Comparaison entre recherche simple, recherche sémantique, RAG et chat agentique

Mode	Base principale	Type de résultat	Usage métier dominant
Recherche simple	champs textuels et filtres explicites	correspondances directes	retrouver rapidement un profil ou une offre connus
Recherche sémantique	embedding global des profils	profils proches par similarité	explorer des candidats au-delà des mots-clés exacts
RAG hybride	chunks, embeddings et index lexical	extraits contextualisés avec citations	répondre à des questions complexes sur le contenu des profils
Chat agentique	outils métier, retrieval et grounding	réponse raisonnée et actionnable	interroger, comparer, synthétiser et piloter en langage naturel

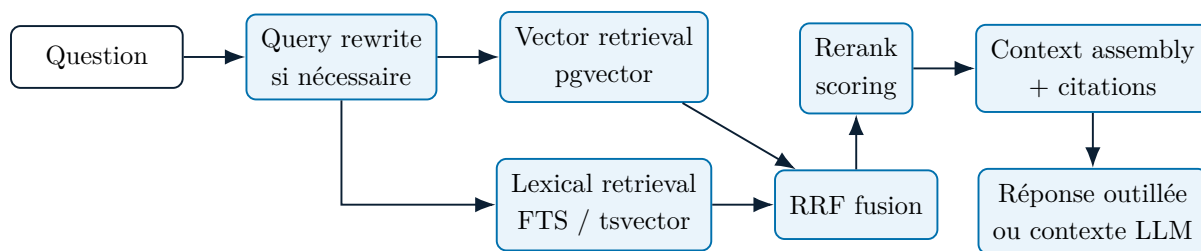
Ce tableau montre une progression de complexité mais aussi de valeur. Plus le niveau monte, plus la plateforme s'éloigne d'une simple recherche documentaire pour devenir un système d'assistance à la décision branché sur ses propres données métier.

Justification critique des choix IA. Le choix du RAG, du retrieval hybride et de l'agent outillé résulte d'un compromis entre pertinence, explicabilité, coût et gouvernance. Un modèle génératif seul serait plus simple à appeler, mais il ne pourrait pas garantir que la réponse s'appuie sur les CV, offres et historiques réellement présents dans la plateforme. À l'inverse, une recherche SQL classique serait rapide et déterministe, mais elle resterait insuffisante pour des formulations métiers proches sans mots-clés identiques. Le RAG hybride est donc retenu parce qu'il combine une réponse contextualisée avec des sources récupérées, tandis que l'agent outillé est retenu uniquement pour les demandes qui nécessitent plusieurs opérations métier contrôlées.

Table 6.3 : Justification analytique des choix IA

Choix	Pourquoi ce choix	Compromis observé	Mécanisme de maîtrise
RAG plutôt que génération seule	Les travaux sur le RAG montrent l'intérêt d'ancrer la génération dans des documents récupérés [20][22].	Latence plus élevée et dépendance à la qualité de l'index.	Chunks par section, citations, cache, fallback et évaluation quantitative.
Hybrid Search plutôt que vectoriel seul	Les compétences exactes, certifications et technologies nommées exigent un signal lexical ; les synonymes et formulations proches exigent un signal dense.	Deux listes doivent être fusionnées et rerankées.	RRF, inspiré de la fusion de rangs [24], puis reranking et seuils.
Réécriture de requête ciblée	Les requêtes longues de recruteurs mélangent souvent besoin, contexte et contraintes.	Appel LLM supplémentaire, donc coût et latence.	Activation seulement si la complexité le justifie, timeout et fallback lexical.
Agent outillé plutôt que chatbot libre	Les approches ReAct et Toolformer montrent l'intérêt de coupler raisonnement et actions outillées [27][28].	Risque d'action non désirée ou de réponse hors périmètre.	Registre d'outils, RBAC, validation Zod, confirmations et grounding.
Pas de fine-tuning initial	Les données CV et offres changent fréquemment ; les faits doivent rester actualisables sans réentraîner un modèle.	Moins de spécialisation linguistique qu'un modèle ajusté.	Prompts spécialisés, retrieval contextualisé et perspectives de fine-tuning ciblé.

Le tableau 6.3 montre que la performance attendue n'est pas seulement une performance algorithmique. Elle dépend aussi de la capacité à expliquer, limiter et vérifier l'IA dans un domaine sensible. Le projet privilégie donc des mécanismes réversibles et auditable : récupérer avant de générer, fusionner plusieurs signaux plutôt qu'en absolutiser un seul, confirmer avant de modifier, et mesurer les gains au lieu de les affirmer.

**Figure 6.2 :** Pipeline RAG hybride du projet

6.3.3 Réécriture de requête et gestion de la complexité

Le pipeline ne traite pas toutes les requêtes de la même manière. Il classe leur complexité et ajuste dynamiquement le *topK* ainsi que l'usage de la réécriture. Pour les requêtes complexes, la fonction `rewriteQueryWithTimeout()` ajoute une étape de reformulation avec timeout et fallback. Cela améliore la qualité de récupération sans rendre le système fragile en cas de lenteur du modèle.

Cette décision illustre une approche pragmatique de l'IA : améliorer la pertinence

quand cela apporte une vraie valeur, mais prévoir un repli déterministe lorsque le coût ou la latence deviennent trop élevés.

6.4 Réalisation

6.4.1 IA dans le détail d'une offre

Le composant `JobDetailClient` constitue l'un des premiers points où l'IA est exposée directement au recruteur. Il permet de générer un screening pour un candidat sur un poste, de consulter le score et les écarts, de préparer les questions d'entretien et d'orienter la suite du processus. L'IA intervient donc au moment de la décision, là où elle produit le plus de valeur métier.

Cette intégration directe évite de créer une zone IA déconnectée du reste de l'application. Les suggestions et scores apparaissent au moment où le recruteur doit réellement arbitrer.

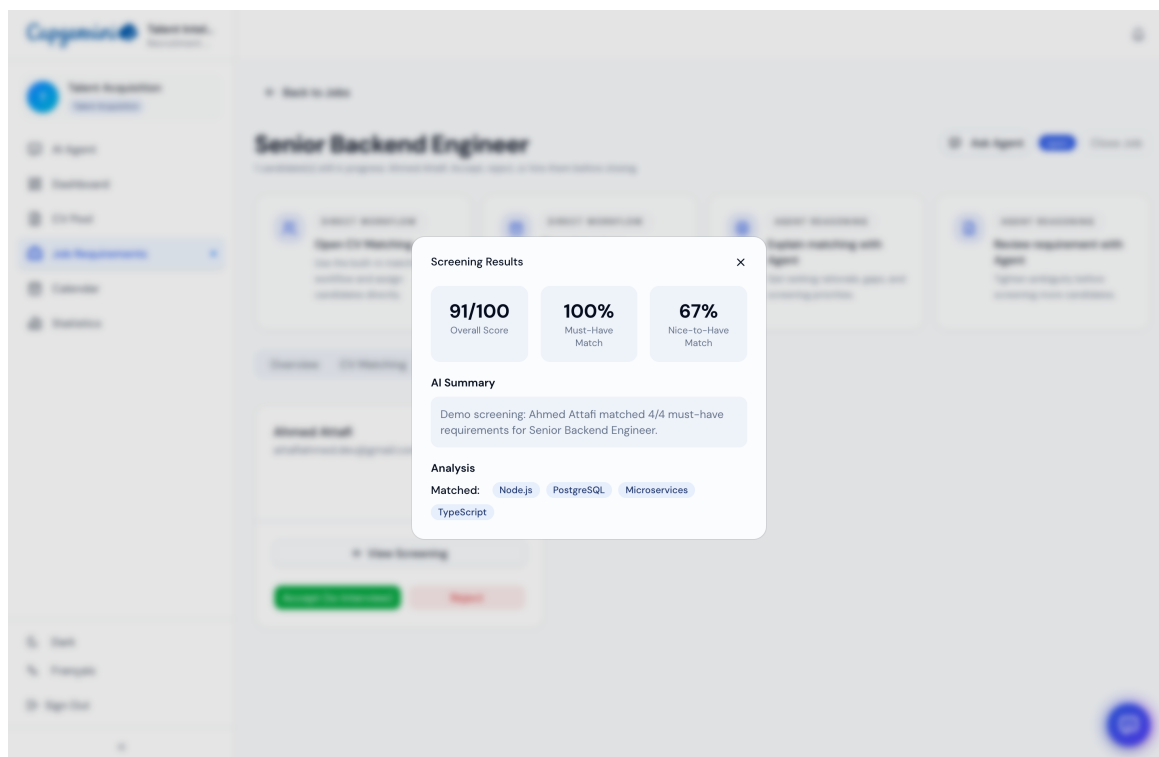


Figure 6.3 : Vue de score IA d'un candidat sur une offre.

6.4.2 Recherche sémantique et services IA avancés

Les services de matching permettent plusieurs stratégies de recherche : recherche explicite par critères, recherche sémantique sur profils complets, recherche hybride combinant ranking lexical et vectoriel et *bulk assignment* des meilleurs profils vers un poste.

Au-delà du matching et du RAG, `ai-features.ts` expose plusieurs services de valeur : comparaison de candidats, score prédictif de pipeline, résumé de profil, optimisation

d'exigences d'une offre, génération d'emails, questions de suivi et *insights* talent. Cette diversité prouve que la couche IA du projet n'est pas mono-usage.

Elle montre également que l'IA est traitée comme une boîte à outils métier, et non comme un unique point d'entrée censé tout résoudre seul.

Scénario end-to-end d'usage IA. Un scénario représentatif peut être résumé ainsi : un recruteur formule une question sur un profil ou sur un besoin de recrutement ; le système évalue la complexité de la requête ; il peut la réécrire ; il lance ensuite les récupérations lexicales et vectorielles nécessaires ; il fusionne et rerank les résultats ; puis, si la demande le nécessite, l'agent appelle un outil métier autorisé avant de produire une réponse grounded diffusée en SSE.

Cette chaîne montre que la valeur du module IA ne tient pas à une seule génération finale. Elle tient à l'enchaînement discipliné entre récupération d'information, raisonnement outillé, validation des entrées et contrôle des sorties. C'est précisément cette discipline qui rend l'IA compatible avec un contexte RH où l'explicabilité et la maîtrise des accès priment sur la simple fluidité conversationnelle.

6.4.3 Autopilot d'entretien

Les composants `InterviewAutoPilotGuideView` et `interview-autopilot-guide.tsx` exposent les guides générés pour les étapes TA, Manager et HR. Cette fonctionnalité aide à homogénéiser la qualité des entretiens et à mieux exploiter les informations déjà présentes dans le CV, le poste et l'historique du pipeline.

L'IA n'est donc pas utilisée seulement pour retrouver ou classer, mais aussi pour préparer une interaction humaine plus pertinente avec le candidat.

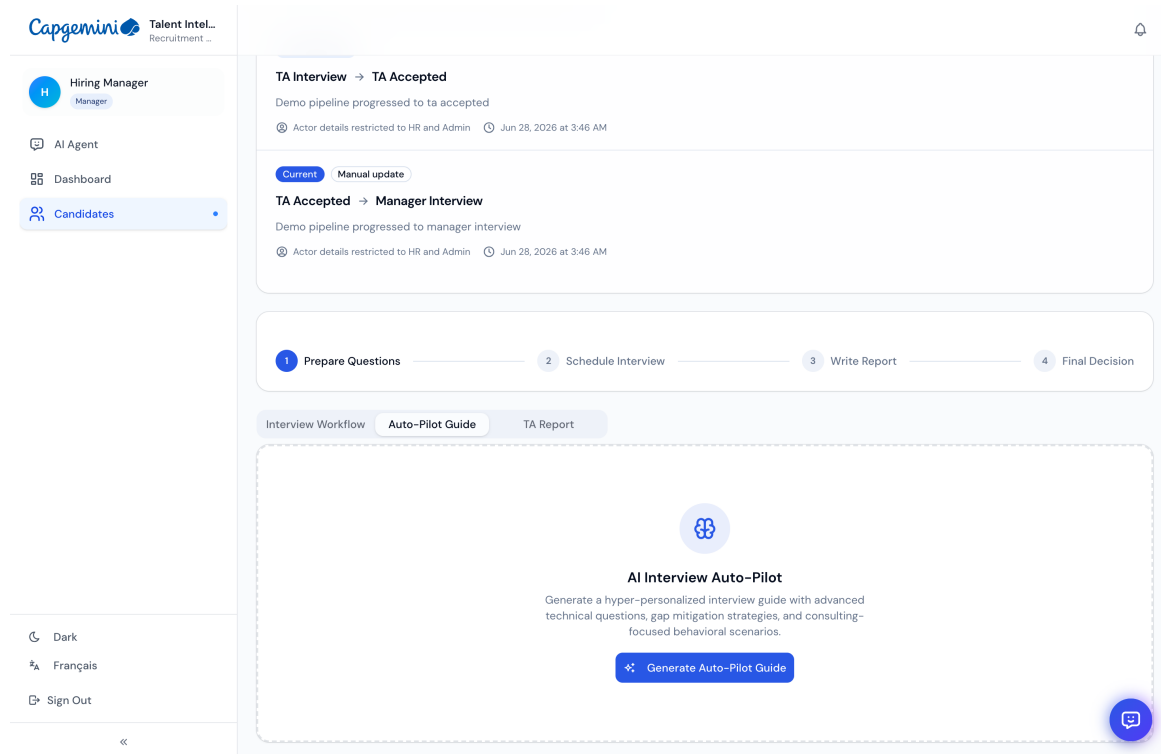


Figure 6.4 : Interface d'Auto-Pilot d'entretien.

6.4.4 Page statistiques, workspace agentique et réponses sourcées

La page statistique du recruteur charge les statistiques du vivier, les statistiques des jobs et les *smart insights*. Le composant **StatisticsChat** complète cette vue par une interaction conversationnelle SSE, mais il n'est plus seul : la navigation expose aussi une page / **agent** plein écran conçue pour les analyses longues, la consultation de l'historique et les traces détaillées. Cette couche conversationnelle n'est pas décorative. Elle ouvre un accès agentique aux données du système, avec mémoire conversationnelle, outils filtrés par rôle, exécution traçable et prompts contextuels injectés depuis les pages métier. Les boutons *Ask Agent* disséminés dans le vivier, les offres, les fiches Manager, les fiches RH et les surfaces Admin réutilisent les données du contexte courant pour préremplir la demande. Du point de vue métier, l'apport du sprint est concret. La recherche sémantique réduit l'effort de repérage des profils proches, le RAG améliore la justification des réponses complexes, et le chat agentique réduit le coût de navigation entre plusieurs écrans spécialisés. Le recruteur ne gagne donc pas seulement en rapidité ; il gagne aussi en lisibilité sur les raisons qui conduisent à une recommandation, un classement ou une synthèse.

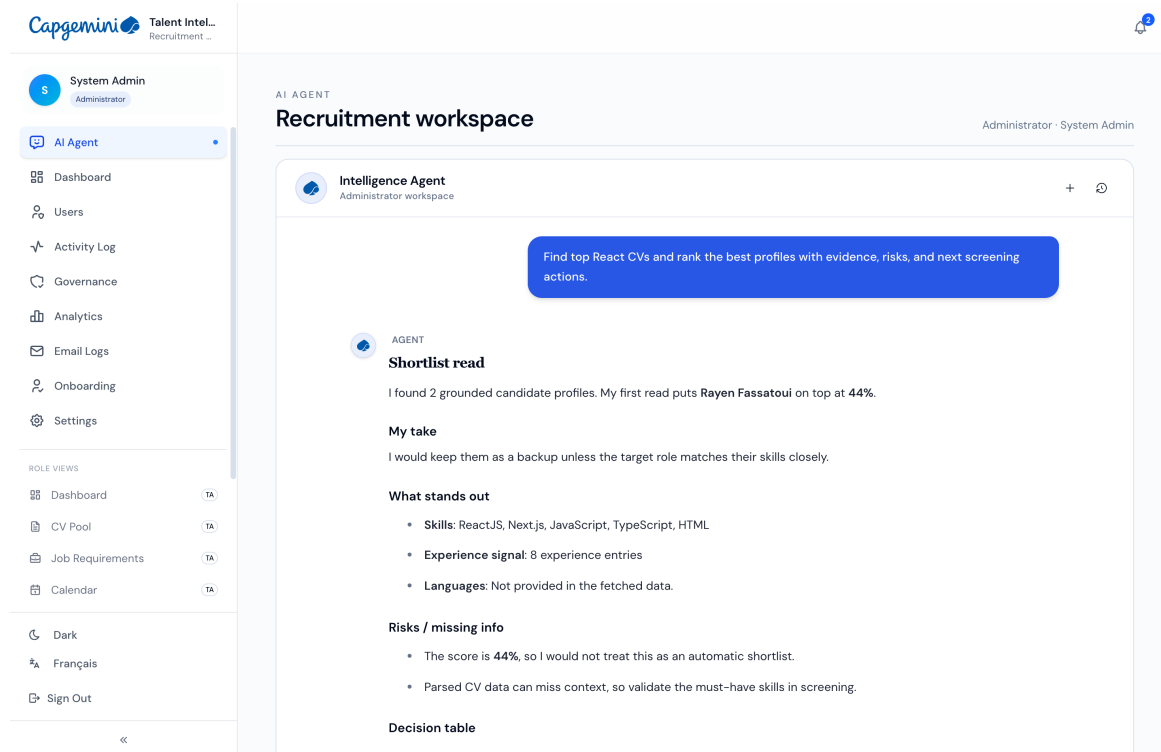


Figure 6.5 : Workspace agentique et réponse sourcée dans le chat.

6.4.5 Fonctionnement détaillé du chat analytique

Le chat du projet n'est pas un simple champ de texte connecté à un LLM. Il constitue une interface de pilotage qui permet à l'utilisateur autorisé de poser des questions métier directement sur l'état réel de la plateforme. Un recruteur peut par exemple demander un résumé du vivier, interroger la répartition des jobs, rechercher des candidats correspondant à un besoin précis ou demander une comparaison appuyée sur les données disponibles. Son fonctionnement repose sur plusieurs briques coordonnées :

- l'authentification de l'utilisateur avant tout échange ;
- la récupération de l'historique conversationnel, en retirant les marqueurs techniques avant de les réinjecter dans le contexte du modèle ;
- la sélection dynamique des outils accessibles selon le rôle ;
- l'appel à des services métier réels plutôt qu'à une génération libre ;
- la mise en attente des outils modifiant les données jusqu'à confirmation explicite ;
- la diffusion progressive de la réponse en SSE ;
- l'enregistrement persistant des conversations et messages en base ;
- l'émission de métadonnées de preuve : sources, blocs d'évidence, garde-fous de grounding, liens profonds vers les écrans métier, graphiques analytiques sérialisés, cartes typées et événements de trace outillée.

Cette architecture est importante pour le rapport car elle montre que le chat n'est pas un composant isolé. Il est branché sur le cœur applicatif et réutilise les mêmes règles de sécurité, les mêmes services métier et les mêmes contraintes de traçabilité que le reste de la plateforme. L'interface finale privilégie désormais une réponse directement exploitable par un recruteur : *Candidate read*, *My take*, *What stands out*, *Risks / missing info* et *Next Steps*. Les détails de preuve restent visibles dans le panneau source-backed, mais ils ne remplacent plus la conclusion métier. Du point de vue utilisateur, cela change profondément la manière d'accéder au système. Au lieu de naviguer manuellement entre plusieurs pages, l'utilisateur peut exprimer une intention en langage naturel, puis laisser l'agent interroger les bons outils pour reconstruire une réponse structurée, sourcée et re-navigable. Le chat devient ainsi une surcouche d'accès aux fonctionnalités, et non un gadget conversationnel déconnecté du produit.

La dernière itération améliore aussi la lisibilité de cette couche. Les sorties d'outils ne restent pas uniquement dans un journal technique : elles peuvent être transformées en cartes de réponse typées. Une carte candidat expose par exemple le fit, les compétences, les écarts et un lien vers le CV ; une carte pipeline résume les compteurs et les goulets d'étranglement ; une carte gouvernance met en avant les lignes d'audit, les acteurs et les événements à risque. En parallèle, le composant de trace regroupe les outils en phases *Planning*, *Retrieval*, *Analysis*, *Confirmation*, *Execution* et *Verification*. Cette organisation rend le raisonnement agentique plus compréhensible pour l'utilisateur métier.

Restitution visuelle des analyses. Une amélioration importante concerne les questions analytiques. Lorsque les outils retournent des données de pipeline, de tendances d'upload, de compétences, de langues, de jobs ou d'écarts demande-offre, le backend construit des objets graphiques typés à partir des résultats validés. Ces objets sont transmis dans les métadonnées du flux et persistés avec le message sous forme d'événements internes. Côté interface, `ChatAnalyticsChart` les rend sous forme de courbes, barres ou barres comparatives, sans demander au modèle de dessiner lui-même un graphique.

Mode de génération des figures IA. Les graphiques produits par l'assistant ne sont pas des images libres dessinées par le modèle. Le modèle peut contribuer à l'interprétation, mais la figure elle-même est générée par le backend à partir d'objets typés : source de données, série, libellés, valeurs numériques, type de graphique et liens de preuve. Cette séparation réduit le risque d'hallucination visuelle : l'IA ne dessine pas une courbe inventée, elle déclenche ou commente une restitution construite à partir des données validées du système. Dans le mémoire, les diagrammes d'architecture sont également générés comme figures vectorielles $\text{\LaTeX}$ /TikZ, ce qui les rend reproductibles dans Overleaf et évite les problèmes de résolution liés à des captures bitmap.

Table 6.4 : Restitutions visuelles générées par le chat analytique

Type de question	Source de données	Restitution attendue
Tendance du vrier	Statistiques CV	Courbe des uploads de CV sur la fenêtre récente.
Pipeline candidat	Insights ou dashboard	Barres par étape de recrutement.
Compétences et langues	Agrégats CV et jobs	Classement visuel des compétences ou langues les plus représentées.
Écart demande-offre	Analyse des skills gaps	Barres comparatives entre demande côté jobs et supply côté CV.
Réponse candidat	Matching, recherche ou comparaison	Carte typée avec score, compétences, écarts, lien CV et action de suivi.
Lecture pipeline	Dashboard ou insights	Carte typée avec compteurs, phase dominante et recommandation d'action.
Audit gouvernance	Logs et confirmations IA	Carte typée avec volume d'audit, événements à risque et lien vers les vues admin.
Trace outillée	Événements SSE de tools	Timeline par phases : récupération, analyse, confirmation, exécution et vérification.

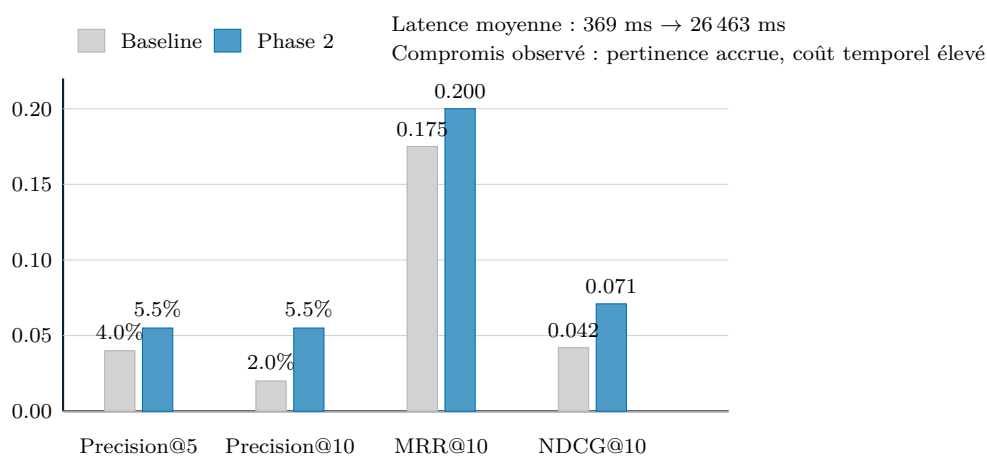
Cette évolution complète la logique de grounding : les graphiques et les cartes ne sont pas des illustrations décoratives générées librement, mais une autre forme de restitution des mêmes faits observés. Elle répond aussi à un besoin d'usage très concret : pour une question d'analytics, une courbe ou une barre comparative est souvent plus lisible qu'un paragraphe, tandis qu'une carte typée rend plus immédiatement visibles les métriques, les risques et les actions disponibles.

6.5 Évaluation et résultats

Le projet contient une preuve quantitative d'amélioration du pipeline RAG dans le rapport d'évaluation RAG de phase 2. L'évaluation couvre 40 requêtes avec une couverture *ground truth* de 85%. Les résultats comparatifs entre le baseline et la phase 2 sont les suivants.

Table 6.5 : Résultats de l'évaluation RAG phase 2

Métrique	Baseline	Phase 2	Variation
Precision@5	4.0%	5.5%	+37.5%
Precision@10	2.0%	5.5%	+175.0%
MRR@10	0.175	0.200	+14.3%
NDCG@10	0.042	0.071	+68.5%
Error rate	0.0%	0.0%	0.0%
Latence moyenne	369 ms	26463 ms	+7076.9%

**Figure 6.6** : Visualisation des métriques RAG et du compromis de latence

Comme le montrent le tableau 6.5 et la figure 6.6, la phase 2 améliore les métriques de pertinence mais rend visible un coût temporel majeur. Cette représentation graphique évite de présenter l'IA comme un gain univoque : elle montre simultanément l'amélioration de qualité et le prix opérationnel de cette amélioration.

Lecture des métriques. Les métriques utilisées ne mesurent pas toutes la même dimension. *Precision@5* et *Precision@10* indiquent la proportion de résultats pertinents dans les premiers éléments retournés. *MRR@10* évalue la position du premier résultat pertinent, ce qui est important pour l'expérience recruteur. *NDCG@10* prend en compte l'ordre des résultats pertinents dans le classement. Enfin, la latence moyenne mesure le coût opérationnel de la chaîne de retrieval. Ces définitions permettent de lire les résultats comme un compromis entre pertinence et temps de réponse, et non comme un score unique.

Les valeurs obtenues restent modestes en niveau absolu, mais elles sont utiles méthodologiquement parce qu'elles comparent deux versions du pipeline sur le même protocole. Le résultat doit donc être interprété comme une amélioration expérimentale documentée, pas comme une preuve que le retrieval est définitivement optimisé.

L'interprétation est claire : la phase 2 améliore nettement la pertinence des résultats, mais au prix d'une augmentation marquée de la latence. Ce compromis est important

à documenter car il montre une démarche d'ingénierie réaliste : la qualité du retrieval a d'abord été renforcée, quitte à reporter une optimisation fine des performances à une étape ultérieure.

Cette mesure a une forte valeur méthodologique dans le rapport. Elle prouve que le projet ne se contente pas d'affirmer que l'IA fonctionne mieux ; il cherche à objectiver les gains et à rendre visibles les coûts associés.

En complément de cette évaluation retrieval, le projet contient désormais une suite d'évaluation agentique déterministe. Elle vérifie les politiques de prompt anti-hallucination, le blocage des demandes créatives hors périmètre, le grounding des classements candidats, la confirmation obligatoire des mutations, la génération des cartes candidat/pipeline/gouvernance, le masquage des traces sensibles et le regroupement de la timeline d'outils. L'état de validation observé est détaillé dans les artefacts du projet : 123 tests passent sur la suite Vitest complète.

6.5.1 Analyse de complexité, coût et pistes d'optimisation

L'étude expérimentale doit être complétée par une lecture de complexité, car un pipeline IA utile en démonstration peut devenir difficile à exploiter si son coût croît trop rapidement avec le volume de CV, de chunks ou d'appels modèle. Dans ce projet, les principaux coûts proviennent de quatre sources : génération d'embeddings, recherche vectorielle, réécriture/reranking LLM et boucle agentique multi-outils.

Table 6.6 : Analyse de complexité et de coût du sous-système IA

Opération	Complexité dominante	Coût ou risque	Optimisation retenue ou proposée
Embedding CV et chunks	Proportionnelle au nombre de textes indexés et à la dimension vectorielle $d = 1024$.	Coût API et stockage d'environ 1024 valeurs par embedding, soit environ 4 Ko hors métadonnées si valeurs flottantes 32 bits.	Traitement asynchrone après upload, batch embeddings, cache et réindexation ciblée.
Recherche vectorielle	Dépend du nombre de chunks N , de la dimension d et du type d'index.	Temps de recherche et mémoire augmentent avec le corpus.	pgvector, seuil de similarité, topK limité et invalidation par version d'index.
Recherche lexicale	Dépend de la taille de l'index textuel et du nombre de résultats candidats.	Peut manquer les synonymes, mais reste rapide pour termes exacts.	Index <code>tsvector</code> , extraction de mots-clés et fusion avec le signal vectoriel.
Fusion RRF et reranking	RRF est linéaire dans les listes récupérées ; le reranking dépend du nombre final de candidats.	Gain de pertinence au prix d'un traitement supplémentaire.	Limitation des listes vectorielle et lexicale, constante RRF et finalTopK maîtrisé.
Réécriture de requête	Un appel LLM supplémentaire pour les requêtes complexes.	Latence et coût financier variables.	Déclenchement conditionnel, timeout, circuit breaker et fallback déterministe.
Agent multi-outils	Dépend du nombre d'étapes, des appels d'outils et de la génération finale.	Risque de lenteur, d'échec d'outil ou de mutation excessive.	Limites d'étapes, confirmations, seuil d'échecs, streaming SSE et trace par phases.

Le tableau 6.6 montre que l'optimisation prioritaire ne consiste pas à supprimer les étapes avancées, mais à mieux les déclencher. Les pistes les plus pertinentes sont donc :

mettre en cache les embeddings et résultats de retrieval, réserver la réécriture aux requêtes ambiguës, diminuer le *topK* quand la question est simple, réutiliser les résultats récents dans une même conversation, mesurer séparément latence vectorielle, latence lexicale, latence de reranking et latence de génération, puis envisager un fine-tuning léger uniquement sur des tâches stables comme la classification d'intention ou la normalisation des exigences.

6.6 Apport du module

Ce module apporte la dimension la plus distinctive du projet :

- il transforme les données de recrutement en base sémantique exploitable ;
- il introduit une vraie recherche augmentée, et pas seulement un moteur de filtres ;
- il assiste les recruteurs dans le scoring, le matching et la préparation des entretiens ;
- il expose ces capacités à travers un agent multi-outils gouverné ;
- il transforme les réponses analytiques et métier en graphiques ou cartes typées lorsque la visualisation apporte plus de valeur qu'un texte seul ;
- il rend la trace d'exécution plus lisible grâce à un regroupement par phases ;
- il ajoute des garde-fous explicites contre les dérives de l'IA ;
- il empêche les actions IA modifiant les données d'être exécutées sans confirmation humaine ;
- il fournit déjà des mesures quantitatives pour évaluer les gains et les compromis ;
- il ajoute une suite d'évaluation agentique qui vérifie les garde-fous et la restitution d'interface du chat.

Dans l'économie générale du rapport, ce module correspond au moment où la plateforme devient pleinement une plateforme IA de recrutement, et non une application métier à laquelle on aurait ajouté une couche cosmétique d'intelligence artificielle.

Il faut surtout retenir que la valeur du module vient d'une intégration profonde : données enrichies, recherche hybride, outils métier, garde-fous, persistance des résultats, restitution visuelle ou typée des analyses, trace par phases et mesure de la performance.

Conclusion

La couche IA de Capgemini Talent Intelligence est à la fois large et outillée : elle intervient sur les documents, sur les décisions et sur l'accès conversationnel au système. Surtout, elle est encadrée par des règles de validation, de sécurité et de mesure. Le dernier chapitre présentera la gouvernance opérationnelle qui complète cette intelligence par des mécanismes de supervision, de notification et d'administration.

Chapitre 7

Module 5 : notifications, analytique, administration et gouvernance

Introduction

Le dernier module referme la boucle fonctionnelle de la plateforme. Après avoir construit l'accès, le vivier documentaire, le pipeline candidat et la couche IA, il restait à renforcer la gouvernance globale du système. Une plateforme de recrutement d'entreprise ne peut pas se limiter à traiter des candidatures ; elle doit aussi fournir de la visibilité, des traces d'audit, des mécanismes de communication, des exports et un suivi après embauche.

Ce module ne correspond donc pas à un simple bloc de notifications. Il consolide la dimension administrable, pilotable et auditable de Capgemini Talent Intelligence, là où la supervision, la traçabilité et la restitution des données deviennent aussi importantes que les fonctionnalités de recrutement elles-mêmes.

Il montre ainsi comment le produit passe d'un système intelligent à un système gouvernable. Les données, décisions et automatisations produites précédemment doivent désormais pouvoir être suivies, expliquées, contrôlées et exportées.

7.1 Sprint 5 : objectif et périmètre

7.1.1 Objectif du sprint

L'objectif est de donner aux utilisateurs et aux administrateurs les moyens de superviser le système, de suivre les événements importants et de prolonger le recrutement jusqu'à l'intégration effective du candidat recruté. Plus précisément, ce sprint vise à :

- notifier les acteurs au bon moment ;
- conserver la mémoire des actions, des emails et des changements de statut ;
- fournir aux administrateurs une lecture consolidée de la plateforme ;
- offrir des exports exploitables pour le suivi et l'audit ;

- prolonger le cycle de vie du recrutement jusqu'à l'onboarding.

Il répond à une contrainte réaliste des outils d'entreprise : une bonne automatisation n'a de valeur durable que si elle laisse des preuves, des historiques et des points de pilotage lisibles pour les équipes responsables.

7.1.2 Backlog du sprint

Table 7.1 : Backlog fonctionnel du module gouvernance et administration

ID	Thème	User Story	Statut
33	Notifications	En tant qu'utilisateur connecté, je peux recevoir et consulter des notifications in-app.	Réalisé
34	Rappels d'entretien	En tant qu'utilisateur concerné, je peux recevoir des rappels d'entretien du jour.	Réalisé
35	Logs d'emails	En tant qu'administrateur, je peux consulter l'historique des emails envoyés.	Réalisé
36	Journal d'activité	En tant qu'administrateur, je peux auditer les actions effectuées sur la plateforme.	Réalisé
37	Vue système	En tant qu'administrateur, je peux consulter un aperçu global de la plateforme.	Réalisé
38	Analytics RH	En tant qu'administrateur, je peux analyser les performances du recrutement et la santé du pipeline.	Réalisé
39	Exports	En tant qu'administrateur, je peux exporter les données clés en Excel ou ZIP.	Réalisé
40	Onboarding détaillé	En tant qu'administrateur, je peux suivre l'avancement d'intégration des candidats embauchés.	Réalisé
41	Centre de gouvernance	En tant qu'administrateur, je peux filtrer et exporter les transitions candidat, les confirmations IA et les logs d'activité depuis une même vue.	Réalisé
42	Durcissement release	En tant qu'équipe projet, je peux vérifier que les migrations critiques et les comportements d'audit sont présents avant livraison.	Réalisé
43	Qualité agentique	En tant qu'équipe projet, nous pouvons valider les cartes de réponse, les traces d'outils, les confirmations et le masquage des données sensibles.	Réalisé
44	Protection des données	En tant qu'organisation, je dois encadrer le traitement des CV, des décisions et des traces selon les principes RGPD.	Réalisé
45	Accessibilité et UX	En tant qu'utilisateur, je dois pouvoir utiliser les parcours clés sur différents écrans, au clavier et avec des repères lisibles.	Réalisé

Ce backlog confirme que la gouvernance n'est pas un ajout marginal. Elle couvre la communication, l'audit, l'analytique, l'export, les confirmations IA, le centre de gouvernance et la continuité post-recrutement.

7.2 Implémentation du sprint

7.2.1 Analyse du sprint

Ce sprint s'appuie principalement sur les éléments fonctionnels suivants :

- le service de notifications in-app ;
- le service de communications sortantes et de journalisation des emails ;
- le service de vues agrégées administrateur ;
- le service de traçabilité métier ;
- le service d'export des données ;
- le service de checklists d'intégration ;
- le service de rapport d'audit gouvernance ;
- l'interface de notification ;
- la vue système administrateur ;
- la vue analytique administrateur ;
- la vue du journal d'activité ;
- la vue des logs d'emails ;
- la vue de suivi détaillé de l'intégration ;
- le centre de gouvernance administrateur ;
- les tests de qualité agentique couvrant confirmations, preuves, cartes et traces ;
- les règles de protection des données personnelles et de limitation des risques liés aux CV ;
- les validations UX et accessibilité transverses, notamment responsive design, navigation clavier, thème et internationalisation.

Ce sprint crée donc une couche de gouvernance transversale qui s'appuie sur tout ce qui a été produit auparavant : candidats, entretiens, décisions, emails, IA et onboarding.

Cette transversalité montre que la supervision n'est pas plaquée après coup. Elle réutilise les mêmes données opérationnelles pour produire une lecture de pilotage et une capacité d'audit.

7.2.2 Vue fonctionnelle globale

Les usages couverts se regroupent en cinq blocs :

1. **Communication et alertes** : créer des notifications, les récupérer, compter les éléments non lus, marquer une notification comme lue, tout marquer comme lu et envoyer ou tracer les emails.
2. **Supervision administrateur** : consulter un aperçu global du système, analyser les métriques du recrutement et suivre l'activité transversale de la plateforme.
3. **Restitution et audit** : conserver un journal enrichi des actions, un historique des emails et exporter les données clés au format Excel ou ZIP.
4. **Continuité post-recrutement** : suivre les checklists d'onboarding, visualiser l'état d'intégration des candidats `hired` et prolonger le pilotage au-delà de la décision d'embauche.
5. **Gouvernance des actions sensibles** : rapprocher les transitions candidat, les confirmations d'actions IA et les logs d'activité dans une vue filtrable et exportable.
6. **Protection des données et décisions IA** : encadrer les consentements, la conservation, les accès, les traces, les exports et la responsabilité humaine dans les recommandations.
7. **Accessibilité et expérience utilisateur** : garantir une navigation exploitable sur mobile et desktop, au clavier, avec des libellés compréhensibles, un thème clair/sombre et une structure lisible.

Cette structuration illustre la maturité atteinte par le produit : autour des entités métier centrales s'ajoutent désormais des objets de preuve, de communication et de suivi.

7.2.3 Chaîne de notification et de communication

Le service `notifications.ts` gère les notifications en base via la table `notification` s. Le système peut créer une notification ciblée, récupérer les dernières notifications d'un utilisateur, calculer le nombre non lu, marquer une notification comme lue et tout marquer comme lu.

Deux cas métier sont particulièrement importants : `notifyStageChange()` pour les changements d'étape d'un candidat et `notifyInterviewScheduled()` pour les événements liés à la planification d'entretien. En complément, `ensureTodayInterviewReminders()` vérifie les entretiens du jour afin de générer les rappels nécessaires.

Le service `email.ts` ne se contente pas d'envoyer des messages. Il trace aussi ces envois dans `email_logs`, ce qui garantit qu'une communication importante du recrutement reste auditée et restituable.

Cette couche améliore la réactivité du produit et réduit le risque qu’une étape importante reste invisible, en particulier dans un processus où plusieurs rôles interviennent sur des moments différents.

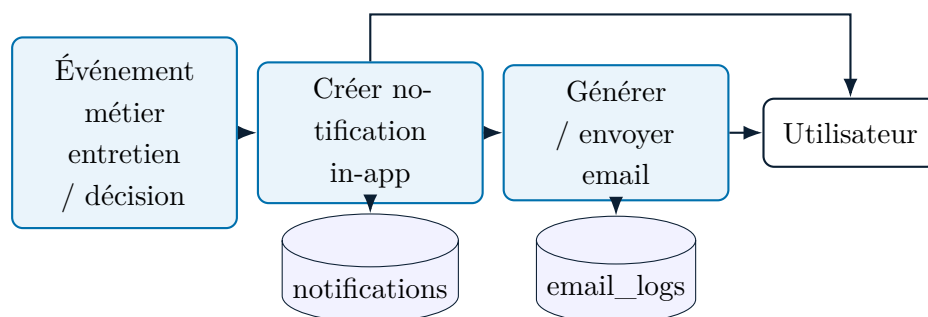


Figure 7.1 : Chaîne de notification et de journalisation des emails

7.2.4 Supervision administrateur

La couche `admin.ts` agrège plusieurs vues de haut niveau : `getSystemOverview()` calcule une vision consolidée du système, `getRecruitmentAnalytics()` fournit des métriques de performance et de répartition, `getEmailLogs()` expose l'historique des emails et `getHiredCandidatesOnboardingDetailed()` restitue l'état détaillé de l'onboarding. Cette agrégation montre que la plateforme ne sépare pas l'opérationnel du décisionnel. Les données utilisées pour exécuter le recrutement sont aussi réutilisées pour le piloter. La version actuelle ajoute à ces surfaces des panneaux d'évidence administrateur et des entrées agentiques gouvernées. Chaque vue peut maintenant résumer les faits observés, signaler les données opérationnelles manquantes et proposer des actions *Agent* dédiées à la synthèse d'activité, à l'interprétation analytique, aux anomalies d'onboarding ou aux risques de communication. Cette réutilisation des mêmes données constitue un point fort du projet : elle évite de dupliquer la vérité métier dans un outil de reporting distinct et renforce la cohérence entre exécution, supervision et assistance à la décision. Le nouveau centre de gouvernance renforce cette lecture. Il ne se limite pas au journal d'activité classique : il agrège les transitions du pipeline, les actions IA en attente ou exécutées et les logs applicatifs, afin de fournir une surface de contrôle unique pour les événements sensibles.

Lecture de gouvernance et portée de supervision. Elle permet surtout de distinguer clairement deux niveaux de lecture. Le niveau opérationnel sert à faire avancer les dossiers candidats, à planifier des entretiens et à envoyer des communications. Le niveau de gouvernance sert, lui, à vérifier que ces actions ont bien été réalisées, à en mesurer les effets, à en conserver la preuve et à fournir aux responsables une vision consolidée du système.

7.2.5 Audit, logs et exports

Le service `activity-log.ts` enregistre les événements importants via `logActivity()`, puis les restitue en lecture simple ou enrichie. En parallèle, `export.ts` permet de générer

plusieurs artefacts : export du vivier de CV, export d'un CV individuel, export ZIP de plusieurs CV, export des candidats acceptés, export des logs d'emails, export du journal d'activité, export de l'onboarding et export Excel détaillé d'un workflow d'acceptation candidat.

Cette partie prouve que la plateforme est pensée pour la restitution, la preuve et l'audit, pas seulement pour l'exécution interactive. Les exports jouent un rôle d'interopérabilité : la donnée reste exploitable hors de l'interface, pour le reporting, le contrôle et la conformité. Dans un cadre d'entreprise, cette capacité d'export dépasse la simple commodité. Elle permet de transformer l'information applicative en matière de reporting, de contrôle, de conformité et de partage. Les exports constituent donc un pont entre la plateforme de recrutement et les besoins plus larges d'audit, de pilotage RH ou de restitution managériale.

7.2.6 Centre de gouvernance et audit consolidé

Le service `governance.ts` construit un rapport d'audit consolidé à partir de trois sources : `candidate_stage_history`, `pending_agent_actions` et `activity_logs`. Chaque ligne est normalisée avec un type, un statut, une action, un acteur, un candidat éventuel, un horodatage et un détail métier.

Cette consolidation répond à un besoin de contrôle que les vues séparées ne couvrent pas entièrement. Un administrateur peut filtrer les événements par période, acteur, candidat, action ou statut, puis exporter les lignes en CSV. Les arguments d'outils sont sanitisés avant restitution afin d'éviter d'exposer des données sensibles comme les tokens, les pièces jointes brutes ou les contenus volumineux.

Le centre de gouvernance joue donc le rôle d'un plan de contrôle source-backed. Il met au même niveau les changements d'étape, les actions agentiques sensibles et les logs applicatifs, ce qui facilite l'analyse d'incident, la revue d'une décision et la préparation d'un audit.

7.2.7 Protection des données personnelles et conformité RGPD

La plateforme manipule des données personnelles sensibles au sens opérationnel du recrutement : identité du candidat, coordonnées, CV, expériences, compétences, rapports d'entretien, décisions et communications. Le projet applique donc une logique de minimisation et de contrôle plutôt qu'une simple logique de stockage. Les informations extraites sont structurées pour les usages métier nécessaires, tandis que les contenus volumineux ou bruts ne sont pas exposés librement dans les réponses agentiques.

Le consentement et la finalité du traitement doivent être compris comme des prérequis organisationnels : les CV sont importés pour un processus de recrutement identifié, puis utilisés pour l'évaluation, le matching, la planification et la décision. Cette finalité limite les usages acceptables de l'IA. L'agent ne doit pas transformer les données candidates en support de génération libre ; il doit rester borné par les outils métier, les rôles et les résultats réellement accessibles.

La conservation des données doit être pilotée par une politique interne : durée de conservation des CV non retenus, durée de maintien des candidats recrutés, purge des pièces jointes obsolètes et archivage des traces nécessaires à l’audit. Dans l’état actuel, le produit fournit les bases techniques de cette gouvernance à travers les logs, les exports et les statuts de pipeline ; la définition exacte des durées relève de la politique RH de l’organisation.

Le risque principal concerne la présence de texte brut et de fichiers CV dans la base. Ce choix facilite l’extraction, la recherche et la justification, mais il impose des garde-fous : accès par rôle, filtrage des outils, masquage des données sensibles dans les traces, limitation des exports et audit des consultations ou modifications importantes. Les services de gouvernance et les sanitizers de l’agent réduisent l’exposition dans les réponses, mais la donnée source reste une donnée à protéger.

Table 7.2 : Mesures de protection des données personnelles

Risque	Mesure intégrée	Complément organisationnel attendu
Consentement insuffisant	Usage limité au recrutement et aux rôles autorisés	Formaliser l’information candidat et la base légale du traitement
Conservation excessive	Statuts, logs et exports permettant d’identifier les dossiers	Définir une durée de rétention et une procédure de purge
Accès non autorisé	RBAC, routes protégées, outils filtrés par rôle	Revue régulière des comptes et des rôles
Exposition du CV brut	Sanitization des traces agentiques et limitation des sorties	Chiffrement, sauvegardes contrôlées et politique d’export
Décision IA opaque	Grounding, sources, cartes typées et confirmation des mutations	Décision finale conservée par un humain responsable
Biais de sélection	Scores accompagnés de limites, écarts et preuves	Revue humaine, diversité des critères et suivi des indicateurs

La décision finale reste humaine. Les mécanismes de screening, de matching, de synthèse ou de recommandation assistent l’utilisateur, mais ne remplacent pas le jugement TA, Manager ou RH. Cette séparation human-in-the-loop est essentielle : l’IA peut accélérer la lecture et signaler des écarts, mais elle ne doit pas produire seule une décision d’embauche ou de rejet.

7.2.8 Accessibilité et validation UX

L’expérience utilisateur a été conçue pour rester exploitable par les quatre rôles principaux sur des écrans et contextes différents. Les pages métier utilisent une structure de

dashboard cohérente, des composants `shadcn/ui` pour les dialogues, boutons, champs, tableaux et sélecteurs, ainsi qu’une navigation latérale stable. Cette cohérence réduit l’effort d’apprentissage et améliore l’opérabilité du produit.

La validation responsive repose sur une lecture *mobile-first* des écrans : les listes, cartes, panneaux et tableaux doivent rester utilisables sur petit écran, tablette et desktop. Les parcours critiques à vérifier sont la connexion, l’upload CV, la consultation du vivier, la création d’offre, le détail candidat, la planification d’entretien, la réponse agentique et les vues administrateur.

Sur le plan clavier et lecteur d’écran, les éléments importants sont les champs labellisés, les boutons avec texte ou nom accessible, les dialogues qui conservent le focus, les états d’erreur compréhensibles et la présence d’un lien de saut vers le contenu principal. Le projet embarque aussi une logique de thème clair/sombre et d’internationalisation, ce qui améliore la lisibilité et l’adaptation au contexte utilisateur.

Table 7.3 : Validation UX et accessibilité

Axe	Preuve attendue	Critère d’acceptation
Responsive design	Captures 320 px, 768 px, 1024 px et 1440 px	Les parcours critiques restent lisibles et actionnables
Navigation clavier	Parcours Tab, Entrée, Échap sur formulaires et dialogues	Aucune action critique n’exige uniquement la souris
Lecteur d’écran	Libellés de champs, titres, boutons et états dynamiques	Les contrôles principaux sont annoncés de manière compréhensible
Thème sombre	Vérification des pages principales en mode clair et sombre	Contraste et hiérarchie restent suffisants
Internatio-nalisation	Textes partagés via les locales et navigation stable	Les libellés communs restent cohérents entre langues
Lien de saut	Accès direct au contenu depuis le clavier	La navigation répétitive peut être contournée

Cette validation ne prétend pas remplacer un audit WCAG complet outillé. Elle formalise cependant les contrôles nécessaires pour éviter les défauts majeurs : absence de focus visible, composants non atteignables au clavier, libellés ambigus, contraste insuffisant ou rupture d’usage sur mobile.

7.3 Réalisation

7.3.1 Cloche de notification et communication in-app

Le composant `notification-bell.tsx`, injecté dans `DashboardShell`, matérialise la couche de notification côté interface. Il permet de charger les notifications à l’ouverture du panneau, d’afficher le compteur d’éléments non lus, de marquer une notification individuellement et de tout marquer comme lu. Cette intégration rend la gouvernance visible dans l’expérience quotidienne.

Elle renforce aussi la dimension temps réel du produit : les événements importants remontent au plus près du point d’usage, au lieu de rester enfouis dans des vues administratives.

The screenshot displays the Capgemini Talent Intelligence Recruitment dashboard. On the left is a sidebar with navigation links: System Admin, AI Agent, Dashboard, Users, Activity Log, Governance, Analytics, Email Logs, Onboarding, Settings, and Role Views. The main content area shows an 'AGENT' section with a 'Shortlist read' status. It includes a summary of candidate profiles, a 'My take' section, 'What stands out' (Skills, Experience signal, Languages), 'Risks / missing info', and a 'Decision table'.

Decision table:

Rank	Name	Score	Key Skills	Experience	Languages
1	Rayen Fassatoui	44%	ReactJS, Nextjs, JavaScript, TypeScript, HTML	8 experience entries	Not provided
2	Mohamed Ahref Ben Abdallah	42%	HTML, SCSS, JavaScript, TypeScript, React	9 experience entries	Not provided

Below the table is an 'Evidence' section. On the right, a 'Notifications' panel is open, showing three items: 'Candidate moved to HR review', 'Interview scheduled today', and 'Agent action requires audit'. Each notification has a 'Mark all read' link and a 'Close' button.

Figure 7.2 : Cloche de notification et état de lecture.

7.3.2 Tableau de bord administrateur

La page de tableau de bord administrateur appelle `getSystemOverviewAction()` et alimente `AdminDashboardClient`. Cette page donne une vision d'ensemble sur l'état du produit avec un discours orienté supervision : volumes, activité et santé fonctionnelle du pipeline. Elle combine maintenant les cartes KPI avec un panneau *Governance snapshot* source-backed et plusieurs CTAs agentiques dédiés à la synthèse d'activité, à la revue des rôles, aux anomalies opérationnelles et au brief de gouvernance. C'est le point de contrôle macro du système, là où l'administrateur peut comprendre rapidement la situation globale de la plateforme puis basculer, si nécessaire, vers une analyse agentique plus approfondie.

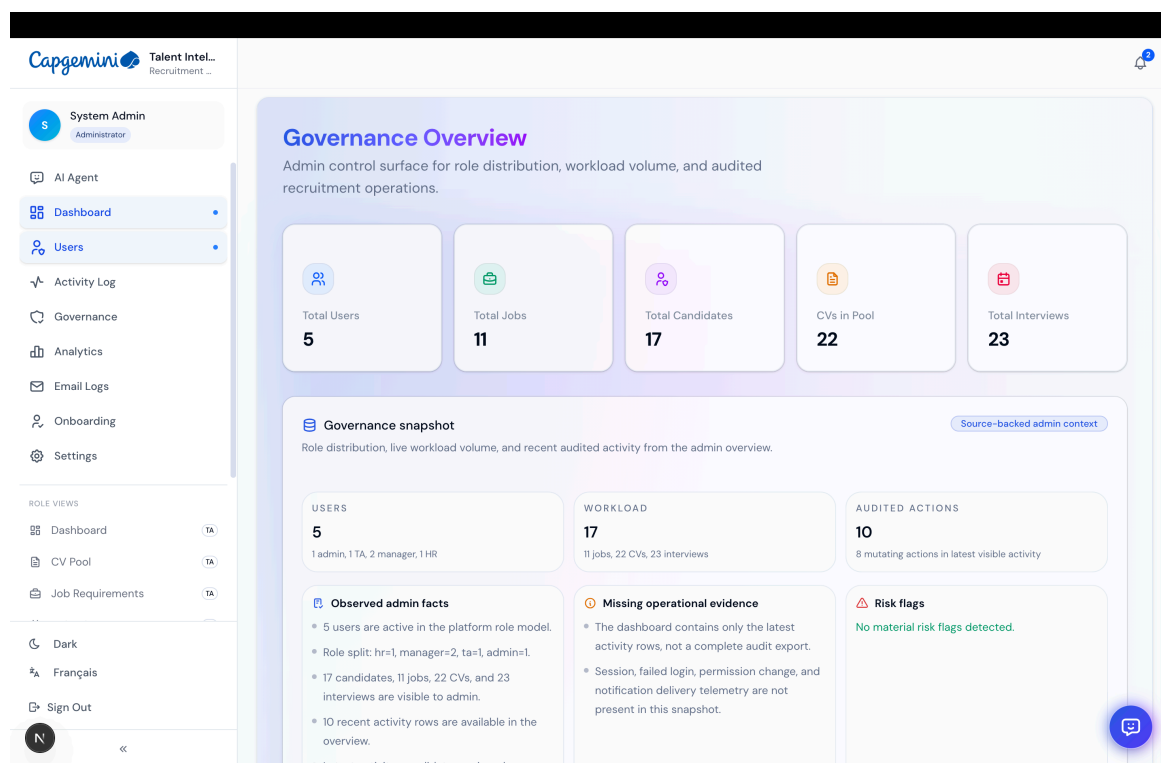


Figure 7.3 : Tableau de bord administrateur.

7.3.3 Analytique administrateur

La page analytique administrateur repose sur `getRecruitmentAnalyticsAction()` et sur `AdminAnalyticsClient`. Elle transforme les données opérationnelles en indicateurs lisibles : performance du pipeline, répartition des volumes, tendances de recrutement et signaux utiles au pilotage global. Elle ne se limite plus aux graphiques. Un panneau d'évidence dédié récapitule désormais les faits observés, les limites de mesure et les risques d'interprétation, tandis que des actions *Explain trends*, *Find bottlenecks* et *Governance risks* ouvrent l'agent avec un contexte déjà cadré. La restitution analytique combine ainsi deux niveaux complémentaires : des graphiques de page pour la supervision globale et des réponses agentiques capables de produire des visualisations ciblées lorsque l'utilisateur demande une courbe, une répartition ou une comparaison demande-offre. Elle donne ainsi à l'administration la possibilité de lire les flux de recrutement comme un ensemble dynamique, et non comme une simple accumulation de dossiers.

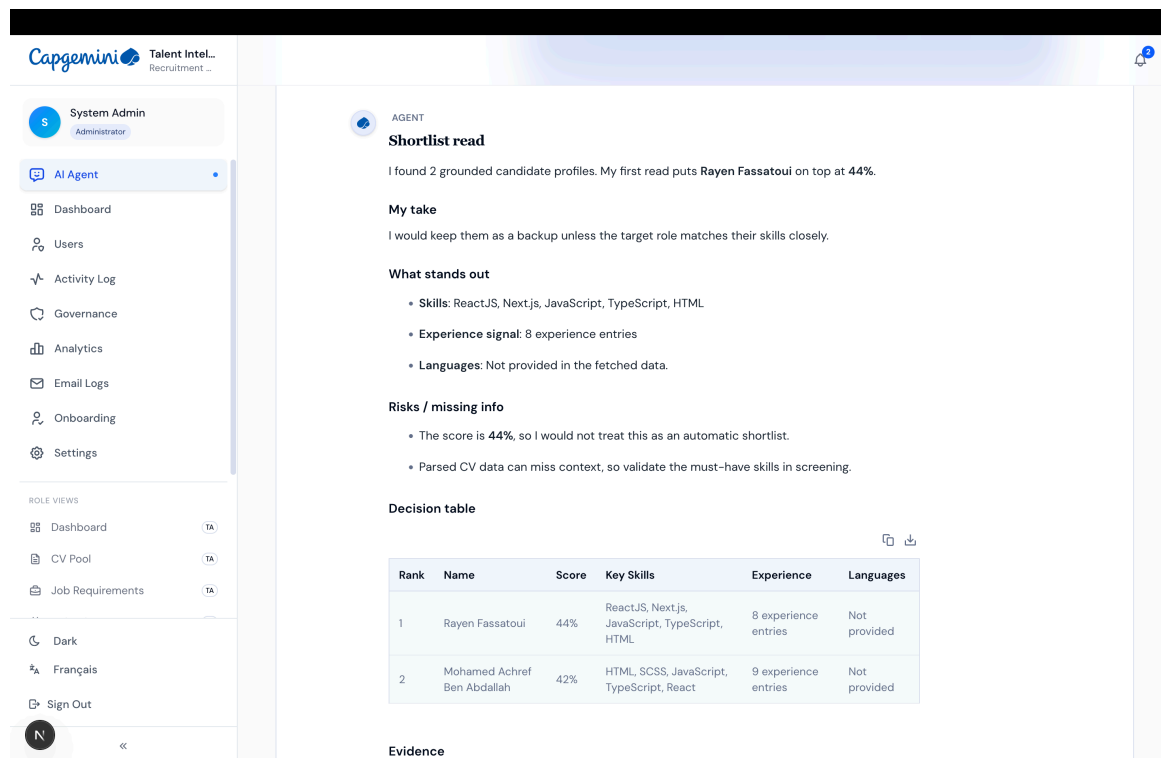


Figure 7.4 : Vue analytique administrateur.

7.3.4 Centre de gouvernance

La page `/admin/governance` charge `getGovernanceAuditReportAction()` puis affiche `AdminGovernanceClient`. Elle présente des compteurs dédiés aux lignes d'audit, aux changements d'étape, aux actions IA, aux logs d'activité, aux actions IA en attente et aux actions IA échouées.

L'interface permet de filtrer par date, acteur, candidat, action ou statut. Chaque ligne ouvre un panneau de détail contenant une preuve sanitisée, avec les arguments d'outil nettoyés lorsque l'événement provient de l'agent. L'administrateur peut aussi exporter le résultat courant en CSV via `exportGovernanceAuditCsvAction()`.

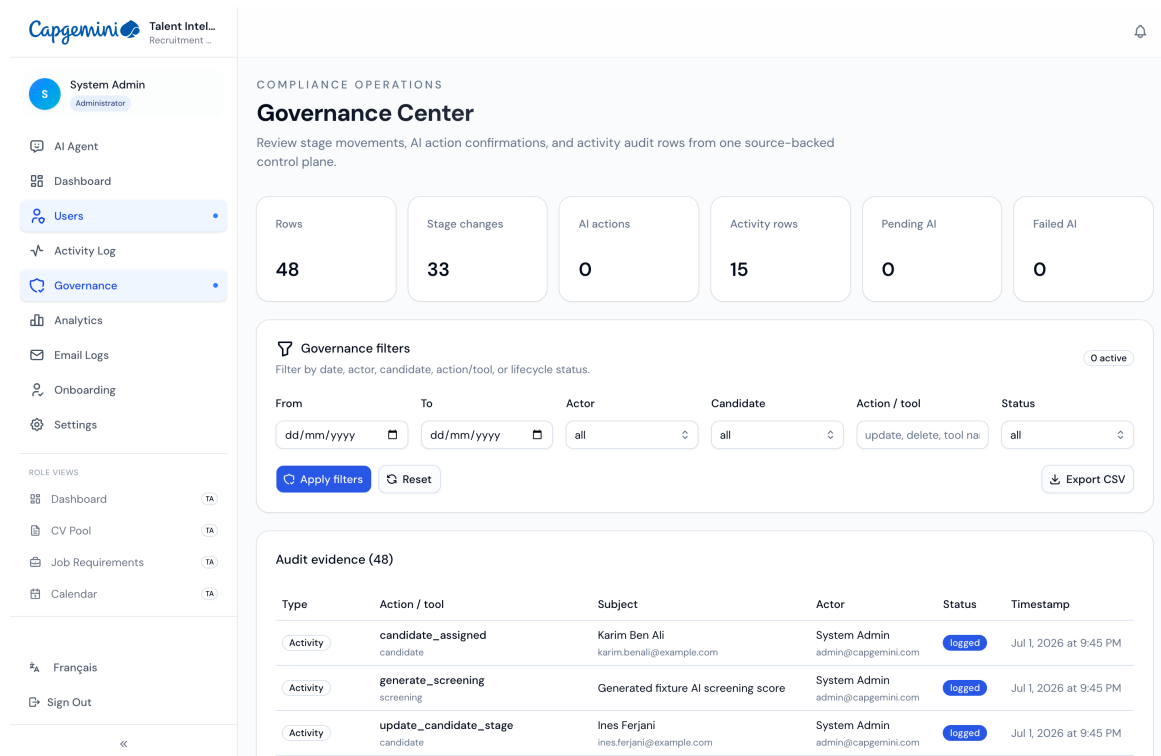


Figure 7.5 : Centre de gouvernance administrateur.

7.3.5 Historique d'activité et logs d'emails

La page du journal d'activité exploite `getActivityLogEnrichedAction(100)` et le composant `AdminActivityClient`. L'objectif est d'exposer une lecture enrichie du journal d'activité afin de reconstituer les séquences d'action importantes. Un panneau d'évidence et des actions *Agent* permettent en parallèle de synthétiser les lignes chargées, de signaler les actions destructrices et de rappeler les limites de couverture de l'audit courant. Cette fonctionnalité est centrale pour l'audit interne, l'analyse d'incident, la compréhension des décisions et la justification d'un enchaînement métier. Dans une plateforme de recrutement augmentée par l'IA, cette traçabilité a encore plus de valeur, car elle permet d'observer ce qui a été automatisé, ce qui a été décidé et dans quel ordre. Le journal d'activité ne sert donc pas seulement à voir ce qui s'est passé. Il sert aussi à rendre le système explicable lorsqu'une décision, une communication ou un changement de statut doit être revu.

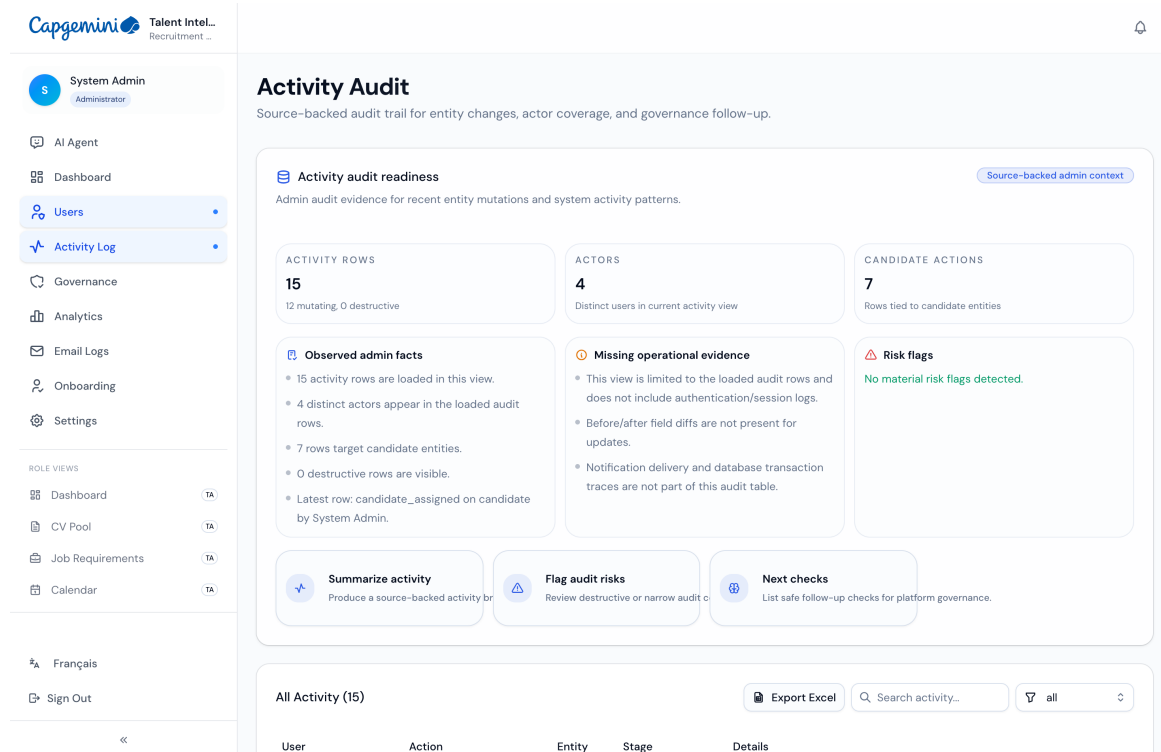


Figure 7.6 : Journal d'activité.

La page des logs d'emails charge `getEmailLogsAction()` puis transmet les données au composant `AdminEmailsClient`. Cette vue constitue l'*audit trail* des communications sortantes de la plateforme, mais elle bénéficie à présent des mêmes principes : panneau *Communication evidence readiness*, CTAs agentiques dédiés aux risques de livraison et liens profonds vers les entrées auditées. Elle est particulièrement importante dans le contexte RH, où les invitations, confirmations, refus et communications de décision doivent rester consultables et vérifiables. La consultation centralisée de ces logs évite qu'une partie sensible du processus reste cachée dans une boîte noire extérieure au produit. Cette séparation entre journal d'activité et logs d'emails clarifie la gouvernance : le premier explique les actions métier réalisées dans l'application, tandis que les seconds documentent les communications envoyées vers les candidats ou les parties prenantes.

7.3.6 Suivi détaillé de l'onboarding

La page de suivi détaillé de l'onboarding s'appuie sur `getHiredCandidatesOnboardingDetailedAction()` et `AdminOnboardingClient`. Elle ne montre pas seulement une liste de candidats embauchés ; elle expose aussi l'état détaillé des tâches d'intégration, ce qui prolonge le recrutement après la décision finale. Le tableau embarque désormais un panneau d'évidence consacré aux anomalies d'onboarding : checklists absentes, coordonnées incomplètes, profils pauvres en signaux et autres trous de couverture. Les CTAs *Find anomalies*, *Summarize readiness* et *Risk review* transmettent ensuite ces constats à l'agent sans réintroduire de génération libre. Ce prolongement vers l'onboarding est important pour la cohérence globale du produit : le cycle de vie géré par la plateforme ne s'arrête pas

à la validation finale, mais continue jusqu'au suivi de l'entrée effective dans l'organisation.

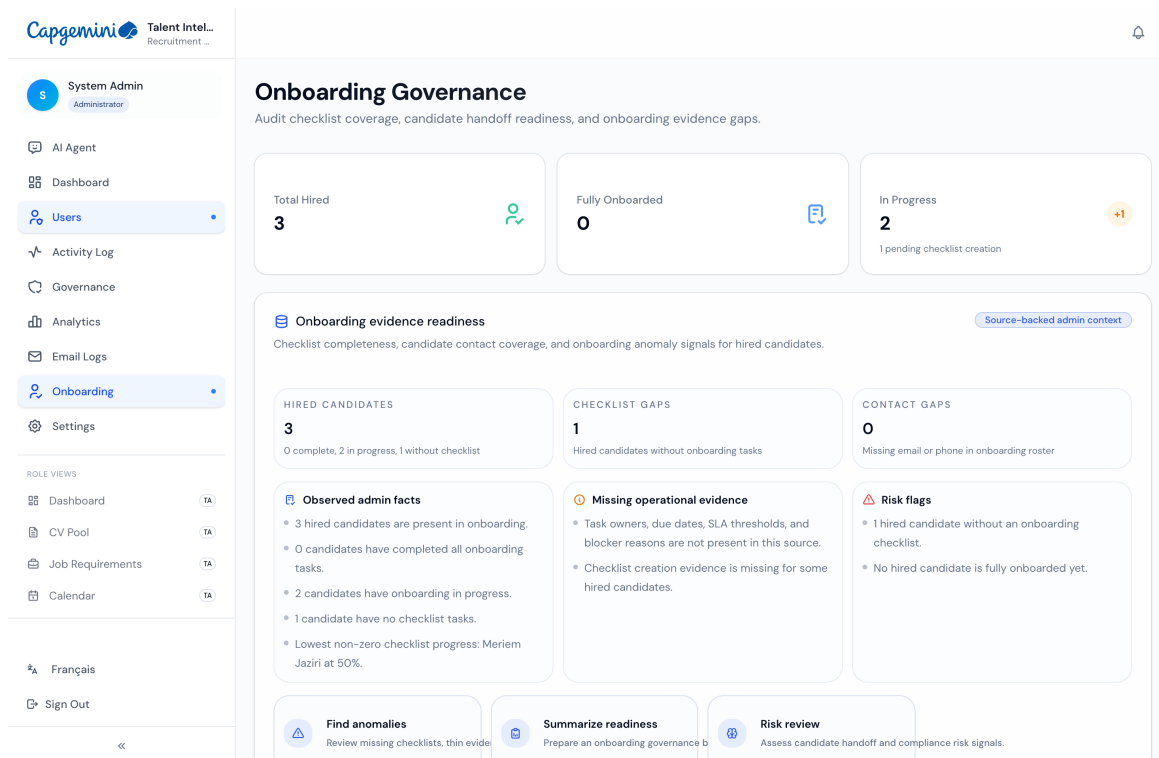


Figure 7.7 : Suivi de l'onboarding.

7.3.7 Exports et interopérabilité

Les actions d'export disponibles dans `actions.ts` et `export.ts` donnent à la plateforme une capacité d'interopérabilité concrète. Les données peuvent sortir sous des formats directement exploitables par les équipes métier, de contrôle ou d'audit. Le produit n'enferme donc pas l'information dans son interface.

Cette capacité d'export renforce la maturité du système : un produit d'entreprise crédible doit aussi pouvoir alimenter d'autres usages organisationnels hors de son interface principale.

7.3.8 Vérification de durcissement release

Le script `db/verify-release-hardening.ts` complète la gouvernance par une vérification technique ciblée. Il contrôle la présence des tables et index critiques liés à l'historique des étapes et aux confirmations IA, peut appliquer les migrations manquantes lorsque le schéma est absent, puis exécute un scénario comportemental temporaire.

Ce scénario crée un utilisateur de vérification, un poste, un CV et un candidat, valide que l'affectation et une transition manuelle produisent bien l'historique attendu, puis confirme une action IA qui fait progresser le candidat. Le script vérifie ensuite que l'action passe au statut `executed`, que la transition agentique est enregistrée, puis supprime les

lignes temporaires. Il ne s'agit donc pas d'un simple contrôle de schéma, mais d'une preuve de bout en bout sur les mécanismes de gouvernance les plus sensibles.

Cette logique est complétée par les tests automatisés de l'agent et de son interface. Ils vérifient que les actions sensibles restent en attente de confirmation, que les cartes candidat/pipeline/gouvernance proviennent de résultats d'outils, que les traces d'exécution sont classées par phases et que les champs sensibles sont masqués avant affichage ou stockage technique. Cette couche de vérification relie donc directement l'expérience utilisateur du chat aux exigences de gouvernance décrites dans ce chapitre.

Lecture d'audit. La gouvernance du module repose sur une séparation claire des traces. Les notifications servent à informer les utilisateurs au moment utile. Le journal d'activité permet de reconstituer les actions réalisées dans l'application. Les logs d'emails documentent les communications sortantes. Les exports rendent ces informations exploitables hors de l'interface. Cette séparation évite de mélanger alerte, preuve, communication et restitution, ce qui renforce la lisibilité du système en cas de contrôle ou de revue métier.

7.4 Apport du module

Ce module apporte la maturité de gouvernance du projet :

- il améliore la réactivité via les notifications et les rappels ;
- il améliore la traçabilité via les logs d'activité et d'emails ;
- il améliore le pilotage grâce aux dashboards et aux analytics administrateurs ;
- il améliore l'interopérabilité par les exports ;
- il prolonge la continuité métier jusqu'à l'onboarding ;
- il améliore la gouvernance des actions IA sensibles par une confirmation explicite et auditée ;
- il fournit un centre de gouvernance consolidé pour les transitions, confirmations et logs ;
- il renforce la qualité agentique par des tests couvrant confirmations, cartes typées, traces par phases et masquage des données sensibles ;
- il encadre les données personnelles par des principes de consentement, rétention, accès contrôlé, audit et décision humaine ;
- il améliore l'accessibilité et l'expérience utilisateur par des composants accessibles, une navigation cohérente, le support responsive, le thème clair/sombre et l'internationalisation.

Dans l'économie générale du rapport, ce module montre que l'intelligence et l'automatisation déployées dans les chapitres précédents sont encadrées par une couche de supervision explicite. C'est cette combinaison entre IA, métier et gouvernance qui donne à Capgemini Talent Intelligence sa crédibilité comme produit d'entreprise.

On peut donc considérer ce dernier module comme la couche de confiance et de pilotage finale du système. Il transforme la plateforme en produit non seulement fonctionnel et intelligent, mais aussi administrable, explicable et durable dans un contexte professionnel.

Conclusion

Ce cinquième module transforme la plateforme en système pilotable, gouvernable et auditable. Notifications, logs, dashboards administrateurs, analytics, centre de gouvernance, confirmations IA, exports et suivi d'onboarding complètent désormais les modules métier et IA déjà mis en place. Le produit ne se limite plus à exécuter un recrutement intelligent ; il permet aussi de le superviser, de l'expliquer et de le prolonger au-delà de la décision finale.

Conclusion générale

Le projet Capgemini Talent Intelligence a permis de concevoir une plateforme de recrutement d'entreprise cohérente, couvrant l'ensemble du cycle métier : centralisation des CV, structuration des offres, progression des candidats avec historique auditable, organisation des entretiens, notifications, analytique, gouvernance des actions IA et administration. Contrairement à une application limitée au suivi administratif, la solution rassemble dans un même environnement les opérations, les données et les mécanismes de décision qui structurent réellement le travail des équipes TA, Manager, HR et Admin.

Sur le plan technique, le projet montre la pertinence d'une architecture *feature-driven* pour un produit full-stack moderne. La séparation entre routes, actions serveur, services métier, composants d'interface et schéma de données favorise la lisibilité, la maintenabilité et l'évolution incrémentale. Cette organisation a également facilité l'intégration de briques avancées comme les embeddings, les chunks RAG, les guides d'entretien IA et l'agent conversationnel multi-outils, sans dénaturer le cœur métier de l'application.

La contribution la plus distinctive réside dans l'industrialisation raisonnée de l'intelligence artificielle. Le projet ne s'appuie pas sur un modèle unique utilisé de manière opaque ; il combine extraction structurée, matching, génération guidée, autopilot d'entretien, conversation outillée, garde-fous d'exécution et validation des sorties. L'évaluation comparative du pipeline RAG sur 40 requêtes, avec 85% de couverture de vérité terrain, montre un gain de pertinence mesurable : amélioration de Precision@5, Precision@10, MRR@10 et NDCG@10, tout en maintenant un taux d'erreur nul.

Validation fonctionnelle et technique

La validation retenue pour ce rapport combine deux niveaux complémentaires : d'une part une validation fonctionnelle par scénarios couvrant les principaux parcours utilisateurs du produit, et d'autre part une validation technique fondée sur les preuves automatisées documentées dans les artefacts du projet. Cette approche permet de ne pas limiter la démonstration à la seule couche IA, mais d'élargir la vérification au fonctionnement global de la plateforme.

Table 7.4 : Synthèse de validation fonctionnelle

Domaine	Scénario de validation	Résultat attendu	Résultat retenu
Accès et identité	Connexion puis accès à l'espace métier.	Redirection par rôle et blocage sans session valide.	Couvert par l'authentification, la redirection par rôle et le shell dashboard.
Vivier de CV	Import d'un CV PDF ou DOCX.	Extraction du texte, profil consultable et enrichissement lancé.	Couvert par l'upload, l'extraction, les doublons et l'indexation progressive.
Pipeline candidat	CV affecté puis transmis entre TA, Manager et RH.	Candidat créé, transitions historisées et responsabilités tracées.	Couvert par le workflow candidat, les affectations et l'historique d'étapes.
Entretiens et décisions	Entretien planifié puis formalisé par un rapport.	Entretien créé, étape mise à jour et onboarding ouvert si embauche.	Couvert par la planification, le reporting et la checklist d'onboarding.
Administration et gouvernance	Consultation des vues admin et du centre de gouvernance.	Supervision consolidée, traces d'audit, confirmations IA et exports.	Couvert par les vues admin, les logs, les exports et le centre de gouvernance.
Chat analytique	Question posée depuis le chat ou /agent.	Outils RBAC, streaming SSE, preuves, confirmations, graphiques, cartes typées et trace par phases.	Couvert par l'orchestration, le grounding, les confirmations, les cartes de réponse, les graphiques et la timeline d'outils.

Validation technique automatisée. Sur le plan technique, les preuves automatisées principales sont documentées dans les artefacts du projet. Dans l'état observé, `bunrunlint`, `bunrunbuild` et `bunruntest` passent ; la suite Vitest complète valide 123 tests après alignement du harnais `ChatInput` avec le contrat de propriétés du composant.

Les vérifications automatisées couvrent notamment :

- le grounding des noms candidats, avec rejet des noms absents des résultats d'outils, acceptation des noms autorisés et remplacement déterministe des classements hallucinés ;
- l'isolation de périmètre, avec filtrage des résultats pour les utilisateurs non administrateurs, prévention des fuites inter-utilisateurs et séparation des clés de cache selon l'utilisateur et le rôle ;
- le comportement du chat en cas d'absence de résultats exploitables, avec réponse de repli grounded au lieu d'une fabrication libre ;
- la confirmation des actions IA modifiant les données, depuis la création d'une action en attente jusqu'à son exécution, son annulation ou son expiration ;
- les transitions du pipeline candidat, avec validation des transitions autorisées, rejet des transitions incohérentes et formatage de la timeline ;
- la génération des métadonnées d'évidence, des liens profonds et des résumés de gouvernance administrateur ;
- la qualification des panneaux d'*evidence readiness* côté candidat et côté vivier documentaire ;

- la génération, la sérialisation et l'extraction des graphiques du chat analytique, afin que les réponses visuelles restent fondées sur des sorties d'outils validées ;
- la génération, la sérialisation et l'extraction des cartes de réponse typées du chat, afin que les réponses candidat, pipeline et gouvernance soient rendues à partir de sorties d'outils validées ;
- le regroupement de la trace d'outils par phases, avec classification en récupération, analyse, confirmation, exécution et vérification ;
- le centre de gouvernance administrateur, incluant l'affichage des lignes d'audit et l'ouverture de détails sanitisés.

Enfin, la couche de retrieval fait l'objet d'une validation quantitative spécifique déjà documentée dans le projet. Le rapport d'évaluation RAG de phase 2 couvre 40 requêtes avec 85% de couverture *ground truth* et montre une amélioration mesurable de Precision@5, Precision@10, MRR@10 et NDCG@10. Cette mesure complète utilement la validation fonctionnelle, car elle montre non seulement que le produit fonctionne, mais aussi que sa couche de recherche augmentée est évaluée de manière chiffrée.

Atteinte des objectifs initiaux

L'évaluation finale doit être reliée aux objectifs formulés au départ du projet. Le tableau 7.5 synthétise cette lecture : il ne s'agit pas seulement de vérifier que des écrans existent, mais de montrer que chaque objectif produit une preuve fonctionnelle, technique ou expérimentale.

Table 7.5 : Évaluation de l'atteinte des objectifs initiaux

Objectif initial	Preuve de réalisation	Limite identifiée	Statut
Centraliser les données RH	CV, offres, candidats, entretiens, notifications, logs et onboarding sont regroupés dans une base PostgreSQL commune.	La qualité dépend encore de la complétude des CV et des champs renseignés.	Atteint
Structurer le pipeline candidat	États explicites, affectations TA/Manager/HR, rapports et historique immuable des transitions.	Des scénarios end-to-end production seraient utiles pour mesurer les délais réels par étape.	Atteint
Accélérer le matching	Matching explicite, recherche sémantique, hybrid search et bulk assignment.	Le RAG avancé améliore la pertinence mais augmente fortement la latence.	Partiellement optimisé
Rendre l'IA explicable	Sources, grounding, cartes typées, trace outillée, confirmations et résultats RAG mesurés.	L'explicabilité doit être testée avec des utilisateurs métier réels.	Atteint
Gouverner les actions sensibles	RBAC, validation Zod, pending actions, logs d'activité, gouvernance admin et exports.	Une politique de rétention et d'audit long terme reste à définir selon les contraintes organisationnelles.	Atteint
Produire une base évolutive	Architecture feature-driven, services métier, composants réutilisables et tests automatisés.	Les modules IA devront être profilés avant passage à un volume de données plus important.	Atteint avec réserves

Cette lecture montre que les objectifs fonctionnels majeurs sont atteints, tandis que l’optimisation de performance et la validation métier en conditions réelles restent les deux axes les plus importants pour transformer le prototype avancé en solution industrialisée.

Discussion des résultats

Les résultats observés permettent de tirer plusieurs enseignements utiles au-delà du simple constat chiffré. D’abord, l’amélioration de la pertinence du pipeline RAG s’explique par le passage d’une récupération trop limitée à une récupération hybride plus riche. La combinaison entre réécriture de requête, recherche vectorielle, recherche lexicale, fusion par RRF, reranking et assemblage de contexte donne au système davantage de chances de retrouver des passages réellement alignés avec l’intention métier de la requête.

La chaîne de retrieval enrichie donne au système une lecture plus complète de la demande utilisateur. Chaque étape apporte un signal distinct : la reformulation clarifie l’intention, la récupération vectorielle capte les proximités sémantiques, la recherche lexicale préserve les termes exacts, la fusion réduit la dépendance à un seul score et le reranking réordonne les résultats selon leur utilité finale.

Les mécanismes de grounding jouent ici un rôle déterminant. Sans eux, le gain de fluidité conversationnelle pourrait se payer par des réponses plus séduisantes que fiables. Le grounding garantit que les noms candidats, les classements et les restitutions restent bornés par les résultats réellement accessibles à l’utilisateur. Dans un contexte RH, cette contrainte est essentielle, car une réponse plausible mais non fondée peut dégrader la confiance dans l’outil et produire des erreurs de décision.

Pour le recruteur, l’impact réel de ces résultats est double. D’un côté, la plateforme réduit l’effort de recherche, de comparaison et de préparation grâce à une récupération plus pertinente et à un accès conversationnel outillé. De l’autre, elle conserve une logique de contrôle compatible avec un usage professionnel : les réponses sont contextualisées, les outils sont filtrés par rôle et les sorties sensibles sont vérifiées. Le résultat n’est donc pas seulement une IA plus performante, mais une IA plus exploitable dans un processus de recrutement gouverné. L’ajout de visualisations directement dans les réponses agentiques renforce ce résultat côté usage. Les questions analytiques ne produisent plus uniquement une synthèse textuelle : lorsque les données s’y prêtent, elles affichent une courbe ou des barres qui rendent les tendances, les goulets d’étranglement et les écarts de compétences plus immédiatement compréhensibles. La confirmation explicite des actions modifiant les données complète cette logique. Elle évite qu’une réponse agentique transforme directement l’état du système sans validation humaine, tout en conservant une trace exploitable par le centre de gouvernance. Les cartes de réponse typées et la trace par phases prolongent la même logique d’explicabilité. Elles réduisent l’effort de lecture : les métriques, liens, limites et actions de suivi apparaissent sous forme structurée, tandis que le travail interne de l’agent est présenté comme une timeline métier plutôt que comme une liste brute d’appels techniques.

Vue d'ensemble des acquis

La vue d'ensemble du projet met en évidence une progression cohérente entre les objectifs initiaux, le Product Backlog et les sprints de réalisation. Le produit démarre par un accès sécurisé, construit ensuite ses objets métier principaux, organise le workflow candidat, ajoute une couche IA gouvernée puis termine par des capacités de supervision. Cette continuité donne au rapport une lecture claire : chaque module apporte une valeur immédiate tout en préparant les modules suivants.

Le résultat obtenu est une plateforme unifiée où les données RH ne sont plus dispersées entre des écrans isolés. Les CV, les offres, les candidats, les entretiens, les notifications, les logs et les analyses IA s'inscrivent dans un même cycle métier. Cette vue globale est essentielle, car elle montre que Capgemini Talent Intelligence n'est pas seulement une démonstration technique ; c'est un système orienté usage, pensé pour aider les équipes à suivre, comprendre et justifier leurs décisions.

Limites du projet

Malgré les résultats obtenus, plusieurs limites doivent être soulignées. La première concerne la latence du pipeline RAG avancé : l'amélioration de la pertinence s'accompagne d'un coût de calcul important lorsque la requête déclenche l'ensemble de la chaîne de retrieval enrichie. La deuxième limite tient à la dépendance envers des services externes de génération et d'embeddings, qui peuvent introduire des contraintes de disponibilité, de coût et de variabilité de performance.

Une autre limite concerne la qualité des données d'entrée. La valeur produite par le matching, le screening, le RAG et l'agent conversationnel dépend directement de la qualité des CV importés, de la cohérence des offres d'emploi et de la richesse des informations renseignées dans le pipeline candidat. Enfin, la validation automatisée observable couvre désormais le grounding, l'isolation de périmètre, les confirmations agentiques, les transitions candidat, les panneaux d'évidence, les cartes de réponse typées, la trace d'outils par phases, le centre de gouvernance et l'évaluation RAG ; elle gagnerait encore à être complétée par davantage de scénarios end-to-end métier exécutés dans un environnement proche production.

Menaces à la validité et risques éthiques

Dans un projet de recrutement augmenté par l'IA, la validation ne peut pas se limiter à la réussite technique des fonctionnalités. Les résultats doivent être lus avec prudence, car ils dépendent du corpus de CV, des requêtes choisies, du comportement des utilisateurs et du cadre éthique dans lequel l'outil est utilisé. Cette précaution est particulièrement importante dans le domaine RH, où une recommandation algorithmique peut influencer une décision humaine sensible. Les travaux sur les algorithmes de recrutement soulignent

d’ailleurs que les promesses de réduction du biais doivent être évaluées de manière critique et non acceptées comme un effet automatique de l’automatisation [26].

Table 7.6 : Menaces à la validité et mesures de maîtrise éthique

Dimension	Menace ou risque	Impact possible	Mesure de maîtrise
Validité interne	Les résultats RAG dépendent du jeu de données local, des chunks générés et des requêtes retenues.	Une amélioration mesurée peut refléter le corpus testé plus qu’une supériorité générale.	Comparaison baseline/phase 2 sur le même protocole, conservation des métriques et extension future du jeu de requêtes.
Validité externe	Le système n’est pas encore évalué dans un environnement de production avec plusieurs recruteurs réels.	Les gains observés peuvent différer selon les pratiques métier, les volumes et les profils traités.	Prévoir une expérimentation utilisateur avec TA, Manager et HR, puis mesurer temps, satisfaction et qualité perçue.
Validité de construction	Precision@K, MRR et NDCG mesurent la pertinence de retrieval, pas directement la qualité d’une décision RH.	Un bon score informationnel ne garantit pas une décision juste ou acceptable métier.	Compléter les métriques IR par feedback utilisateur, audit des décisions et indicateurs de qualité d’entretien.
Validité statistique	Le protocole couvre 40 requêtes avec 85% de vérité terrain.	La variance reste possible, surtout par catégorie de requête ou niveau de difficulté.	Augmenter le nombre de requêtes, équilibrer les catégories et documenter les intervalles de confiance.
Biais algorithmique	Les CV et historiques peuvent contenir des biais sociaux, linguistiques ou de parcours.	Le score IA pourrait reproduire ou amplifier des déséquilibres existants.	Conserver une décision humaine, afficher les critères, auditer les écarts et éviter les décisions automatiques finales.
Hallucination et opacité	Un agent génératif peut citer un candidat absent ou formuler une justification non fondée.	Perte de confiance et risque de décision sur information incorrecte.	Grounding des candidats, citations, sources, cartes typées, trace d’outils et fallback lorsque les données sont insuffisantes.
Données personnelles	Les CV contiennent des informations sensibles ou indirectement identifiantes.	Risque de fuite, de consultation hors rôle ou de conservation excessive.	RBAC, séparation des rôles, journaux d’activité, minimisation des sorties et politique future de rétention.
Sur-confiance utilisateur	Un score ou une synthèse IA peut être perçu comme plus objectif qu’il ne l’est réellement.	L’utilisateur peut déléguer trop vite son jugement au système.	Présenter les limites, les gaps, les preuves et les recommandations comme aide à la décision, jamais comme décision automatique.

Le tableau 7.6 montre que la contribution du projet ne réside pas seulement dans l’ajout de fonctions IA, mais aussi dans leur encadrement. Les mécanismes de grounding, de confirmation, de RBAC, de logs et d’explication ne sont donc pas secondaires : ils conditionnent la possibilité d’utiliser l’IA dans un contexte RH sans transformer l’assistance en automatisation opaque.

Perspectives

Plusieurs prolongements peuvent être envisagés. La première perspective consiste à améliorer les métriques et la mesure de complexité : augmenter le nombre de requêtes d’évaluation, distinguer les métriques par type de besoin métier, suivre la latence par étape du

pipeline, mesurer le coût financier par requête et intégrer des indicateurs de satisfaction utilisateur. Cette évolution permettrait de comparer objectivement plusieurs stratégies : SQL seul, vectoriel seul, RAG hybride, RAG avec reranking et agent multi-outils.

La deuxième perspective concerne l'optimisation du pipeline RAG. Les pistes prioritaires sont la réduction du *topK* lorsque la requête est simple, le cache des embeddings et des résultats, l'activation conditionnelle de la réécriture, la mesure séparée du coût de reranking et l'étude d'index vectoriels plus adaptés aux volumes importants. Un fine-tuning ciblé peut aussi être envisagé, non pour mémoriser les CV, mais pour stabiliser des tâches répétitives : classification d'intention, extraction d'exigences, reformulation de requêtes ou normalisation des compétences.

La troisième perspective porte sur le recrutement prédictif. À partir d'un historique suffisant et gouverné, la plateforme pourrait estimer les risques de blocage dans le pipeline, prédire les délais de passage entre étapes, recommander les profils avec une probabilité de réussite contextualisée et détecter les écarts entre demande métier et disponibilité du vivier. Cette évolution devra cependant rester explicable et contrôlée, car une prédiction RH non justifiée peut introduire des biais de décision.

Enfin, l'intégration de modèles multimodaux représente une perspective de recherche intéressante. Elle permettrait d'exploiter non seulement le texte des CV et offres, mais aussi des documents scannés, portfolios, certificats, tableaux ou supports d'entretien. Cette ouverture doit être accompagnée de règles de gouvernance renforcées : consentement, minimisation des données, filtrage des informations sensibles et séparation claire entre aide à l'analyse et décision humaine.

Apports personnels

Au regard des réalisations présentées dans ce mémoire, les apports personnels du travail mené peuvent être synthétisés autour de plusieurs axes complémentaires. Le premier apport réside dans la structuration d'une architecture cohérente, capable d'articuler authentification, services métier, persistance, interfaces multi-rôles et composants IA dans un même produit full-stack.

Le deuxième apport concerne l'intégration d'une couche d'intelligence artificielle non cosmétique, directement reliée aux données du système : extraction de CV, matching, pipeline RAG hybride, génération de guides, screening et chat analytique outillé. Le troisième apport porte sur la formalisation du workflow de recrutement lui-même, avec une progression explicite des candidats, une répartition claire des responsabilités entre TA, Manager et RH, un historique immuable des transitions, ainsi qu'une continuité jusqu'à l'onboarding et à la gouvernance. Enfin, ce travail a permis de relier l'implémentation technique à une logique de contrôle, grâce à la validation des entrées, au filtrage RBAC, au grounding des réponses, à la confirmation des actions IA sensibles et à la traçabilité des actions critiques.

La dernière amélioration d'interface renforce cet apport en transformant le chat agentique en surface de décision plus lisible : les cartes typées structurent les résultats, la

timeline par phases rend les appels d'outils auditable et la suite de tests agentiques stabilise ces comportements.

Cette progression confirme que le travail réalisé couvre les dimensions principales attendues d'une plateforme de recrutement augmentée : sécurité d'accès, centralisation des données, workflow opérationnel, assistance IA, validation quantitative et gouvernance. L'ensemble forme un produit cohérent, documenté et aligné avec les besoins des rôles TA, Manager, HR et Admin.

En définitive, Capgemini Talent Intelligence constitue une base solide pour un système de recrutement intelligent orienté entreprise. Le projet démontre qu'il est possible d'intégrer l'IA de manière utile, gouvernée et mesurable, à condition de l'ancrer dans les données du métier, dans des workflows explicites et dans une architecture technique disciplinée.

Bibliographie et webographie

Sources institutionnelles et métier.

1. Dossier technique du projet Capgemini Talent Intelligence : documentation d'architecture, schéma de données, services métier, routage applicatif, scripts de validation et rapport d'évaluation RAG. Consultation locale des artefacts du projet.
2. Capgemini, *About us*. <https://www.capgemini.com/about-us/>. Dernier accès : 23 mai 2026.
3. Capgemini, *Capgemini Engineering*. <https://www.capgemini.com/about-us/who-we-are/our-brands/capgemini-engineering/>. Dernier accès : 23 mai 2026.
4. LinkedIn Talent Solutions. <https://business.linkedin.com/talent-solutions>. Dernier accès : 23 mai 2026.
5. Greenhouse Recruiting. <https://www.greenhouse.com/recruiting>. Dernier accès : 23 mai 2026.
6. Workday Recruiting. <https://www.workday.com/en-us/products/human-capital-management/recruiting.html>. Dernier accès : 23 mai 2026.

Documentation technique officielle.

7. Next.js Documentation. <https://nextjs.org/docs>. Dernier accès : 23 mai 2026.
8. React Documentation. <https://react.dev/>. Dernier accès : 23 mai 2026.
9. TypeScript Documentation. <https://www.typescriptlang.org/docs/>. Dernier accès : 23 mai 2026.
10. Bun Documentation. <https://bun.sh/docs>. Dernier accès : 23 mai 2026.
11. Tailwind CSS Documentation. <https://tailwindcss.com/docs>. Dernier accès : 23 mai 2026.
12. shadcn/ui Documentation. <https://ui.shadcn.com/docs>. Dernier accès : 23 mai 2026.
13. Better Auth Documentation. <https://www.better-auth.com/docs>. Dernier accès : 23 mai 2026.

14. Drizzle ORM Documentation. <https://orm.drizzle.team/docs/overview>. Dernier accès : 23 mai 2026.
15. Neon Documentation. <https://neon.com/docs>. Dernier accès : 23 mai 2026.
16. PostgreSQL Documentation. <https://www.postgresql.org/docs/>. Dernier accès : 23 mai 2026.
17. pgvector Documentation. <https://github.com/pgvector/pgvector>. Dernier accès : 23 mai 2026.
18. Zod Documentation. <https://zod.dev/>. Dernier accès : 23 mai 2026.
19. Vitest Documentation. <https://vitest.dev/guide/>. Dernier accès : 23 mai 2026.

Références scientifiques.

20. Patrick Lewis, Ethan Perez, Aleksandra Piktus et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*, Advances in Neural Information Processing Systems 33, NeurIPS 2020. <https://papers.nips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>. Dernier accès : 4 juillet 2026.
21. Jeff Johnson, Matthijs Douze et Hervé Jégou, *Billion-scale similarity search with GPUs*, 2017. <https://arxiv.org/abs/1702.08734>. Dernier accès : 23 mai 2026.
22. *A Survey of Retrieval-Augmented Generation (RAG) for Large Language Models*, IEEE Xplore, document 11300411. <https://ieeexplore.ieee.org/document/11300411>. Dernier accès : 4 juillet 2026.
23. Stephen E. Robertson et Hugo Zaragoza, *The Probabilistic Relevance Framework : BM25 and Beyond*, Foundations and Trends in Information Retrieval, 3(4), 333–389, 2009. <https://doi.org/10.1561/15000000019>. Dernier accès : 4 juillet 2026.
24. Gordon V. Cormack, Charles L. A. Clarke et Stefan Büttcher, *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*, Proceedings of the 32nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, 758–759, ACM, 2009. <https://doi.org/10.1145/1571941.1572114>. Dernier accès : 4 juillet 2026.
25. Shiqiang Guo, Folami Alamudun et Tracy Hammond, *Résumé-Job Matching System*, Expert Systems with Applications, 60, 169–182, Elsevier, 2016. <https://doi.org/10.1016/j.eswa.2016.04.013>. Dernier accès : 4 juillet 2026.
26. Manish Raghavan, Solon Barocas, Jon Kleinberg et Karen Levy, *Mitigating Bias in Algorithmic Hiring : Evaluating Claims and Practices*, Proceedings of the 2020 ACM Conference on Fairness, Accountability, and Transparency, ACM FAccT 2020. <https://doi.org/10.1145/3351095.3372828>. Dernier accès : 4 juillet 2026.

27. Shunyu Yao, Jeffrey Zhao, Dian Yu et al., *ReAct : Synergizing Reasoning and Acting in Language Models*, International Conference on Learning Representations, 2023. https://openreview.net/forum?id=WE_vluYUL-X. Dernier accès : 4 juillet 2026.
28. Timo Schick, Jane Dwivedi-Yu, Roberto Dessì et al., *Toolformer : Language Models Can Teach Themselves to Use Tools*, Advances in Neural Information Processing Systems 36, NeurIPS 2023. <https://proceedings.neurips.cc/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html>. Dernier accès : 4 juillet 2026.
29. Lei Wang, Chen Ma, Xueyang Feng et al., *A Survey on Large Language Model Based Autonomous Agents*, Frontiers of Computer Science, 18, article 186345, Springer, 2024. <https://doi.org/10.1007/s11704-024-40231-1>. Dernier accès : 4 juillet 2026.